Sprint 3 — Pipeline ETL

Relatório explicativo do código e da conduta: o que cada parte faz e por quê

Projeto Integrador II · Opção D · Inteligência Tributária de Palmas

BIINAR/UFT · situação em 21/10/2026

Apresentação

INTRODUÇÃO — O TODO

Este relatório explica a Sprint 3 até o menor elemento do código e da conduta, e o faz segundo um método de escrita declarado. O método foi escolhido porque um texto que apenas relata o que aconteceu não permite ao leitor compreender por que aconteceu, e sem o porquê nenhuma decisão pode ser avaliada, defendida ou corrigida. Todo método de escrita é analítico ou sintético: a análise decompõe um todo em suas partes, do mais complexo ao menos complexo, e a síntese encontra o padrão que une as partes. O relatório usa os dois, em lugares definidos, e as subseções a seguir explicam como.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A introdução de cada seção apresenta o todo que a seção vai examinar. Ela nomeia o sujeito da seção, que pode ser um módulo, uma etapa ou uma regra de conduta, diz para que ele existe e anuncia as partes em que será decomposto. Essa apresentação vem primeiro porque o leitor só consegue situar uma parte quando já conhece o todo a que ela pertence.

O desenvolvimento de cada seção é analítico e decompõe o todo em partes, do mais geral ao mais específico. Cada parte recebe um subtítulo, e cada parágrafo trata de uma única parte. A ordem das partes segue a ordem em que o código as executa ou a ordem em que uma depende da outra, porque essa é a ordem em que a lógica se constrói.

Cada parágrafo mantém um sujeito único e explícito do começo ao fim. O tópico frasal, impresso em negrito, nomeia esse sujeito e afirma o que ele é ou faz; os períodos seguintes explicam por que ele existe, como funciona e que consequência tem, sempre retomando o mesmo sujeito pelo nome ou por uma referência inequívoca. A última frase de cada parágrafo fecha a ideia com uma conclusão parcial. Essa estrutura existe para que a lógica não se perca: o leitor sabe, em cada frase, de quem se está falando e por que aquilo importa.

A conclusão de cada seção é sintética e encontra o padrão comum às partes analisadas. Ela não repete o desenvolvimento; ela responde à pergunta sobre o que todas as partes têm em

comum e que garantia esse padrão oferece ao projeto. A síntese vem por último porque só pode ser feita depois que cada parte foi compreendida.

O documento tem três partes, e cada uma aplica esse método a um nível diferente. A Parte I trata da conduta da equipe: objetivos, decisões, regras, método, defeitos e pendências. A Parte II percorre o código módulo por módulo e função por função, na ordem em que o dado atravessa o pipeline. A Parte III trata da verificação: testes, quality gates, execução de referência e o que a releitura do código encontrou.

Os números do relatório vêm de fontes que qualquer pessoa pode conferir. São o próprio código, o log da execução de referência e o relatório de qualidade gravado pelo pipeline; um único número, a distorção de área da projeção, foi medido especialmente para este relatório. Nomes de arquivos, funções e variáveis aparecem em fonte monoespaçada.

CONCLUSÃO — SÍNTESE

O método do relatório reproduz, portanto, o próprio movimento da compreensão: primeiro o todo, depois cada parte com a sua razão de ser, por fim o padrão que as une. Quem lê uma seção inteira sabe o que aquela parte do pipeline faz, por que foi construída assim e que garantia ela oferece ao conjunto.

Sumário

Parte I — A conduta da Sprint 3 . . . . . **5**

1. Objetivo e entregas da Sprint 3 5
2. Decisões que orientaram o trabalho 7
3. Regras de conduta 8
4. Método de trabalho 10
5. Tratamento de defeitos e transparência 11
6. Situação e pendências 12

Parte II — O código, etapa por etapa **14**

7. Visão geral da arquitetura 14
8. Ponto de entrada: etl_sefin.py 16
9. Orquestrador 18
10. Domínio: constantes e erros 20
11. Observabilidade: log e identificador 22
12. Camadas, manifesto e depósito do bruto 24
13. Política de coleta 27
14. Consulta ao robots.txt 29
15. Cliente HTTP 30
16. Transporte HTTP 34
17. Interface das fontes e paginação 36
18. Adaptadores das APIs paginadas 38
19. Adaptadores de arquivos 40
20. Catálogo e execução da coleta 43
21. Leitura do bruto 45
22. Fundamentos da limpeza 47
23. Limpeza da arrecadação da API 49
24. Limpeza da extração NUCLEOGOV 52
25. Limpeza das geometrias 54
26. Limpeza do Censo e do IPCA 56
27. Enriquecimento da receita 58
28. Enriquecimento territorial 62
29. Parâmetros da simulação 64
30. Gerador da simulação 67
31. Atributos da base analítica 71
32. Validações de qualidade 73

33. Encadeamento da transformação . . . 76
34. Gravação das camadas e formato dos arquivos . . . 78
35. Operações geométricas . . . 81
36. Registro da execução no banco . . . 82
37. Carga no MySQL . . . 83
38. Esquema SQL da Sprint 3 . . . 87
39. EDA: leitura, estatísticas e estilo . . . 90
40. EDA: gráficos e mapas . . . 94
41. EDA: seções e relatório . . . 96
42. Referências e conferência no Crossref . . . 99

Parte III — Verificação e resultados . . . 101

43. Testes e mini-Palmas . . . 101
44. Quality gates, mutação e integração contínua . . . 103
45. Execução de referência . . . 106
46. Proposta HU-04 fora do pipeline . . . 108
47. Observações da releitura do código . . . 109
48. Conclusão . . . 111

Referências . . . 113

Parte I — A conduta da Sprint 3

1. Objetivo e entregas da Sprint 3

INTRODUÇÃO — O TODO

A Sprint 3 teve um único objetivo: fazer o dado público de Palmas chegar sozinho, íntegro e documentado da fonte até o banco de dados construído na Sprint 2. Esse objetivo existe porque a Sprint 2 entregou o banco modelado e um carregador para a extração da receita, mas não um fluxo que fosse buscar os dados nas fontes, conferi-los e prepará-los. Sem esse fluxo, cada atualização dependeria de uma pessoa repetir à mão passos que ninguém consegue auditar depois. A Sprint 3, portanto, não acrescenta ao projeto um novo tipo de análise; ela constrói a máquina que abastece qualquer análise futura, inclusive o modelo de recuperabilidade da Sprint 4. Para organizar esse todo, o enunciado dividiu o trabalho em quatro entregas, de E1 a E4, e as tarefas do Módulo 5 acrescentaram exigências de limpeza, indicadores, figuras e painel, que o desenvolvimento a seguir analisa uma a uma.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A entrega E1: a coleta automatizada

A entrega E1 é a coleta automatizada, isto é, a parte do pipeline que busca os dados nas fontes sem intervenção humana. O enunciado exigiu que essa coleta usasse API, tratasse paginação e respeitasse limite de taxa, porque essas são as três dificuldades que toda coleta real encontra. Os servidores públicos expõem os dados por interfaces programáticas, entregam grandes volumes em páginas e protegem a si mesmos contra clientes que pedem demais. A E1 também exigiu que o dado bruto fosse guardado com a data no nome, porque um dado sem data não permite saber qual versão da fonte produziu cada resultado. No projeto, a entrega E1 corresponde ao pacote `pipeline/coleta/` e à camada `data/bruto`.

A entrega E2: o pipeline de ponta a ponta

A entrega E2 é o pipeline completo, que liga a coleta à carga no banco por meio de um único script, o `etl_sefin.py`. O enunciado pediu um script único porque um fluxo espalhado em vários comandos depende da memória de quem o executa, e a ordem errada dos passos produz um banco incoerente sem que ninguém perceba. Por essa razão, a E2 exige as três funções clássicas, extract, transform e load, e exige também validações impeditivas, log da execução, cruzamento com dados do IBGE, atributos para o modelo e carga no MySQL da Sprint 2. Cada uma dessas exigências responde a um risco concreto: a validação impede que dado ruim chegue ao banco, o log permite reconstruir o que aconteceu e o cruzamento com o IBGE dá contexto territorial à receita. A entrega E2, por conseguinte, é o esqueleto que sustenta todas as outras.

A entrega E3: a análise exploratória

A entrega E3 é a análise exploratória dos dados, chamada de EDA, e tem a função de mostrar o que os dados dizem antes de qualquer modelo. O enunciado exigiu estatísticas descritivas, gráficos honestos, mapa por setor censitário e uma frase de leitura em cada figura. A frase de leitura foi exigida porque um gráfico sem interpretação deixa o leitor tirar a conclusão que quiser, inclusive uma errada. No projeto, a EDA não é escrita à mão: o pacote `pipeline/eda/` calcula os números e redige as frases a partir dos próprios dados. Desse modo, a entrega E3 continua verdadeira mesmo quando os dados mudam, porque o texto é regenerado junto com eles.

A entrega E4: a reprodutibilidade

A entrega E4 é a reprodutibilidade, ou seja, a garantia de que outra pessoa consegue executar tudo do zero e chegar ao mesmo resultado. O enunciado a exigiu porque um resultado que só a equipe consegue produzir não pode ser verificado por um avaliador nem herdado pela sprint seguinte. A E4 se materializa num README que explica a instalação e a execução passo a passo. Além disso, o gate de qualidade G14 verifica, a cada rodada, que duas transformações da mesma entrada geram exatamente os mesmos bytes. A entrega E4, assim, transforma a reprodutibilidade de promessa em propriedade conferida por máquina.

As tarefas do Módulo 5

As tarefas do Módulo 5 complementam as quatro entregas com exigências de processo distribuídas em duas semanas. Na primeira semana, as tarefas pediram o mapa das fontes e um padrão de nomes, `fonte_assunto_AAAA-MM-DD`, porque um nome padronizado diz, sem abrir o arquivo, de onde ele veio, sobre o que trata e quando foi coletado. Na segunda semana, as tarefas pediram bases limpas, pelo menos três indicadores deflacionados pelo IPCA, pelo menos quatro figuras PNG a 150 dpi e um painel no Power BI. Os indicadores precisam ser deflacionados porque um real de 2010 não compra o mesmo que um real de 2026, e comparar valores nominais entre anos confunde inflação com crescimento. As tarefas do Módulo 5, portanto, impõem disciplina ao que as entregas pedem em termos gerais.

O painel do Power BI

O painel `.pbix` é a única entrega que o código do pipeline não produz. O painel não pode ser gerado por script porque o arquivo `.pbix` é montado na interface gráfica do Power BI Desktop, que não oferece uma forma programática e estável de criar visuais. Por essa razão, a equipe escreveu um guia de montagem em `docs/sprint3/power_bi.md`, que indica as views do banco a usar, os cartões, a série real e o filtro cruzado pedidos. O painel continua pendente porque depende de trabalho manual da equipe, e não de nova programação.

CONCLUSÃO — SÍNTESE

A análise das entregas revela um padrão comum: cada exigência do enunciado corresponde a um local no código e a uma evidência que permite conferi-la. A coleta está em `pipeline/coleta/` e se comprova pelo manifesto do bruto; o pipeline está em `etl_sefin.py` e se comprova pelo log; a EDA está em `pipeline/eda/` e se comprova pelo relatório gerado; a reprodutibilidade se comprova pelo gate G14. Esse padrão de "exigência, local e evidência" é o que torna a Sprint 3 verificável por alguém de fora da equipe. A única exceção, o painel, é justamente a parte que não pode ser verificada por máquina, e por isso ficou documentada por um guia.

2. Decisões que orientaram o trabalho

INTRODUÇÃO — O TODO

As decisões da Sprint 3 formam o conjunto de escolhas que o enunciado não determinava e que a equipe precisou tomar antes de escrever o código. Essas decisões merecem um registro próprio porque o código mostra o que foi feito, mas não mostra por que foi feito assim; Perry e Wolf (1992) descrevem a arquitetura de software como elementos, forma e justificativa, e a justificativa é a parte que se perde primeiro quando a equipe muda. Por isso, cada decisão foi registrada com data e com as alternativas descartadas, nos documentos chamados ADR (registros de decisão de arquitetura). O desenvolvimento a seguir separa esse todo em decisões de arquitetura, que moldam o código, e decisões de processo, que moldam a forma de trabalhar.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A decisão pelos pipes e filtros (ADR-001)

A primeira decisão de arquitetura organizou o pipeline no estilo de pipes e filtros, em que cada etapa recebe dados, devolve dados e não conhece as demais. A equipe escolheu esse estilo porque o pipeline combina sete fontes com formatos e ritmos de atualização diferentes, e uma mudança na API da Prefeitura não poderia obrigar a revisar o cruzamento territorial. Um script único, no estilo de caderno de notas, seria mais curto, mas misturaria rede, regra e banco na mesma função e impediria testar uma regra isolada. Um orquestrador de fluxos como o Airflow, por outro lado, acrescentaria um servidor e um agendador a um processo que roda em minutos numa única máquina. A decisão pelos pipes e filtros, desse modo, é a escolha intermediária: separa as responsabilidades sem acrescentar infraestrutura.

A decisão sobre a fonte NUCLEOGOV (ADR-002)

A segunda decisão de arquitetura definiu como a receita de 2019 em diante entra no pipeline, já que ela só existe no portal NUCLEOGOV. O portal responde HTTP 403 a qualquer acesso automatizado, inclusive ao arquivo robots.txt, o que significa que o operador do servidor recusa explicitamente coletores automáticos. A equipe decidiu respeitar essa recusa, porque contorná-la com troca de identidade, proxy ou VPN tornaria a coleta ilegítima (Krotov; Johnson; Silva, 2020). Em vez disso, a extração oficial, distribuída pela coordenação da disciplina, entra por uma cópia local, e o pipeline só aceita essa cópia se o seu hash for idêntico ao do arquivo publicado. A decisão sobre o NUCLEOGOV, portanto, troca a conveniência do download automático pela certeza de que o arquivo é exatamente o oficial.

A decisão de simular a inadimplência (ADR-003)

A terceira decisão de arquitetura determinou que a inadimplência do IPTU seria simulada, e não observada. Essa decisão foi necessária porque nenhuma fonte pública traz dívida, devedor ou pagamento por contribuinte, e a base da Secretaria da Fazenda ainda não foi fornecida; sem dado de inadimplência, o pipeline não teria o que mapear nem o que entregar à Sprint 4. A simulação usa os setores censitários reais e impõe um gradiente de renda, em que setores mais pobres atrasam mais, porque essa direção tem apoio na literatura (Alm et al., 2014). A equipe aceitou o custo dessa escolha: o padrão territorial da inadimplência simulada é consequência da premissa e não pode ser apresentado como descoberta sobre Palmas. A decisão de simular, assim, permite construir e testar todo o caminho do dado, com a condição de declarar a circularidade em cada lugar onde ela aparece.

As decisões de processo

As decisões de processo definiram como a equipe trabalharia, independentemente do que o código faz. A primeira foi continuar no repositório da Sprint 2, numa branch chamada `sprint-3`, porque o banco, o parser e os testes da Sprint 2 seriam reaproveitados, e um repositório novo romperia esse histórico. A segunda foi gerar a análise exploratória por script, porque um relatório digitado à mão envelhece no momento em que os dados mudam. A terceira, tomada em 21/10/2026, foi que nada vai ao GitHub sem pedido explícito, depois que a equipe questionou um acesso ao GitHub que não havia pedido. Desde então, o pipeline só acessa o GitHub com a opção `--nucleogov-github`, e o merge na `main`, a tag `v0.3.0` e qualquer envio dependem de autorização. As decisões de processo, por conseguinte, protegem a equipe contra ações que ela não escolheu.

CONCLUSÃO — SÍNTESE

A síntese das decisões mostra um padrão: em todas elas, a equipe preferiu a opção que mantém o controle e a verificabilidade, mesmo quando havia uma opção mais cômoda. Os pipes e filtros permitem testar cada regra sozinha; a cópia conferida do NUCLEOGOV permite provar a origem do dado; a simulação declarada permite separar premissa de achado; a regra sobre o GitHub impede ações não autorizadas. Esse conjunto funciona como o contrato da sprint, e o código e os documentos seguem esse contrato. Mudar qualquer uma dessas escolhas exige, por isso, um novo registro, e não uma edição silenciosa.

3. Regras de conduta

INTRODUÇÃO — O TODO

As regras de conduta da Sprint 3 são os limites éticos e de segurança que valem para o pipeline e para a equipe, mesmo quando nenhuma linha de código os impõe. Essas regras precisam ser escritas porque um pipeline automático age em nome de quem o executa: ele acessa servidores públicos, lida com credenciais e, no futuro, tratará dados de contribuintes. Uma ação errada de um programa tem a mesma consequência de uma ação errada de uma pessoa, com o agravante de se repetir a cada execução. O desenvolvimento a seguir decompõe as regras em quatro grupos: a ética da coleta, a segurança das credenciais, a privacidade e a autorização.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A ética da coleta

A ética da coleta determina que o pipeline respeite a vontade de quem opera cada servidor. A primeira consequência dessa regra é que uma recusa é aceita: quando o portal NUCLEOGOV respondeu HTTP 403, o pipeline registrou a recusa e não tentou de novo com outro User-Agent, proxy ou VPN, porque a recusa é a manifestação expressa do operador. A segunda consequência é a identificação honesta: o pipeline se apresenta como `PI2-UFT-Sefin-ETL/0.3`, nome que diz quem está coletando, em vez de imitar um navegador comum, o que enganaria o operador sobre a natureza do acesso. A terceira consequência é o ritmo: o pipeline espera pelo menos um segundo entre dois pedidos ao mesmo servidor, porque Krotov, Johnson e Silva (2020) mostram que a carga imposta ao servidor pesa na legitimidade da raspagem. A ética da coleta, assim, trata o servidor público como um recurso compartilhado, e não como uma fonte inesgotável.

A segurança das credenciais

A segurança das credenciais determina que senhas e tokens fiquem num único lugar, o arquivo `.env`, e nunca no Git, no log ou na linha de comando. Cada um desses lugares proibidos tem um motivo: o Git guarda o histórico para sempre, de modo que uma senha enviada uma vez continua recuperável mesmo depois de apagada; o log é lido por várias pessoas e copiado para documentos; e a linha de comando fica registrada no histórico do terminal e é visível para outros processos da máquina. O arquivo `.env`, ao contrário, fica fora do controle de versão, porque está listado no `.gitignore`. Como defesa adicional, o log mascara os valores de quatro variáveis sensíveis, trocando-os por `***`, de modo que mesmo um erro de programação não exponha uma senha. A segurança das credenciais, por conseguinte, combina uma regra de conduta com uma proteção automática que a reforça.

A privacidade

A privacidade determina que nenhum dado pessoal entre no pipeline sem proteção. Na Sprint 3, essa regra é cumprida pela própria natureza das fontes: a arrecadação vem agregada por conta contábil e mês, o Censo vem agregado por setor censitário, e as quadras do GeoPalmas não identificam proprietários. Por isso, a coleta atual não aciona a Lei Geral de Proteção de Dados (Lei nº 13.709/2018). A situação muda quando chegar a base da Secretaria da Fazenda, que identifica contribuintes; para esse momento, a regra exige pseudonimizar os identificadores antes de qualquer gravação. A pseudonimização está especificada, mas ainda não implementada: a variável `PSEUDONIMO_CHAVE` já consta do modelo `.env.example` e do filtro do log, e a implementação depende da chegada da base. A privacidade, desse modo, está garantida hoje pela ausência de dado pessoal e planejada para quando ele existir.

A autorização

A autorização determina que certas ações só aconteçam com pedido explícito do usuário. Essas ações são enviar ao GitHub, fazer merge, criar tag, fazer commit e consultar o Crossref. Todas têm em comum o fato de produzirem efeito fora da máquina local ou de serem difíceis de desfazer: um envio ao GitHub torna o conteúdo visível fora da máquina da equipe, um merge altera o histórico da branch principal, e uma consulta ao Crossref envia os DOIs do catálogo a um serviço externo. Por isso, nenhuma delas é executada por iniciativa própria, e as referências bibliográficas continuam marcadas como "conferência pendente" até que a consulta seja autorizada. A autorização, assim, mantém nas mãos da equipe as decisões que têm consequência pública.

CONCLUSÃO — SÍNTESE

A síntese das regras de conduta revela que todas protegem alguém que não participa da execução do pipeline. A ética da coleta protege o operador do servidor; a segurança das credenciais protege a própria equipe e as instituições que lhe dão acesso; a privacidade protege os contribuintes; a autorização protege a decisão da equipe contra ações que ela não escolheu. O padrão comum é o reconhecimento de que um programa automático age em nome de pessoas, e por isso precisa dos mesmos limites que uma pessoa responsável respeitaria.

4. Método de trabalho

INTRODUÇÃO — O TODO

O método de trabalho da Sprint 3 é a sequência repetível de passos que cada mudança no código percorreu antes de ser aceita. O método precisa ser descrito porque um resultado só merece confiança quando outra pessoa consegue refazer o caminho que o produziu; uma afirmação da equipe, sem esse caminho, valeria apenas pela palavra de quem a fez. O desenvolvimento a seguir decompõe o método em quatro práticas: a divisão do código pelas decisões que mudam, os testes antes da execução real, a evidência de cada afirmação e o uso de commits locais.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A divisão do código pelas decisões que mudam

A primeira prática dividiu o código pelas decisões que tendem a mudar, e não pelas etapas de um fluxograma. Esse critério vem de Parnas (1972), que mostrou que um sistema dividido por etapas obriga a alterar vários módulos sempre que uma decisão muda, enquanto um sistema dividido por decisões concentra cada mudança num único lugar. No pipeline, cada pasta de `pipeline/` esconde uma decisão: a forma de uma fonte, a regra de limpeza de uma base ou o critério de um cruzamento territorial. Para que essa divisão não se perca com o tempo, a ferramenta `import-linter` verifica nove contratos de dependência, cinco deles criados na Sprint 3, e reprova o código se um módulo importar outro que não deveria conhecer. A divisão do código, portanto, é ao mesmo tempo uma intenção de projeto e uma regra conferida por máquina.

Os testes antes da execução real

A segunda prática exigiu que cada regra fosse testada antes de o pipeline tocar nos dados reais. Os testes não usam rede nem banco: respostas HTTP gravadas substituem os servidores, e uma cidade em miniatura, chamada mini-Palmas, substitui o bruto real. Essa escolha existe porque um teste que depende da internet falha quando o servidor está fora do ar, e então o teste deixa de dizer se o código está certo. Além disso, os testes provocam falhas de propósito, porque uma validação que nunca foi vista interrompendo a execução pode ser decorativa, isto é, pode estar escrita de modo que nunca reprova. Os testes antes da execução real, por conseguinte, provam tanto que o código faz o que deve quanto que as proteções reagem quando devem.

A evidência de cada afirmação

A terceira prática exigiu que cada número dos documentos viesse de uma execução registrada. A execução de referência recebeu o identificador `20261021T065740-9b9054`, e esse mesmo identificador aparece no log, no banco de dados e no relatório de qualidade, o que permite conferir que os três registros descrevem a mesma execução. Essa prática foi adotada porque números escritos de memória tendem a ser arredondados, desatualizados ou simplesmente errados. A revisão dos documentos confirmou esse risco ao encontrar valores estimados, que foram trocados pelos valores medidos. A evidência de cada afirmação, assim, substitui a confiança na memória pela conferência do registro.

Os commits locais

A quarta prática manteve todo o trabalho em commits locais, sem envio ao GitHub. O commit local guarda o histórico na máquina da equipe, de modo que cada mudança pode ser revista, comparada e desfeita, sem que nada se torne público antes da decisão da equipe. Em 21/10/2026, a branch `sprint-3` tinha 28 commits à frente da `main`, e nenhum deles havia sido enviado. Cada commit declara, no rodapé Co-Authored-By, a participação do assistente de IA, porque a autoria do código precisa ser transparente para a avaliação; a equipe ainda vai decidir se mantém esse rodapé. Os commits locais, portanto, conciliam o registro completo do histórico com o controle da equipe sobre o que é publicado.

CONCLUSÃO — SÍNTESE

A síntese do método mostra que as quatro práticas têm o mesmo propósito: substituir a confiança na palavra da equipe por uma verificação que qualquer pessoa pode repetir. A divisão do código é conferida pelos contratos, as regras são conferidas pelos testes, os números são conferidos pelo log e pelo banco, e o histórico é conferido pelos commits. O método torna, assim, cada resultado da Sprint 3 verificável, e não apenas declarado.

5. Tratamento de defeitos e transparência

INTRODUÇÃO — O TODO

O tratamento de defeitos da Sprint 3 é a forma como a equipe reagiu aos erros encontrados durante o desenvolvimento e a execução real. Esse tratamento merece análise porque a execução com dados reais revelou falhas que os dados montados à mão não mostravam, e a maneira de corrigir um erro diz tanto sobre a qualidade do trabalho quanto a ausência de erros. A regra adotada foi que cada defeito recebe uma correção e uma verificação automática que impede sua volta. O desenvolvimento a seguir separa os defeitos em três grupos, segurança, cálculo e execução real, e trata à parte um erro de registro que exigiu transparência.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os defeitos de segurança

Os defeitos de segurança foram dois, ambos encontrados numa revisão dedicada a esse tema. O primeiro era a injeção de fórmula nos arquivos CSV: um texto vindo da fonte e iniciado por `=` seria interpretado como fórmula quando alguém abrisse o arquivo no Excel, e uma fórmula maliciosa pode executar comandos ou enviar dados para fora. A correção acrescenta um apóstrofo no início desse tipo de texto, o que faz a planilha exibi-lo como texto em vez de executá-lo. O segundo defeito era a falta de limite na descompactação do ZIP do Censo: um arquivo malicioso, pequeno quando compactado, poderia se expandir até esgotar a memória da máquina. A correção impôs um teto de 256 MiB e uma razão máxima de 50 para 1 entre o tamanho descompactado e o compactado. Os defeitos de segurança, assim, foram fechados com regras que tratam todo arquivo externo como potencialmente hostil.

Os defeitos de cálculo

Os defeitos de cálculo foram dois e afetavam a exatidão dos números. O primeiro estava no fator do IPCA: o Python calculava o valor real com o fator em precisão total, enquanto o banco recebia o fator arredondado em seis casas decimais, e por isso a view do banco e os arquivos divergiam em até R$ 15. A correção fixou o fator em dez casas e passou a usar o mesmo fator nos dois lugares, o que fez o banco e os arquivos coincidirem ao centavo. O segundo defeito estava na

marcação da quebra de fonte: o indicador comparava cada ano apenas com o ano imediatamente anterior, e como 2018 foi excluído da série, o ano de 2019 não era marcado como o início do trecho do NUCLEOGOV. A correção passou a comparar com o último ano que tem dado. Os defeitos de cálculo, desse modo, mostraram que uma regra certa em condições normais pode falhar justamente na exceção.

Os defeitos da execução real

Os defeitos da execução real só apareceram quando o pipeline rodou com os dados verdadeiros. O primeiro foi a perda de um arquivo na carga: 61 arquivos foram coletados, mas só 60 foram gravados no banco, porque a tabela usava o SHA-256 do conteúdo como chave, e dois arquivos diferentes tinham conteúdo idêntico; a correção passou a usar o nome do arquivo como chave. O segundo foi a duplicação de uma quadra depois de reaplicar o script `sprint3.sql`, porque a busca da versão vigente da quadra dependia de uma tabela que o script recriava; a correção passou a comparar as geometrias diretamente, com a função `ST_Equals` do MySQL. O terceiro foi uma execução que ficou registrada como "EM_ANDAMENTO" para sempre, porque o código só fechava o registro nas falhas previstas; a correção fez toda exceção fechar o registro como FALHA. Os defeitos da execução real, por conseguinte, confirmam que nenhum teste substitui por completo o contato com os dados verdadeiros.

A transparência sobre um erro de registro

A transparência sobre um erro de registro tratou de uma afirmação falsa feita na própria trilha de auditoria. A mensagem do commit `ea64aa6` afirmou que a falha de uma execução tinha ficado registrada no banco, mas isso não era verdade, porque aquela falha foi justamente do tipo não previsto, que deixava o registro aberto. O commit seguinte, `fd739bf`, corrigiu o código e declarou explicitamente que a mensagem anterior estava errada, e o registro daquela execução foi fechado à mão como FALHA, com a explicação no campo de erro. A equipe escolheu declarar o erro em vez de reescrever o histórico, porque apagar uma mensagem errada destruiria a confiabilidade de todas as outras. A transparência, assim, manteve a trilha verdadeira, ainda que com uma correção manual que ficou documentada.

CONCLUSÃO — SÍNTESE

A síntese do tratamento de defeitos revela um padrão: cada erro foi transformado numa proteção permanente, e nenhum foi escondido. Os defeitos de segurança viraram limites, os de cálculo viraram regras de precisão, os da execução real viraram testes, e o erro de registro virou uma declaração explícita no histórico. Esse padrão importa porque um projeto que esconde seus erros obriga o leitor a desconfiar de tudo, enquanto um projeto que os declara permite confiar no restante.

6. Situação e pendências

INTRODUÇÃO — O TODO

A situação da Sprint 3 em 21/10/2026 é a de um trabalho tecnicamente concluído, com pendências que dependem de decisão ou de trabalho manual. Essa distinção é importante porque orienta o que fazer até a entrega, marcada para 27/10/2026: uma pendência técnica exigiria programar, testar e executar de novo, enquanto uma pendência de decisão ou de trabalho manual exige apenas que alguém a assuma. O desenvolvimento a seguir separa esse todo no que está pronto, no que falta e na situação particular da pasta de trabalho `PI2_Sprint3`.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O que está pronto

O que está pronto abrange as quatro entregas do enunciado, de E1 a E4. A execução de referência coletou sete fontes, aprovou as 33 avaliações das validações impeditivas, carregou o banco e gerou a análise exploratória com um único comando. Essas quatro etapas mostram que o pipeline funciona de ponta a ponta com dado real, e não apenas com os dados de teste. Além disso, treze dos quatorze quality gates passaram, o que inclui testes, cobertura, complexidade, tipos, mutação, desempenho e reprodutibilidade. O que está pronto, portanto, é o pipeline inteiro, verificado por execução real e por critérios objetivos de qualidade.

O que falta

O que falta são três frentes, e nenhuma exige funcionalidade nova. A primeira é a conferência dos DOIs no Crossref: o gate G13 fica reprovado até que cada DOI do catálogo seja conferido, e a conferência depende de autorização porque envia dados a um serviço externo. A segunda é o painel `.pbix` e os nomes da equipe nos documentos, que dependem de trabalho manual; o guia do painel já está escrito. A terceira é a integração na `main`, que exige o merge `--no-ff`, a tag `v0.3.0` e o envio ao GitHub, e cada um desses passos depende de autorização explícita. O que falta, desse modo, é de fechamento, e não de construção.

A pasta de trabalho da Sprint 3

A pasta `PI2_Sprint3` é uma cópia enxuta que guarda só a Sprint 3 e a parte da Sprint 2 de que ela depende. A pasta foi criada para que a equipe tenha num único lugar os arquivos referentes à sprint, sem o restante do repositório. Por escolha da equipe, a pasta não tem o histórico Git, e essa ausência tem uma consequência: a função que identifica a versão do código consulta o Git, e, sem ele, registra a versão como "desconhecida". Por isso, uma execução feita nessa pasta funciona normalmente, mas perde o vínculo com um commit específico. A pasta de trabalho, assim, é adequada para estudar e executar o pipeline, mas a evidência oficial continua sendo a do repositório com histórico.

CONCLUSÃO — SÍNTESE

A síntese da situação mostra que a Sprint 3 chegou ao ponto em que o trabalho restante é de decisão, e não de engenharia. Todas as pendências têm dono e caminho definidos: a conferência das referências aguarda autorização, o painel e os nomes aguardam a equipe, e a integração aguarda o OK para os passos que envolvem o repositório remoto. A próxima etapa, portanto, é de fechamento.

Parte II — O código, etapa por etapa

7. Visão geral da arquitetura

INTRODUÇÃO — O TODO

A arquitetura do pipeline é uma sequência de filtros ligados apenas pelos dados que um entrega ao outro, como mostra a Figura 1. A coleta busca os dados nas fontes e grava o bruto; a limpeza lê o bruto e produz tabelas tipadas; o enriquecimento acrescenta inflação e território; a simulação e os atributos preparam a base para o modelo; a carga leva tudo ao MySQL; e a EDA lê as camadas finais para escrever o relatório. Entre os filtros, as validações impeditivas funcionam como portões, porque um filtro nunca deve receber uma entrada que já se sabe defeituosa. O desenvolvimento a seguir decompõe essa arquitetura em duas partes: a divisão do código em pacotes e os contratos que impedem um pacote de depender de outro indevidamente.

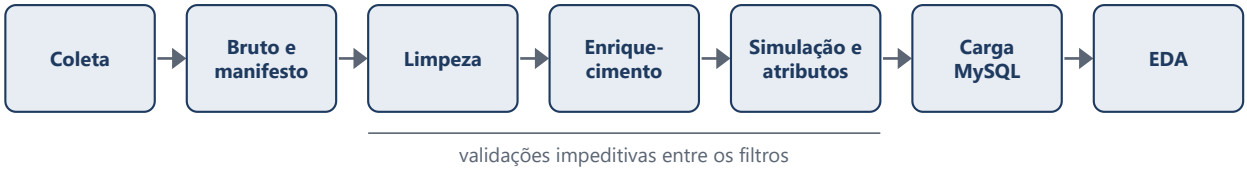


Figura 1 — Sequência dos filtros do pipeline da Sprint 3.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os pacotes e suas responsabilidades

O pacote `pipeline/` reúne todo o código da Sprint 3 e se divide em subpastas, cada uma com uma única responsabilidade. Essa divisão segue o critério de Parnas (1972): cada pasta esconde uma decisão que pode mudar, de modo que a mudança fique confinada a ela. Se a Prefeitura alterar o formato da API, por exemplo, só a pasta da coleta precisa mudar; se a regra de deflação for revista, só a pasta do enriquecimento muda. A tabela abaixo apresenta cada pasta e a decisão que ela guarda.

Pasta ou módulo	Responsabilidade
`etl_sefin.py`	ponto de entrada: opções da linha de comando e as funções extract, transform e load
`pipeline/orquestrador.py`	fachada que encadeia as etapas, abre o log e registra a execução
`pipeline/dominio.py`	constantes e erros compartilhados, com o código de saída de cada erro
`pipeline/observabilidade.py`	identificador da execução, formato do log e máscara de segredos
`pipeline/camadas.py`	pastas das camadas, nome dos arquivos, manifesto e depósito do bruto
`pipeline/coleta/`	política de requisição, robots.txt, cliente HTTP e adaptadores das sete fontes
`pipeline/transformacao/`	leitura do bruto, encadeamento dos filtros e gravação das camadas
`pipeline/limpeza/`	um filtro por fonte, com quarentena e contagens
`pipeline/enriquecimento/`	receita real, indicadores, sazonalidade e cruzamento territorial
`pipeline/simulacao/`	parâmetros e gerador da dívida de IPTU sintética
`pipeline/atributos/`	base analítica e agregados por setor

Pasta ou módulo	Responsabilidade
`pipeline/qualidade/`	validações impeditivas e informativas
`pipeline/carga/`	registro da execução e carga transacional no MySQL
`pipeline/eda/`	estatísticas, gráficos, mapas e o relatório de EDA
`pipeline/tabelas.py` e `pipeline/geo.py`	formato dos arquivos e operações geométricas

O contrato da pureza dos filtros

O contrato da pureza, escrito em duas regras do `import-linter`, uma para a limpeza e outra para o enriquecimento, a simulação e os atributos, determina que esses filtros sejam puros, isto é, que não importem a biblioteca de rede `requests`, o driver de banco `pymysql`, a coleta nem os módulos da Sprint 2 que acessam o banco. Esse contrato existe porque um filtro que acessa a rede ou o banco só pode ser testado com servidor e banco disponíveis, e então um teste lento ou instável passaria a decidir se uma regra de limpeza está certa. Com o contrato, cada filtro recebe dados em memória e devolve dados em memória, e o teste fornece esses dados diretamente. O módulo `etl.parser` da Sprint 2 continua permitido, porque também é puro e é reaproveitado na limpeza do NUCLEOGOV. O contrato da pureza, assim, garante que as regras de negócio sejam testáveis em segundos e sem infraestrutura.

O contrato da independência entre os filtros

O contrato da independência determina que os quatro pacotes de filtros, limpeza, enriquecimento, simulação e atributos, sejam independentes entre si, de modo que nenhum importe outro. Esse contrato é o que torna o estilo de pipes e filtros real, e não apenas desejado: se o enriquecimento pudesse chamar uma função da limpeza, uma mudança na limpeza poderia quebrar o enriquecimento por um caminho invisível. Com o contrato, os filtros só se comunicam pelos dados que o orquestrador passa de um para o outro. A independência entre os filtros, por conseguinte, faz com que cada filtro possa ser substituído sem que os demais percebam.

Os contratos da coleta e da EDA

Os contratos da coleta e da EDA isolam as pontas do pipeline. O contrato da coleta proíbe que ela importe a limpeza, o `pymysql` ou o pacote `etl` da Sprint 2, porque a coleta deve apenas trazer e guardar os dados como chegaram, sem interpretá-los nem gravá-los no banco; se a coleta conhecesse a limpeza, uma falha de interpretação poderia impedir que o bruto fosse guardado. O contrato da EDA proíbe que ela importe a rede, o banco, a coleta, a carga e os filtros, porque a EDA deve descrever o que as camadas contêm, e não recalcular por conta própria o que os filtros já calcularam. Esses contratos, somados aos quatro herdados da Sprint 2, formam os nove verificados pelo gate G8. Os contratos das pontas, desse modo, garantem que entrada e saída do pipeline não se misturem com o processamento do meio.

CONCLUSÃO — SÍNTESE

A síntese da arquitetura mostra que toda a sua organização obedece a um único princípio: separar o que muda em ritmos diferentes. A rede, o disco e o banco mudam por razões técnicas externas e ficam nas pontas; as regras de negócio mudam por decisão da equipe e ficam no meio, em filtros puros; e os contratos garantem que essa separação não se desfaça com o tempo. Essa é a razão pela qual o pipeline pode crescer, receber novas fontes e trocar regras sem que cada mudança se espalhe pelo código inteiro.

8. Ponto de entrada: etl_sefin.py

INTRODUÇÃO — O TODO

O arquivo `etl_sefin.py` é a porta de entrada do pipeline, o único comando que o usuário precisa executar. A função desse arquivo é traduzir o pedido do usuário, escrito na linha de comando, em opções que o restante do código entende, e devolver ao sistema operacional um código que diga se tudo correu bem. O arquivo não contém nenhuma regra de negócio, porque a porta de entrada precisa ser simples o bastante para que qualquer pessoa entenda o que acontece ao executá-la. O desenvolvimento a seguir analisa as três funções pedidas pelo enunciado, as opções da linha de comando e a função `main`.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

As três funções pedidas pelo enunciado

As funções `extract`, `transform` e `load` existem no script porque o enunciado as exige nominalmente, mas cada uma apenas repassa a chamada ao orquestrador. A função `extract` devolve o relatório da coleta; a função `transform` devolve os brutos lidos e os produtos da transformação; a função `load` devolve as contagens da carga. Essas funções não contêm a lógica porque, se a contivessem, o script ficaria longo e misturaria a interface com a execução; manter a lógica no orquestrador permite que os testes chamem as etapas diretamente, sem passar pela linha de comando. As três funções, portanto, cumprem a exigência formal do enunciado sem duplicar código.

A opção --offline

A opção `--offline` impede qualquer acesso à rede durante a execução. Com essa opção, a etapa extract não faz nenhuma requisição, e a transformação usa a última coleta completa que o manifesto registra para cada fonte. A opção existe porque uma execução precisa poder ser reproduzida mesmo quando uma fonte está fora do ar, quando não há internet ou quando o avaliador quer refazer a transformação sobre exatamente o mesmo bruto. A opção `--offline`, assim, separa a reprodutibilidade da disponibilidade das fontes.

A opção --recoletar

A opção `--recoletar` obriga o pipeline a pedir tudo de novo às fontes. Sem essa opção, um arquivo já coletado no mesmo dia é reaproveitado, porque pedir de novo a mesma página no mesmo dia aumentaria a carga sobre o servidor público sem trazer informação nova. Com a opção, o depósito do bruto deixa de reaproveitar, e o pipeline consulta cada fonte como se fosse a primeira vez. A opção existe para o caso em que a equipe sabe que a fonte mudou durante o dia ou suspeita de uma coleta defeituosa. A opção `--recoletar`, por conseguinte, é a exceção controlada à regra da cortesia.

As opções --sem-banco e --sem-eda

As opções `--sem-banco` e `--sem-eda` desligam as duas etapas finais do pipeline. A opção `--sem-banco` pula a carga e o registro da execução no banco: as camadas `data/limpo` e `data/pronto` continuam sendo gravadas, e a EDA continua sendo gerada, mas nada vai ao MySQL; ela existe porque nem toda máquina tem o banco instalado, e a transformação precisa poder ser verificada sem ele. A opção `--sem-eda` dispensa a geração do relatório e das figuras, o que poupa tempo quando a equipe está testando apenas a coleta ou a carga. Essas duas opções, desse modo, permitem executar só a parte do pipeline que interessa a cada situação.

As opções do NUCLEOGOV

As opções `--nucleogov-arquivo` e `--nucleogov-github` controlam de onde vem a extração NUCLEOGOV. A opção `--nucleogov-arquivo` indica o caminho da cópia local; quando ela não é usada, vale a variável de ambiente `PI2_NUCLEOGOV_ARQUIVO`, porque o caminho de um arquivo é uma configuração da máquina e não deveria precisar ser digitado a cada execução. A opção `--nucleogov-github` autoriza o download a partir do repositório da coordenação, e é uma opção explícita porque a equipe decidiu que nenhum acesso ao GitHub acontece sem pedido. As opções do NUCLEOGOV, assim, tornam visível na própria linha de comando uma decisão que tem consequência externa.

A função main

A função `main` monta as opções a partir da linha de comando e devolve o código de saída. Antes de qualquer outra coisa, ela chama `carregar_env`, que lê o arquivo `.env` e define as variáveis de ambiente que ainda não existem; as variáveis já definidas não são sobrescritas, para que um valor fornecido explicitamente, como o de um servidor de integração contínua, prevaleça sobre o arquivo. Essa leitura acontece antes da criação do log por um motivo preciso: o filtro do log precisa conhecer os valores das senhas para mascará-los, e só os conhece se o `.env` já tiver sido lido. Em seguida, a função cria um objeto `Opcoes` e chama `orquestrador.executar`. A função `main`, portanto, prepara o ambiente na ordem que a segurança exige e delega todo o resto.

A função caminho_do_ambiente

A função `caminho_do_ambiente` lê um caminho de arquivo de uma variável de ambiente e o limpa antes de usá-lo. A limpeza remove espaços e aspas nas pontas, porque, no Windows, a opção "copiar como caminho" do Explorador de Arquivos devolve o caminho entre aspas, e um caminho com aspas literais não corresponde a nenhum arquivo. A função também expande o símbolo `~` para a pasta do usuário e devolve nada quando a variável está vazia. A função `caminho_do_ambiente`, assim, evita um erro comum de configuração antes que ele se transforme numa mensagem confusa de arquivo inexistente.

O código de saída

O código de saída é o número que o processo devolve ao sistema operacional ao terminar, e a chamada `sys.exit(main())` entrega a ele o valor devolvido por `main`. Esse número importa porque um agendador de tarefas ou um servidor de integração contínua não lê o log; ele só sabe se a execução deu certo pelo código de saída. O pipeline usa quatro códigos: 0 para sucesso, 2 para validação de qualidade reprovada, 3 para falha de coleta e 1 para qualquer outro erro. O código de saída, por conseguinte, permite que uma máquina distinga um servidor fora do ar de um dado inválido e reaja de forma diferente a cada caso.

CONCLUSÃO — SÍNTESE

A síntese do ponto de entrada revela que todas as suas partes servem para traduzir, e não para decidir. As funções pedidas traduzem a exigência do enunciado em chamadas ao orquestrador; as opções traduzem a vontade do usuário em configuração; a função `main` traduz o ambiente em condições seguras; e o código de saída traduz o resultado em um número que outra máquina entende. Esse papel de tradutor é o que permite ao script permanecer curto e estável enquanto o pipeline evolui.

9. Orquestrador

INTRODUÇÃO — O TODO

O orquestrador, no módulo `pipeline/orquestrador.py`, é a fachada que conhece a ordem das etapas e as executa uma após a outra. O padrão de fachada foi escolhido porque o pipeline tem muitas partes, coleta, limpeza, validações, carga e EDA, e quem o usa não precisa conhecer cada uma; precisa apenas pedir "execute" e receber o resultado (Valente, 2020, §6.6). Além de encadear as etapas, o orquestrador cria o contexto da execução, transforma falhas em códigos de saída e registra cada execução no banco. O desenvolvimento a seguir analisa os objetos que guardam o estado, a preparação, cada uma das quatro etapas e o tratamento das falhas.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O objeto Opcoes

O objeto `Opcoes` guarda as escolhas que o usuário fez para uma execução. Seus campos correspondem às opções da linha de comando: `offline`, `recoletar`, `banco`, `eda`, `nucleogov_arquivo` e `nucleogov_github`. Há ainda dois campos que a linha de comando não oferece: `raiz`, que permite aos testes usar uma pasta temporária em vez das pastas reais do projeto, e `parametros`, que contém os parâmetros da simulação. Esses dois campos existem porque os testes precisam executar o pipeline completo sem tocar nos dados verdadeiros e com uma simulação reduzida. O objeto `Opcoes`, assim, reúne num só lugar tudo o que pode variar de uma execução para outra.

O objeto Execucao

O objeto `Execucao` guarda o estado de uma execução em andamento. Ele reúne o identificador, o instante de início, as pastas das camadas, o depósito do bruto, o log, as opções, a versão do código, o verificador de qualidade e um resumo que vai sendo preenchido pelas etapas. Esse objeto existe para que cada etapa receba um único argumento com tudo de que precisa, em vez de uma lista longa de parâmetros que mudaria a cada nova necessidade. Além disso, o resumo acumulado no objeto é o que, no fim, vai para o banco como descrição da execução. O objeto `Execucao`, por conseguinte, funciona como a memória compartilhada de uma execução.

A preparação

A função `preparar` cria o contexto antes de qualquer etapa e executa seus passos numa ordem que tem razão de ser. Primeiro, ela fixa o instante de início no fuso de Palmas, porque é esse instante que dá nome à execução e define o dia de hoje para a coleta e para a transformação. Depois, cria as quatro pastas das camadas, porque o log e o bruto precisam de onde ser gravados. Em seguida, gera o identificador da execução e abre o arquivo de log com esse identificador no nome, de modo que todas as mensagens seguintes já saiam marcadas. Por fim, cria o depósito do bruto com a regra de reaproveitamento e consulta a versão do código no Git. A preparação, desse modo, garante que nenhuma etapa comece sem log, sem identificador e sem lugar para gravar.

A etapa extract

A etapa extract é a que busca os dados nas fontes, e seu comportamento depende do modo offline. No modo offline, a etapa apenas registra no log que nenhuma requisição será feita, porque a transformação usará a última coleta completa já registrada. Fora do modo offline, a etapa cria o cliente HTTP com a política lida do ambiente, monta a lista das sete fontes com a data de hoje e as opções do NUCLEOGOV, e chama `coletar_tudo` para percorrê-las. Ao final, a etapa grava no

resumo da execução as fontes coletadas, as falhas e o número de requisições, porque esses três dados descrevem, para quem lê o banco depois, o que a coleta conseguiu. A etapa extract, assim, isola numa única chamada todo o contato do pipeline com a internet.

A etapa transform

A etapa transform transforma o bruto nas camadas limpo e pronto, e começa por uma verificação que precede qualquer leitura. Essa verificação é a validação V12, que confere a cadeia de hashes do manifesto e o SHA-256 de cada arquivo bruto; se ela reprovar, a execução para antes de ler um único dado, porque um bruto adulterado ou incompleto contaminaria todos os resultados seguintes. Aprovado o bruto, a etapa chama `ler_brutos` para ler os arquivos, `transformar` para aplicar os filtros e as validações, e `gravar` para escrever as camadas. Junto com os arquivos, a etapa grava, no relatório de qualidade da camada pronta, o contexto da execução, com o identificador, a versão do código e os parâmetros da simulação, para que os arquivos produzidos possam ser ligados à execução que os gerou. A etapa transform, por conseguinte, só produz resultados a partir de um bruto comprovadamente íntegro.

A etapa load

A etapa load leva os produtos da transformação ao MySQL, e o módulo de carga é importado só no momento em que a etapa é chamada. Esse import tardio tem um motivo prático: o módulo de carga depende do driver `pymysql`, e, se ele fosse importado no topo do arquivo, o pipeline inteiro deixaria de funcionar numa máquina sem o driver, mesmo com a opção `--sem-banco`. A etapa recebe a conexão já aberta pelo registro da execução, para que a carga e o registro usem o mesmo banco. A etapa load, desse modo, só exige o banco de quem realmente quer carregar os dados.

A etapa EDA

A etapa EDA gera o relatório de análise exploratória e as figuras a partir das camadas gravadas. Ela também é importada só quando chamada, pelo mesmo motivo da carga: a EDA depende de bibliotecas de gráficos que não são necessárias para coletar ou transformar. Ao terminar, a etapa registra no log quantos arquivos gerou. A etapa EDA, assim, fecha a execução transformando os dados em texto e imagem sem impor suas dependências às etapas anteriores.

A função executar e o tratamento das falhas

A função `executar` realiza a execução completa: prepara o contexto, abre o registro no banco, chama as etapas na ordem e devolve o código de saída. O ponto central dessa função é o tratamento das falhas, que distingue dois tipos de erro. Uma falha prevista é um `ErroPipeline`, que carrega no atributo `codigo_saida` o número correspondente ao tipo de problema; nesse caso, a função registra o erro no log, fecha o registro da execução como FALHA e devolve o código do próprio erro. Uma falha não prevista é qualquer outra exceção, como um defeito de programação ou um disco cheio; nesse caso, a função grava no log o rastro completo do erro, também fecha o registro como FALHA e devolve o código 1. Esse segundo caso foi acrescentado depois que uma execução real ficou registrada para sempre como "EM_ANDAMENTO". A função `executar`, por conseguinte, garante que toda execução termine com um registro fechado, seja qual for o motivo do fim.

A descrição das opções

A função `_descrever` escreve as opções da execução numa linha de texto que vai para o log e para o banco. Essa descrição permite saber, depois, com que configuração cada execução rodou. Do arquivo NUCLEOGOV local, porém, a função registra apenas o nome, limitado a 120 caracteres, e nunca o caminho completo. A razão é de privacidade: o caminho completo costuma conter o nome de usuário do computador, como em `C:\Users\nome\Downloads`, e esse dado pessoal não tem nenhuma utilidade para auditar a execução. A descrição das opções, assim, registra o que é útil e omite o que identifica pessoas.

O registro da execução no banco

O registro da execução no banco é aberto antes da primeira etapa e fechado depois da última, e cada uma dessas operações usa uma transação própria. A função `_abrir_registro` conecta ao banco, garante que o esquema exista e grava o início da execução com a situação EM_ANDAMENTO. A função `_fechar_registro` grava o fim, a situação, o resumo, o erro e todas as validações avaliadas, e depois fecha a conexão. As transações são separadas da carga por um motivo essencial: se a carga falhar, a sua transação é desfeita, e, se o registro estivesse na mesma transação, a falha apagaria também a prova de que a execução aconteceu. O registro da execução, portanto, sobrevive justamente aos erros que precisa documentar.

CONCLUSÃO — SÍNTESE

A síntese do orquestrador mostra que suas partes se organizam em torno de um único papel: ser o único lugar que conhece a ordem do pipeline. Os objetos `Opcoes` e `Execucao` guardam o estado, a preparação cria as condições, as etapas são chamadas em sequência e o tratamento de falhas garante um fim registrado. As etapas não sabem umas das outras, e só a fachada conhece a ordem; por isso, alterar a sequência do pipeline exige mudar um único arquivo, e nenhum filtro precisa saber que a mudança aconteceu.

10. Domínio: constantes e erros

INTRODUÇÃO — O TODO

O módulo `pipeline/dominio.py` guarda o que todas as etapas do pipeline compartilham: as constantes do município e a hierarquia de erros. Esse módulo existe porque um valor usado em vários lugares, como o código do município, precisa ser definido uma única vez; se fosse repetido em cada módulo, bastaria um erro de digitação num deles para que o pipeline descartasse, sem aviso, os dados de Palmas. O módulo não faz entrada nem saída, o que permite que qualquer outro módulo o importe sem arrastar dependências. O desenvolvimento a seguir analisa as constantes e, depois, a hierarquia de erros.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

As constantes do município

As constantes do município identificam Palmas nos sistemas do IBGE e da Prefeitura. O código IBGE de Palmas é `1721000`, e o do Tocantins é `17`; esses códigos são usados para filtrar, entre os dados nacionais do Censo e da malha de setores, apenas o que pertence ao município. A constante `TRIBUTOS` lista os três tributos do projeto, IPTU, ISSQN e ITBI, com os mesmos códigos da tabela `tributo` criada na Sprint 2, para que a carga encontre a correspondência no banco sem tradução. As constantes do município, assim, garantem que todas as etapas falem dos mesmos lugares e dos mesmos tributos.

O fuso horário de Palmas

O fuso horário de Palmas é definido como UTC−3 fixo, sem tabela de transições. Essa simplificação é correta porque o Brasil não adota horário de verão desde 2019, quando o Decreto nº 9.772/2019 o extinguiu. Com um fuso fixo, o pipeline não depende do banco de dados de fusos horários instalado no sistema operacional, que pode faltar ou estar desatualizado em algumas instalações. O fuso de Palmas importa porque é ele que define a data no nome dos arquivos brutos e o instante de início de cada execução. O fuso fixo, por conseguinte, elimina uma fonte de diferença entre máquinas sem perder exatidão.

O erro genérico do pipeline

O erro `ErroPipeline` é a raiz da hierarquia de erros e representa qualquer falha que interrompe o pipeline. Ele usa o código de saída 1, e todos os outros erros do pipeline herdam dele. Essa herança permite que o orquestrador trate todas as falhas previstas com um único bloco, lendo o código de saída do próprio erro, em vez de conhecer cada tipo de erro separadamente. O erro genérico, assim, é o que permite acrescentar novos tipos de falha sem alterar o orquestrador.

O erro de coleta e o erro de segurança

O erro `ErroColeta` indica que uma fonte não entregou os dados, e usa o código de saída 3. Esse erro cobre situações como queda de rede, resposta HTTP recusada, robots.txt que proíbe o acesso e conteúdo que não pode ser interpretado. O erro `ErroSeguranca` é um tipo particular de `ErroColeta`, reservado às violações da política de transporte: tentativa de usar HTTP sem criptografia, link de paginação que leva a outro servidor e certificado inválido. O erro de segurança herda do erro de coleta porque, do ponto de vista do pipeline, as duas situações têm a mesma consequência, a fonte não pôde ser coletada; o nome distinto, porém, deixa claro no log que o problema foi de segurança, e não de disponibilidade. Os erros de coleta, por conseguinte, separam o problema de rede do problema de confiança sem complicar o tratamento.

O erro de qualidade

O erro `ErroQualidade` indica que uma validação impeditiva reprovou, e usa o código de saída 2. Diferentemente dos outros erros, ele guarda dois atributos: a regra reprovada e a evidência numérica que levou à reprovação. Esses atributos existem porque um alerta de qualidade só é útil se disser o que falhou e quanto falhou; uma mensagem genérica de "dado inválido" obrigaria alguém a refazer a investigação inteira. O erro de qualidade, assim, transporta junto com a falha a informação necessária para corrigi-la.

CONCLUSÃO — SÍNTESE

A síntese do domínio mostra que constantes e erros cumprem a mesma função: dar a todas as etapas um vocabulário comum. As constantes definem de que lugar, de que tributos e de que horário se fala; os erros definem que tipos de falha existem e que número cada uma devolve. Com esse vocabulário, um agendador consegue distinguir um servidor fora do ar de um dado inválido apenas pelo código de saída, e cada etapa consegue falar das mesmas coisas sem repeti-las.

11. Observabilidade: log e identificador

INTRODUÇÃO — O TODO

A observabilidade é a capacidade de saber, depois de uma execução, o que aconteceu dentro dela, e no pipeline ela é implementada pelo módulo `pipeline/observabilidade.py`**.** Essa capacidade é necessária porque o pipeline roda sem ninguém olhando; se uma coleta falhar às três da manhã, a única testemunha será o log. O módulo cumpre esse papel com cinco peças: o identificador da execução, o filtro que marca cada linha e mascara segredos, a configuração dos destinos do log, a medição das etapas e a identificação da versão do código. O desenvolvimento a seguir analisa cada uma dessas peças.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O identificador da execução

O identificador da execução combina a data e a hora do início com um sufixo aleatório, no formato `AAAAMMDDTHHMMSS-xxxxxx`**, como em** `20261021T065740-9b9054`**.** O identificador precisa existir porque cada execução deixa rastros em três lugares diferentes, o arquivo de log, a tabela `execucao_pipeline` do banco e o relatório de qualidade, e só um nome comum permite afirmar que esses rastros pertencem à mesma execução. A data vem no início porque um texto que começa pela data se ordena cronologicamente em qualquer listagem de arquivos. O sufixo, por sua vez, vem de `secrets.token_hex(3)`, que produz três bytes aleatórios em hexadecimal, e existe porque duas execuções podem começar no mesmo segundo, por exemplo num teste automatizado; com mais de dezesseis milhões de sufixos possíveis, a chance de repetição no mesmo segundo é desprezível. O identificador, assim, diz ao mesmo tempo quando a execução começou e qual execução ela é.

O filtro de contexto

O filtro de contexto é a peça que acrescenta o identificador da execução a cada linha do log e que, ao mesmo tempo, mascara segredos. Para acrescentar o identificador, o filtro preenche o campo `execucao` de cada registro, que o formato do log imprime entre colchetes. Esse filtro foi instalado nos destinos do log, e não no objeto de log principal, por uma razão técnica do Python: um filtro instalado no objeto de log só vê as mensagens escritas diretamente nele, enquanto um filtro instalado no destino vê também as mensagens dos objetos de log filhos, como `pipeline.coleta`. Se o filtro estivesse no lugar errado, as mensagens da coleta sairiam sem identificador e sem máscara. O filtro de contexto, por conseguinte, alcança todas as mensagens do pipeline porque está instalado no ponto por onde todas passam.

A máscara de segredos

A máscara de segredos é a segunda função do filtro de contexto e protege os valores de quatro variáveis sensíveis: `DB_PASSWORD`**,** `GITHUB_TOKEN`**,** `PSEUDONIMO_CHAVE` **e** `PI2_TOKEN_API`**.** Quando o filtro é criado, ele lê os valores dessas variáveis no ambiente; depois, se algum desses valores aparecer numa mensagem, ele é trocado por `***`. A máscara existe como defesa em profundidade, isto é, como uma segunda barreira para o caso de a primeira falhar: o código foi escrito para nunca escrever uma senha no log, mas um erro de programação futuro, como imprimir a configuração inteira para depurar um problema, poderia fazê-lo. Valores com menos de quatro caracteres não entram na máscara, porque um valor curto, como "ab", aparece por coincidência em muitas palavras e mascará-lo destruiria mensagens legítimas. Depois de mascarar, o filtro substitui a mensagem original pela mascarada e apaga os argumentos de formatação, porque, se os argumentos

ficassem, o formatador remontaria a mensagem original com o segredo. A máscara de segredos, assim, protege as credenciais mesmo contra os erros da própria equipe.

A configuração dos destinos

A configuração dos destinos define para onde as mensagens do log vão: para o terminal e para um arquivo. O arquivo fica em `logs/etl_<id>.log`, codificado em UTF-8 para que acentos e símbolos saiam corretos, e cada linha traz data, nível, identificador, origem e mensagem. Antes de criar os destinos, a configuração remove os destinos antigos, porque, quando o pipeline roda mais de uma vez no mesmo processo, como acontece nos testes, cada execução acrescentaria um novo destino, e cada mensagem passaria a ser escrita duas, três ou mais vezes. A configuração também desliga a propagação para o log raiz do Python, para que as mensagens não sejam repetidas por outro sistema de log que o ambiente tenha configurado. A configuração dos destinos, por conseguinte, garante que cada mensagem seja escrita exatamente uma vez em cada destino.

A medição das etapas

A medição das etapas é feita pelo gerenciador de contexto `etapa`, que envolve cada etapa do pipeline e registra seu início, seu fim e sua duração. Durante a etapa, o código pode acrescentar métricas a um dicionário, como o número de fontes coletadas ou de validações aprovadas, e essas métricas saem na linha final da etapa. Se a etapa falhar, o gerenciador registra a falha e o tempo decorrido até ela, e então repassa o erro ao orquestrador, sem engoli-lo. Esse comportamento existe porque um erro silencioso é o pior tipo de erro: a execução pareceria ter terminado bem, e o problema só seria descoberto quando alguém usasse um dado errado. A medição das etapas, assim, garante que nenhuma etapa termine sem deixar registro, nem com sucesso nem com falha.

A versão do código

A função `versao_codigo` identifica qual versão do código produziu a execução, consultando o Git. Ela obtém o identificador curto do commit atual e, se houver algum arquivo rastreado com alteração ainda não registrada em commit, acrescenta o sufixo "-modificado". Esse sufixo existe porque um commit identifica o código só quando o código não foi alterado depois dele; sem o sufixo, uma execução feita com alterações locais pareceria ter usado o código exato do commit. Quando o Git não está disponível, a função devolve "desconhecida", e é isso que acontece na pasta `PI2_Sprint3`, que não tem histórico: a execução continua normalmente, mas perde o vínculo com um commit. A versão do código, desse modo, completa a rastreabilidade ligando cada resultado não só a uma execução, mas também ao código que a produziu.

CONCLUSÃO — SÍNTESE

A síntese da observabilidade revela que suas cinco peças respondem a cinco perguntas que alguém faz depois de uma execução: qual execução foi, o que aconteceu em cada momento, quanto tempo cada etapa levou, com que código ela rodou e se algum segredo vazou. O identificador responde à primeira, os destinos e o filtro respondem à segunda, a medição responde à terceira, a versão responde à quarta e a máscara garante a resposta negativa à quinta. Por isso, o log dispensa a reconstrução de memória: tudo o que é preciso saber sobre uma execução está escrito, marcado e protegido.

12. Camadas, manifesto e depósito do bruto

INTRODUÇÃO — O TODO

O módulo `pipeline/camadas.py` define onde cada dado fica guardado e como o dado bruto é preservado. O dado bruto é o dado exatamente como a fonte o entregou, e o princípio central do módulo é que ele nunca é alterado nem sobrescrito. Esse princípio existe porque todas as etapas seguintes podem ser refeitas a partir do bruto, mas o bruto não pode ser refeito a partir delas: se ele se perder, a fonte pode ter mudado, e a versão original desaparece para sempre. O desenvolvimento a seguir analisa as camadas, o nome dos arquivos, os hashes, o manifesto com seus tipos de registro e o depósito que grava o bruto.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

As camadas

As camadas são as quatro pastas em que o pipeline organiza seus dados: `data/bruto`, `data/limpo`, `data/pronto` e `logs`. A camada bruta guarda o que veio das fontes; a camada limpa guarda as tabelas tipadas, uma por fonte; a camada pronta guarda as tabelas analíticas, prontas para painel e modelo; e a pasta de logs guarda a história de cada execução. Essa separação existe porque cada camada tem uma regra de vida diferente: o bruto só cresce e nunca muda, enquanto as camadas limpa e pronta são recriadas a cada execução a partir do bruto. O objeto `Camadas` define as quatro pastas a partir de uma raiz, que é a pasta do projeto em uso normal e uma pasta temporária nos testes. As camadas, assim, separam o que é evidência permanente do que é produto recriável.

O nome dos arquivos

O nome de cada arquivo bruto segue o padrão `fonte_assunto_AAAA-MM-DD`, com dois sufixos opcionais: `_pNNNN` para o número da página e `_vN` para uma nova versão no mesmo dia. Esse padrão existe porque um nome padronizado diz, sem abrir o arquivo, de onde ele veio, sobre o que trata e em que dia foi coletado. A página tem quatro dígitos, com zeros à esquerda, para que a ordem alfabética dos arquivos coincida com a ordem das páginas; sem os zeros, a página 10 apareceria antes da página 2. Fonte e assunto só aceitam letras minúsculas, dígitos e hífen, e a extensão só aceita minúsculas e dígitos, porque espaços, acentos e maiúsculas causam problemas diferentes em cada sistema operacional. Um nome fora do padrão levanta erro antes de qualquer gravação. O nome dos arquivos, por conseguinte, é ao mesmo tempo uma etiqueta legível e uma regra conferida pelo código.

Os hashes

Os hashes são impressões digitais do conteúdo dos arquivos, calculadas com o algoritmo SHA-256. A função `sha256_bytes` calcula o hash de um conteúdo que já está na memória, e a função `sha256_arquivo` calcula o hash de um arquivo lendo-o em blocos de 1 MiB. A leitura em blocos existe porque alguns arquivos brutos têm dezenas de megabytes, e carregá-los inteiros na memória só para calcular o hash seria um desperdício. O hash é usado porque qualquer alteração no conteúdo, mesmo de um único byte, produz um hash completamente diferente; assim, comparar dois hashes equivale a comparar dois arquivos inteiros. Os hashes, desse modo, permitem verificar a integridade do bruto sem guardar uma segunda cópia de cada arquivo.

O manifesto

O manifesto é o arquivo `data/bruto/manifesto.jsonl`, que registra, linha a linha, tudo o que entrou no bruto. O formato escolhido é o JSON Lines, em que cada linha é um objeto JSON completo, e o arquivo é usado apenas por acréscimo, nunca por reescrita. Esse formato foi escolhido porque acrescentar uma linha ao fim de um arquivo não toca nas linhas anteriores; assim, mesmo que o pipeline seja interrompido no meio de uma gravação, no máximo a última linha fica incompleta, e o histórico anterior permanece intacto. A serialização de cada linha é determinística, com chaves em ordem alfabética, sem espaços extras e com acentos preservados, de modo que o mesmo registro produz sempre os mesmos bytes, independentemente da máquina. O manifesto, por conseguinte, é o diário de bordo do bruto, escrito de forma que nenhuma entrada antiga possa ser estragada por uma nova.

A cadeia de hashes

A cadeia de hashes é o mecanismo que liga cada linha do manifesto à linha anterior. Cada linha guarda, no campo `sha256_anterior`, o hash da linha que a precede; a primeira linha, que não tem antecessora, usa uma sequência de 64 zeros. Esse encadeamento existe porque, sem ele, alguém poderia apagar ou alterar uma linha no meio do manifesto sem deixar vestígio; com ele, alterar uma linha muda o seu hash, e a linha seguinte deixa de apontar para ele. O método `verificar` percorre a cadeia inteira, confere se cada linha aponta para o hash da anterior e, para cada registro de arquivo, confere se o arquivo existe e se o seu SHA-256 é o registrado. A cadeia, porém, tem um limite declarado: ela não usa segredo, e por isso detecta alterações acidentais ou ingênuas, mas não detecta quem reescreve a cadeia inteira de propósito, recalculando todos os hashes. Para isso seria preciso uma assinatura com chave secreta, chamada MAC (Kurose; Ross, 2013, §8.3.2). A cadeia de hashes, assim, protege o bruto contra o erro e contra a adulteração descuidada, e o seu limite está escrito no próprio código.

Os três tipos de registro

O manifesto contém três tipos de registro, cada um para uma espécie de evento. O registro "arquivo" descreve um arquivo gravado e guarda o nome, a fonte, o assunto, o dia, a página, a URL, o status HTTP, o número de registros, o tamanho, o SHA-256, o instante da coleta, a execução e detalhes livres, como a etiqueta de versão do servidor. O registro "coleta" descreve uma coleta inteira de uma fonte e guarda a lista de arquivos, os registros esperados e os obtidos e a indicação de que a coleta está completa; a transformação só usa coletas completas, porque uma coleta parcial geraria resultados parciais sem aviso. O registro "tentativa" descreve um acesso recusado ou impossível e guarda a URL, o status e uma observação; é assim que a recusa do portal NUCLEOGOV fica documentada, porque uma recusa também é um fato que precisa de prova. Os três tipos de registro, por conseguinte, documentam não só o que o pipeline obteve, mas também o que tentou e não conseguiu.

A gravação no depósito

A gravação no depósito é feita pelo método `gravar`, que guarda um conteúdo no bruto sem jamais sobrescrever um arquivo existente. O método calcula o SHA-256 do conteúdo, pede ao método `_destino` um nome livre e abre o arquivo em modo de criação exclusiva, um modo do sistema operacional que falha se o arquivo já existir. O método `_destino` escolhe o nome tentando a versão 1, depois a 2, e assim por diante; se encontrar um arquivo do mesmo dia com o mesmo SHA-256, devolve o registro existente em vez de criar outra cópia. O modo exclusivo é usado mesmo

depois de `_destino` ter verificado que o nome está livre, porque ele é uma segunda garantia, dada pelo próprio sistema operacional: se um defeito ou uma execução simultânea fizer dois gravadores escolherem o mesmo nome, o segundo falha em vez de apagar o primeiro. A gravação no depósito, desse modo, torna fisicamente impossível que o bruto seja sobrescrito pelo pipeline.

A promoção de um download

A promoção de um download é feita pelo método `gravar_de_temporario`, usado para os arquivos grandes, que são baixados para um arquivo temporário antes de serem aceitos. Esse método calcula o hash do temporário, escolhe o nome definitivo pelo mesmo `_destino` e renomeia o temporário para esse nome. O download vai primeiro para um temporário porque um arquivo grande pode chegar truncado ou corrompido, e só depois de conferido ele deve ganhar o nome definitivo; se a conferência falhar, o bruto nunca chega a conter o arquivo defeituoso. Se o conteúdo baixado for idêntico a um arquivo já guardado no mesmo dia, o temporário é apagado e o registro existente é reaproveitado. A promoção de um download, assim, garante que só entre no bruto o que já foi conferido.

A data do nome

A data que vai no nome dos arquivos é a data de Palmas, e não a do relógio universal. O método `dia_local` converte o instante da coleta, que o computador fornece em UTC, para o fuso de Palmas, UTC−3. Essa conversão é necessária porque, sem ela, uma coleta feita às 22h em Palmas, que corresponde à 1h do dia seguinte em UTC, receberia no nome a data de amanhã. Esse erro não seria apenas estético: a regra que reaproveita os arquivos já coletados "hoje" procura pela data registrada no manifesto, que é a mesma do nome, e uma data errada faria o pipeline pedir de novo páginas que já tinha. A data do nome, por conseguinte, coincide com o dia que a equipe percebe, e a regra de reaproveitamento funciona em qualquer horário.

O registro das coletas

O registro das coletas é feito pelo método `registrar_coleta`, que acrescenta ao manifesto o resumo de cada coleta concluída. Antes de acrescentar, o método compara o resumo com a última coleta registrada para a mesma fonte e o mesmo assunto; se os dados forem idênticos, ele devolve o registro existente em vez de criar uma linha nova. Essa comparação existe porque o pipeline pode ser executado várias vezes no mesmo dia, reaproveitando os arquivos já coletados, e cada execução registraria uma coleta idêntica à anterior, inflando o manifesto com linhas que não acrescentam informação. O método `coleta_mais_recente`, por sua vez, devolve a última coleta completa de uma fonte e levanta `ErroColeta` se não houver nenhuma, porque sem ela a transformação não tem entrada. O registro das coletas, assim, mantém o manifesto enxuto e fornece à transformação um ponto de partida inequívoco.

CONCLUSÃO — SÍNTESE

A síntese deste módulo revela que todas as suas partes servem a uma mesma garantia: o bruto é imutável e rastreável. O nome padronizado diz o que cada arquivo é; o hash prova que ele não mudou; o manifesto registra sua chegada; a cadeia impede que esse registro seja alterado sem vestígio; e o modo exclusivo impede que o arquivo seja sobrescrito. Por causa dessa garantia, qualquer resultado posterior do pipeline pode ser refeito a partir do bruto, e qualquer pessoa pode conferir que o bruto usado é o mesmo que a fonte entregou.

13. Política de coleta

INTRODUÇÃO — O TODO

A política de coleta, no módulo `pipeline/coleta/politica.py`, é o conjunto de regras que decide como o pipeline deve se comportar diante de um servidor: que endereços aceitar, quando repetir um pedido e quanto esperar antes de repeti-lo. Essas regras foram separadas num módulo próprio porque são decisões, e não operações; nenhuma função do módulo acessa a rede, e todas recebem valores e devolvem valores. Essa pureza permite testar cada regra com números fixos, sem servidor, e ter certeza de que a regra está certa antes de ela ser aplicada a um servidor real. O desenvolvimento a seguir analisa as regras em quatro grupos: a identificação do servidor, o transporte seguro, a classificação das falhas e o cálculo das esperas.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O host de uma URL

A função `host_de` extrai de uma URL o nome do servidor, chamado host, e o converte para letras minúsculas. A conversão existe porque nomes de servidor não distinguem maiúsculas de minúsculas: `IBGE.gov.br` e `ibge.gov.br` são o mesmo servidor. Esse detalhe importa porque o host é a chave usada para controlar o ritmo dos pedidos e para associar credenciais; sem a conversão, um mesmo servidor poderia ser contado como dois, e o pipeline o consultaria duas vezes mais depressa do que a política permite. A função `host_de`, assim, garante que cada servidor seja reconhecido como um só, qualquer que seja a grafia da URL.

A exigência de HTTPS

A função `exigir_https` só aceita URLs que usam HTTPS, a versão do HTTP protegida por criptografia TLS. Essa exigência existe porque, sem TLS, qualquer intermediário no caminho da rede, como um roteador comprometido ou uma rede Wi-Fi pública, pode ler e alterar o dado em trânsito sem que o pipeline perceba (Kurose; Ross, 2013, §8.6). Uma única exceção é permitida: URLs HTTP para hosts de teste declarados explicitamente na política, como o servidor local `127.0.0.1` usado nos testes automatizados, que não passam pela rede. Qualquer outra URL sem TLS levanta `ErroSeguranca`. A exigência de HTTPS, por conseguinte, transforma a segurança do transporte de uma boa prática em uma condição para o pedido existir.

A promoção dos links de paginação

A função `promover_https` corrige os links de paginação que um servidor devolve em HTTP. Essa correção é necessária porque a API de arrecadação da Prefeitura, embora acessada por HTTPS, devolve o link da página seguinte, no campo `next`, começando por `http://`. Seguir esse link exporia a coleta exatamente ao risco que a exigência de HTTPS quer evitar; recusá-lo, por outro lado, impediria a paginação. A função resolve o impasse trocando `http://` por `https://` no mesmo host e removendo a porta 80 do endereço, se ela aparecer, porque a porta 80 é a do HTTP sem criptografia. A função, porém, recusa com `ErroSeguranca` um link que aponte para outro host, porque um servidor comprometido poderia usar o link de paginação para conduzir o cliente a um servidor falso. A promoção dos links, assim, preserva a paginação sem abrir mão da segurança.

Os status transitórios

Os status transitórios são os códigos de resposta HTTP que indicam uma falha passageira do servidor: 429, 500, 502, 503 e 504. O código 429 significa que o cliente fez pedidos demais e deve esperar (RFC 6585); o 500 indica um erro interno do servidor; o 502 e o 504 indicam que um servidor intermediário não obteve resposta a tempo; e o 503 indica que o serviço está temporariamente indisponível (RFC 9110). Só esses códigos justificam nova tentativa, porque só eles descrevem uma situação que pode se resolver sozinha com o tempo. Os demais erros não são repetidos: um 403 é uma recusa do servidor, um 404 indica um recurso inexistente, e repetir uma recusa seria insistência, e não robustez. Os status transitórios, por conseguinte, separam o que o tempo resolve do que exige uma correção humana.

A espera exponencial

A função `espera_exponencial` **calcula quanto tempo esperar antes de uma nova tentativa, sorteando um valor uniforme entre zero e o menor entre o teto e a base multiplicada por 2 elevado ao número da tentativa, contado a partir de zero.** O crescimento exponencial existe porque, se o servidor está sobrecarregado, pedidos repetidos em intervalos curtos agravam a sobrecarga; dobrar a espera a cada falha dá ao servidor cada vez mais tempo para se recuperar. O sorteio existe por uma razão diferente: se vários clientes falharem ao mesmo tempo e esperarem exatamente o mesmo intervalo, voltarão todos juntos e provocarão nova sobrecarga. Essa é a mesma ideia do recuo exponencial binário do Ethernet, em que estações que colidiram sorteiam o momento de transmitir de novo (Metcalfe; Boggs, 1976; Kurose; Ross, 2013, §5.3.2). Com base de 1 s e teto de 60 s, as esperas máximas das cinco tentativas são 1, 2, 4, 8 e 16 s, e o teto impede que uma sequência longa de falhas produza esperas absurdas. A espera exponencial, assim, combina paciência crescente com dispersão dos clientes.

O cabeçalho Retry-After

A função `segundos_retry_after` **interpreta o cabeçalho Retry-After, com o qual o servidor diz explicitamente quanto tempo o cliente deve esperar.** O cabeçalho pode vir em dois formatos, um número inteiro de segundos ou uma data HTTP, e a função aceita os dois (RFC 9110, §10.2.3). Quando o valor está ausente ou ilegível, a função devolve nada, e então o cliente usa a espera exponencial. Uma data no passado vira espera zero, porque uma espera negativa não tem sentido, e uma data sem fuso é tratada como UTC, porque as datas HTTP são, por definição, expressas nesse horário. O cabeçalho tem prioridade sobre o sorteio porque o servidor conhece a própria situação melhor do que qualquer estimativa do cliente. A função `segundos_retry_after`, por conseguinte, faz o pipeline obedecer ao servidor sempre que ele diz o que precisa.

Os cabeçalhos de limite de taxa

A função `segundos_limite_taxa` **lê os cabeçalhos** `X-RateLimit-Remaining` **e** `X-RateLimit-Reset`**, usados pela API do GitHub para informar a cota de pedidos.** A função só age quando a cota restante é exatamente zero, porque antes disso o cliente ainda pode fazer pedidos; nesse caso, ela calcula a espera até o instante de reinício informado pelo servidor, expresso em segundos desde 1970. Esses cabeçalhos são diferentes do Retry-After porque não indicam uma falha passageira, e sim um limite contratual de uso. A função dos limites de taxa, assim, permite ao pipeline respeitar uma cota que, se ultrapassada, poderia bloquear o acesso por muito mais tempo.

CONCLUSÃO — SÍNTESE

A síntese da política de coleta mostra que todas as suas regras respondem a uma mesma pergunta: como obter os dados sem prejudicar o servidor e sem confiar no que não foi verificado. A identificação correta do host evita pedidos acelerados por engano; o HTTPS e a recusa de troca de host protegem contra intermediários e servidores falsos; a classificação dos status separa o passageiro do definitivo; e as esperas obedecem ao servidor quando ele fala e o poupam quando ele cala. Como todas essas regras são funções puras, cada uma pode ser testada com valores fixos, e o comportamento do pipeline diante de um servidor real deixa de ser surpresa.

14. Consulta ao robots.txt

INTRODUÇÃO — O TODO

A consulta ao robots.txt, no módulo `pipeline/coleta/robots.py`, é o mecanismo pelo qual o pipeline lê as regras que cada servidor publica para coletores automáticos. O robots.txt é um arquivo de texto na raiz de cada servidor, padronizado pela RFC 9309, em que o operador declara o que pode e o que não pode ser coletado. O pipeline consulta esse arquivo antes do primeiro acesso a cada servidor porque a coleta só é legítima se respeitar a vontade de quem opera o servidor. O desenvolvimento a seguir analisa a interpretação do status da resposta, o caso particular do NUCLEOGOV e o verificador que aplica as regras.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A interpretação do status da resposta

A interpretação do status da resposta decide o que fazer conforme o servidor responde ao pedido do robots.txt, e segue a RFC 9309 (§2.3.1). Um status da família 2xx significa que o arquivo existe e traz regras; essas regras valem para o User-Agent do pipeline, e a situação registrada é REGRAS. Um status da família 4xx significa que o arquivo não existe ou não está disponível; nesse caso, a norma entende que o operador não publicou restrições, o coletor pode acessar qualquer recurso, e a situação registrada é INDISPONIVEL. Um status da família 5xx ou uma falha de rede significa que o servidor está com problemas; nesse caso, a norma manda supor proibição total, porque o coletor não sabe o que o operador quis dizer, e na dúvida deve abster-se. A situação registrada é INALCANCAVEL, e nenhum recurso do host é coletado. A interpretação do status, por conseguinte, transforma a resposta do servidor numa decisão que segue a norma, e não a conveniência do coletor.

O caso do NUCLEOGOV

O portal NUCLEOGOV responde com o status 403, de recusa, até ao pedido do robots.txt, e esse caso mostra a diferença entre a regra técnica e a conduta ética. Pela RFC 9309, um 403 no robots.txt pertence à família 4xx e, portanto, libera o acesso, porque significa apenas que o arquivo de regras não está disponível. O pipeline, porém, não para nessa leitura: quando o próprio recurso, a página do portal, também responde 403, essa resposta é uma recusa direta de acesso, e o pipeline não a contorna. A norma do robots.txt diz o que o coletor pode tentar; a resposta do recurso diz o que o servidor aceita. O caso do NUCLEOGOV, assim, demonstra que o pipeline obedece às duas camadas de vontade do operador, e não apenas à mais permissiva.

O verificador e o seu cache

O verificador, a classe `VerificadorRobots`, consulta o robots.txt de cada host uma única vez e guarda o resultado num dicionário indexado pelo host. O cache existe porque o robots.txt muda raramente, e consultá-lo antes de cada página dobraria o número de pedidos ao servidor, o oposto da cortesia que ele próprio pede. Quando a consulta traz regras, o verificador as entrega ao analisador da biblioteca padrão do Python, `RobotFileParser`, e decodifica o texto com substituição de caracteres inválidos, porque um robots.txt com um byte estranho não deve derrubar a coleta inteira. O verificador também guarda a resposta completa de cada host, que depois é gravada no bruto como evidência datada. O verificador, por conseguinte, aplica as regras com o mínimo de pedidos e deixa prova do que leu.

A decisão sobre cada URL

O método `permitido` decide se uma URL específica pode ser coletada, a partir da situação do host. Se a situação é INDISPONIVEL, tudo é permitido; se é INALCANCAVEL, tudo é proibido; se é REGRAS, o método pergunta ao analisador se o User-Agent do pipeline pode acessar aquela URL. A decisão é tomada por URL, e não por host, porque um operador pode liberar uma parte do site e proibir outra, por exemplo permitindo a API e proibindo a área administrativa. O método `permitido`, assim, respeita as regras no nível de detalhe em que o operador as escreveu.

O atraso pedido pelo operador

O método `atraso_pedido` devolve o Crawl-delay, o intervalo mínimo entre pedidos que o operador pode declarar no robots.txt. Esse método tem uma particularidade: ele só devolve o atraso já conhecido e nunca dispara uma consulta nova. A particularidade existe para evitar uma recursão infinita: o cliente consulta o atraso antes de cada pedido para respeitar o ritmo, inclusive antes do pedido do próprio robots.txt; se a consulta do atraso disparasse a busca do robots.txt, essa busca consultaria o atraso, que dispararia de novo a busca, sem fim. O método `atraso_pedido`, por conseguinte, quebra o ciclo usando apenas o que já foi lido.

CONCLUSÃO — SÍNTESE

A síntese da consulta ao robots.txt revela que suas partes formam um único compromisso: ler a vontade do operador antes de agir e registrar o que foi lido. A interpretação do status segue a norma, o caso do NUCLEOGOV mostra que a norma é piso e não teto, o cache evita sobrecarregar o servidor, a decisão por URL respeita o detalhe das regras, e o atraso pedido é obedecido sem criar ciclos. A resposta de cada host fica gravada no bruto, de modo que qualquer pessoa pode conferir, depois, que a coleta respeitou o que o servidor publicou naquele dia.

15. Cliente HTTP

INTRODUÇÃO — O TODO

O cliente HTTP, a classe `ClienteHttp` no módulo `pipeline/coleta/http.py`, é o componente pelo qual passa toda requisição do pipeline. Esse componente aplica, a cada pedido, as regras da política de coleta: exige HTTPS, consulta o robots.txt, respeita o ritmo por servidor, repete as falhas passageiras e envia credenciais só ao servidor a que pertencem. O cliente é único e compartilhado por todas as fontes porque, se cada fonte tivesse o próprio cliente, bastaria uma delas esquecer uma regra para que a cortesia ou a segurança fosse quebrada. O desenvolvimento a seguir analisa a política que o cliente recebe, as respostas que ele devolve, a injeção de dependências, o método `obter` e cada um dos mecanismos que esse método aciona.

A política do cliente

A política do cliente é o objeto `PoliticaHttp`, que reúne os valores numéricos e textuais das regras. O intervalo mínimo entre pedidos ao mesmo servidor é de 1 s; o tempo máximo para estabelecer a conexão é de 15 s, e o tempo máximo de espera por dados durante a leitura é de 120 s, um valor folgado para que um servidor lento não seja confundido com um servidor fora do ar. São cinco tentativas, com espera base de 1 s, teto de 60 s e espera máxima aceita de 300 s. O objeto é imutável, porque uma política que pudesse ser alterada no meio da execução deixaria de ser uma garantia. O método `do_ambiente` monta a política real: permite acrescentar ao User-Agent um contato institucional, pela variável `PI2_CONTATO_COLETA`, e associa o host `api.github.com` à variável `GITHUB_TOKEN`, de modo que nenhum outro servidor receba credencial. A política do cliente, assim, concentra num objeto imutável todos os números que governam a coleta.

O User-Agent

O User-Agent é o texto com que o pipeline se apresenta a cada servidor: `PI2-UFT-Sefin-ETL/0.3 (projeto academico UFT/BIINAR, Eixo II)`. Esse texto identifica o projeto, a versão e a instituição, porque o operador de um servidor tem o direito de saber quem o está acessando e, se quiser, de bloquear ou contatar o responsável. O User-Agent não imita um navegador, porque isso enganaria o operador sobre a natureza automática do acesso, e não leva e-mail pessoal, porque o endereço de um integrante não deve circular em servidores de terceiros. O contato opcional existe para quando a equipe quiser oferecer um canal institucional. O User-Agent, por conseguinte, é a forma como o pipeline cumpre o dever de honestidade em cada pedido.

As respostas

As respostas do cliente são representadas por dois objetos. O objeto `RespostaBruta`, produzido pelo transporte, guarda o status, os cabeçalhos com os nomes em minúsculas, o conteúdo, a URL final e o número de bytes recebidos; o conteúdo fica vazio quando o download vai direto para um arquivo, porque um arquivo grande não deve ser duplicado na memória. O objeto `Resposta`, devolvido às fontes, acrescenta a duração da transferência e o número de tentativas necessárias. A duração mede só a transferência, e não a pausa de cortesia que a precede, porque misturar as duas faria um servidor rápido parecer lento; esse cuidado foi acrescentado depois que a leitura do log revelou a mistura. As respostas, assim, carregam o que as fontes precisam e o que o log precisa, sem confundir os dois.

A injeção de dependências

A injeção de dependências é a técnica pela qual o cliente recebe de fora o transporte que faz o pedido real e as funções de dormir, de relógio, de sorteio e de hora. Essa técnica segue o princípio da inversão de dependências: o cliente depende de um protocolo, isto é, de uma descrição do que o transporte deve fazer, e não de uma biblioteca concreta (Valente, 2020, §5.6.3). A consequência prática é que, nos testes, o transporte real é substituído por um que devolve respostas gravadas, a função de dormir é substituída por uma que apenas anota quanto se pediu para dormir, e o sorteio é fixado num valor conhecido. O relógio usado para medir intervalos é o monotônico, que nunca anda para trás, porque o relógio comum pode ser ajustado pelo sistema durante a execução e produzir intervalos negativos; a hora comum só é usada para o limite de taxa, cujo reinício o servidor informa nesse formato. A injeção de dependências, por conseguinte, permite testar em milésimos de segundo um comportamento que, com rede real, levaria minutos e

dependeria da sorte.

O método obter

O método `obter` executa um pedido GET com a política completa, numa sequência de passos cuja ordem tem razão de ser. Primeiro, o método exige HTTPS, antes de qualquer outra ação, para que nem mesmo a consulta ao robots.txt seja feita por um canal inseguro. Depois, consulta o robots.txt e levanta `ErroColeta` se a URL não for permitida, porque não se deve gastar um pedido com um recurso proibido. Em seguida, monta os cabeçalhos, combinando o User-Agent, a credencial do host, se houver, e os cabeçalhos extras pedidos pela fonte. Só então entra no laço de tentativas, que se repete até cinco vezes: antes de cada tentativa, o cliente aguarda a vez do servidor; depois, faz o pedido e mede a duração. O método `obter`, assim, só chega a tocar o servidor depois de ter verificado tudo o que podia ser verificado localmente.

O laço de tentativas

O laço de tentativas decide o que fazer com cada resultado de pedido. Se o pedido falha por um problema de rede passageiro, como conexão recusada ou tempo esgotado, o cliente espera e tenta de novo. Se o pedido recebe uma resposta, o cliente a registra no log; se o status está entre os aceitos pela fonte, normalmente apenas o 200, a resposta é devolvida. Qualquer outro status passa pela decisão de espera, que pode mandar esperar e repetir ou interromper a fonte. Depois de cinco tentativas sem sucesso, o cliente desiste e levanta `ErroColeta` com a mensagem "tentativas sem sucesso", e a fonte é marcada como falha. A releitura do código, porém, encontrou um detalhe: depois da quinta falha, o cliente ainda executa a espera e anuncia no log uma "nova tentativa" que não acontece, como se discute na seção sobre as observações da releitura. O laço de tentativas, por conseguinte, insiste apenas o necessário e desiste de forma explícita.

A decisão de espera

A decisão de espera, no método `_espera_para`, determina quanto esperar antes da próxima tentativa, e examina a resposta numa ordem precisa. Primeiro, verifica os cabeçalhos de limite de taxa, e essa verificação vem antes de qualquer outra por um motivo concreto: a API do GitHub sinaliza a cota esgotada com o status 403, que normalmente seria uma recusa definitiva; se o status fosse examinado primeiro, o cliente desistiria de uma fonte que só precisava esperar. Em seguida, se o status não é transitório, o método levanta `ErroHttp` com o status e a URL, e a fonte é interrompida sem repetição. Por fim, se o status é transitório, o método usa o Retry-After, quando o servidor o envia, ou a espera exponencial sorteada, quando não envia. A decisão de espera, assim, lê a resposta na ordem que evita confundir uma cota esgotada com uma recusa.

O limite da espera aceitável

O limite da espera aceitável é a regra que interrompe a fonte quando o servidor pede uma espera maior que 300 s. Essa regra existe porque o pipeline coleta sete fontes em sequência, e uma única fonte que pedisse uma hora de espera prenderia toda a execução, inclusive as fontes que estavam disponíveis. Ao interromper apenas aquela fonte, o cliente permite que as demais sejam coletadas, e a falha fica registrada para ser tratada depois. Quando a espera é aceita, o cliente escreve no log um aviso com o motivo, a duração da espera e o número da tentativa, e só então dorme. O limite da espera, por conseguinte, impede que a cortesia com um servidor se transforme em prejuízo para a coleta inteira.

O ritmo por servidor

O ritmo por servidor é controlado pelo método `_aguardar_vez`, que garante o intervalo mínimo entre dois pedidos ao mesmo host. O intervalo usado é o maior entre 1 s, o mínimo da política, e o Crawl-delay que o operador eventualmente declarou no robots.txt, porque o pedido do operador deve prevalecer quando é mais exigente que a política. O método calcula quanto tempo passou desde o último acesso àquele host e dorme apenas o que falta; o instante do acesso é registrado depois da espera, porque o intervalo deve contar a partir do momento em que o pedido realmente sai. O método devolve quanto tempo esperou, e esse valor aparece no log como pausa de cortesia. O ritmo por servidor, assim, faz com que as 36 páginas da API da Prefeitura levem cerca de 36 segundos, carga desprezível para o servidor público.

As credenciais

As credenciais são acrescentadas aos cabeçalhos pelo método `_cabecalhos_base`, que só envia um token ao host associado a ele na política. O token é lido do ambiente no momento de cada pedido, e não guardado na política ou no cliente, porque um valor guardado num objeto pode acabar impresso por engano quando alguém examina esse objeto durante uma depuração. Se o host não tem variável associada, ou se a variável está vazia, nenhum cabeçalho de autorização é enviado. Essa restrição por host existe porque enviar o token do GitHub a outro servidor entregaria a um terceiro a chave de acesso da equipe. As credenciais, por conseguinte, vão apenas a quem pertencem e não ficam guardadas em lugar algum além do ambiente.

A busca do robots.txt

A busca do robots.txt, no método `_buscar_robots`, é o pedido especial que o verificador usa para obter as regras de cada servidor. Esse pedido difere dos outros em três pontos. Primeiro, ele não consulta o próprio robots.txt, porque perguntar ao robots.txt se é permitido ler o robots.txt seria circular. Segundo, ele faz uma única tentativa, porque uma falha na busca já tem um significado definido pela norma, servidor inalcançável e, portanto, proibição total, e repetir a busca apenas aumentaria a carga sobre um servidor com problemas. Terceiro, ele respeita o ritmo do host e exige HTTPS, como qualquer outro pedido. Quando a busca falha por problema de rede, o método devolve status nulo e conteúdo vazio, e o verificador interpreta isso como "inalcançável". A busca do robots.txt, assim, é o único pedido que dispensa as regras que ele mesmo serve para descobrir, e apenas essas.

O contexto TLS

O contexto TLS, criado pela função `contexto_tls`, é a configuração de segurança usada em toda conexão HTTPS. A função parte do contexto padrão do Python, que já liga a verificação do certificado e a verificação do nome do servidor, e acrescenta a exigência de TLS 1.2 ou superior, porque a RFC 8996 descontinuou o TLS 1.0 e o 1.1, versões com fraquezas conhecidas. Depois, a função confere explicitamente se as duas verificações estão ligadas e levanta `ErroSeguranca` se não estiverem. Essa conferência parece redundante, mas protege contra uma mudança futura no código ou na biblioteca que enfraqueça o padrão sem que ninguém perceba; com ela, uma configuração insegura interrompe a coleta em vez de passar despercebida. O contexto TLS, por conseguinte, garante que o pipeline só converse com o servidor verdadeiro e por um canal atual.

O registro de cada resposta no log

O registro de cada resposta no log é feito pelo método `_registrar`, que escreve uma linha para cada pedido respondido. A linha traz a URL, o status, o tamanho em kilobytes, a duração da transferência, a pausa de cortesia e o número da tentativa. Cada espera também gera uma linha de aviso, com o motivo e o tempo. Esse nível de detalhe existe porque, quando uma coleta demora ou falha, a primeira pergunta é qual pedido demorou e por quê, e a resposta precisa estar no log. O registro das respostas, assim, transforma cada pedido em evidência verificável do comportamento do cliente.

CONCLUSÃO — SÍNTESE

A síntese do cliente HTTP mostra que todos os seus mecanismos convergem para um mesmo resultado: nenhuma fonte consegue pular uma regra, porque todas passam pelo mesmo caminho. A política fixa os números, o User-Agent identifica, o método `obter` verifica antes de pedir, o laço insiste só no que é passageiro, o ritmo poupa o servidor, as credenciais vão só ao dono, e o contexto TLS garante o canal. A injeção de dependências, por fim, permite provar por testes que cada um desses mecanismos funciona, sem depender da internet.

16. Transporte HTTP

INTRODUÇÃO — O TODO

O transporte HTTP, no módulo `pipeline/coleta/transporte.py`, é a peça que faz o pedido real à rede, e é o único módulo da coleta que conhece a biblioteca `requests`. Esse isolamento é deliberado: o cliente decide o que pedir e quando, e o transporte apenas executa o pedido e devolve a resposta num formato neutro. Se um dia a biblioteca `requests` precisar ser trocada, só este módulo muda, e todas as regras do cliente continuam valendo. O desenvolvimento a seguir analisa a sessão persistente, o adaptador de segurança, o controle dos redirecionamentos, a classificação dos erros, a leitura da resposta e a conferência do tamanho.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A sessão persistente

A sessão persistente é o objeto `requests.Session`, que o transporte cria uma única vez e usa para todas as fontes. A sessão mantém abertas as conexões com cada servidor e as reaproveita nos pedidos seguintes, em vez de abrir uma conexão nova a cada página (Kurose; Ross, 2013, §2.2.2). Essa reutilização importa porque abrir uma conexão HTTPS exige uma troca de mensagens para estabelecer a conexão e outra para negociar a criptografia; repetir essas trocas a cada uma das 36 páginas da API seria um custo inútil para o pipeline e para o servidor. A sessão persistente, assim, reduz a carga da coleta sem alterar o que é coletado.

O adaptador de segurança

O adaptador de segurança, a classe `AdaptadorTls`, instala na sessão o contexto TLS criado pela função `contexto_tls`. A biblioteca `requests` já verifica certificados por padrão, mas não permite, de forma direta, exigir uma versão mínima do TLS; o adaptador resolve essa limitação entregando ao gerenciador de conexões o contexto do projeto, com TLS 1.2 ou superior. Todo pedido HTTPS da sessão passa por esse adaptador, e por isso a verificação do certificado não pode ser desligada por nenhuma fonte. O adaptador de segurança, por conseguinte, faz a política de criptografia valer no nível mais baixo da comunicação.

O controle dos redirecionamentos

O controle dos redirecionamentos confere todo o caminho que um pedido percorreu antes de chegar à resposta final. Um servidor pode responder a um pedido com um redirecionamento para outro endereço, e a biblioteca segue esses redirecionamentos automaticamente; o transporte, então, examina a lista de todos os endereços visitados. Se o pedido começou em HTTPS e algum endereço do caminho está em HTTP, a resposta é descartada e o transporte levanta `ErroSeguranca`. Essa conferência existe porque um redirecionamento para HTTP rebaixaria a segurança da conexão sem que o cliente tivesse pedido isso, e o dado recebido por esse caminho poderia ter sido alterado. A biblioteca já retira o cabeçalho de autorização quando o caminho sai do HTTPS, de modo que a credencial não vaza; o descarte da resposta protege, além da credencial, o próprio dado. O controle dos redirecionamentos, assim, impede que a segurança exigida no primeiro pedido se perca no meio do caminho.

A classificação dos erros

A classificação dos erros traduz as exceções da biblioteca `requests` em erros que o cliente sabe tratar. Uma falha na verificação TLS vira `ErroSeguranca`, porque um certificado inválido não se corrige sozinho e pode indicar um ataque; repetir o pedido seria inútil ou perigoso. Uma conexão recusada, um tempo esgotado ou um corpo de resposta interrompido viram `FalhaTransitoria`, porque são problemas de rede que costumam se resolver com o tempo, e o cliente decide se repete. A ordem dos blocos de tratamento importa: na biblioteca `requests`, o erro de TLS é um caso particular do erro de conexão, e, se o bloco da conexão viesse primeiro, uma falha de certificado seria tratada como passageira e repetida. A classificação dos erros, por conseguinte, garante que um problema de confiança nunca seja confundido com um problema de rede.

A leitura da resposta

A leitura da resposta segue dois caminhos, conforme o pedido tenha ou não um arquivo de destino. Sem destino, a resposta é lida inteira na memória; esse caminho vale para as respostas pequenas, como JSON, HTML e robots.txt, e vale também para qualquer status diferente de 200, inclusive quando havia um destino. A exceção para os status diferentes de 200 existe porque uma página de erro ou uma resposta 304 não deve ser gravada no arquivo temporário onde se esperava o conteúdo verdadeiro. Com destino e status 200, o corpo é escrito no arquivo em blocos de 1 MiB, à medida que chega, porque os arquivos do IBGE têm dezenas de megabytes e não precisam passar inteiros pela memória. Em ambos os caminhos, os nomes dos cabeçalhos são convertidos para minúsculas, porque o HTTP não distingue maiúsculas nesses nomes, e a conversão permite procurá-los com uma única grafia. A leitura da resposta, assim, adapta o uso da memória ao tamanho do que chega.

A conferência do tamanho

A conferência do tamanho compara a quantidade de bytes recebida com o valor do cabeçalho Content-Length, em que o servidor declara o tamanho do que enviou. Essa conferência existe porque uma conexão pode ser interrompida no meio de um download sem gerar erro, e o arquivo truncado pareceria completo (Kurose; Ross, 2013, §8.6). A comparação usa os bytes que passaram pela rede, e não os bytes gravados no arquivo, por uma razão técnica: o servidor pode enviar o conteúdo compactado, e o Content-Length descreve o tamanho compactado, enquanto o arquivo gravado contém o conteúdo já descompactado. Comparar o tamanho descompactado com o declarado acusaria truncamento onde não houve. Quando os valores não conferem, o transporte

levanta `FalhaTransitoria`, e o cliente repete o download. A conferência do tamanho, por conseguinte, detecta o download incompleto que nenhum erro de rede anunciou.

CONCLUSÃO — SÍNTESE

A síntese do transporte revela que sua função é tornar o contato com a rede seguro e eficiente sem que o restante do pipeline precise saber como. A sessão economiza conexões, o adaptador impõe a criptografia, o controle dos redirecionamentos impede o rebaixamento, a classificação dos erros separa confiança de disponibilidade, e a leitura e a conferência garantem que o que chega está completo. Como toda essa complexidade fica confinada num único módulo, trocar a biblioteca HTTP exigiria reescrever só este arquivo.

17. Interface das fontes e paginação

INTRODUÇÃO — O TODO

A interface das fontes, no módulo `pipeline/coleta/fonte_base.py`, define o que toda fonte de dados precisa oferecer ao pipeline e contém o laço de paginação que as fontes paginadas compartilham. Essa interface existe porque o pipeline coleta sete fontes muito diferentes, e o orquestrador não deveria precisar conhecer os detalhes de cada uma; ele precisa apenas pedir "colete" e receber um resumo. O laço de paginação comum, por sua vez, existe porque a lógica de percorrer páginas é igual em todas as fontes paginadas, e escrevê-la uma única vez evita que um defeito corrigido numa fonte continue presente nas outras. O desenvolvimento a seguir analisa a interface, a leitura de JSON, o laço de paginação, a regra do total estável e o reaproveitamento das páginas.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A interface Fonte

A interface `Fonte` é uma classe abstrata que define dois atributos e um método. Os atributos são `fonte` e `assunto`, que formam o início do nome dos arquivos brutos; o método é `coletar`, que recebe o cliente HTTP e o depósito do bruto e devolve um resumo da coleta. O orquestrador só conhece essa interface, e cada fonte concreta é um adaptador que traduz o protocolo particular de um servidor para essa forma comum (Valente, 2020, §6.5). A consequência é que acrescentar uma fonte nova não exige mudar o orquestrador: basta escrever um novo adaptador e incluí-lo no catálogo. A interface `Fonte`, assim, é o ponto de encaixe que permite ao pipeline crescer sem se reorganizar.

A leitura de JSON

A função `ler_json` converte os bytes recebidos de um servidor em objeto Python, exigindo duas condições: que o texto esteja em UTF-8 e que seja JSON válido. Qualquer outra coisa vira `ErroColeta`. Essa exigência é importante porque um servidor com problemas às vezes devolve uma página de erro em HTML com o status 200, como se tudo estivesse certo; sem a verificação, essa página seria gravada no bruto como se fosse dado. A leitura acontece antes da gravação, de modo que o bruto nunca recebe uma resposta que não pôde ser interpretada. A função `ler_json`, por conseguinte, impede que uma falha disfarçada de sucesso contamine o bruto.

O laço de paginação

O laço de paginação, na classe `ColetaPaginadaJson`, é escrito uma única vez e reutilizado por todas as fontes paginadas, segundo o padrão chamado método-modelo (Valente, 2020, §6.10). Nesse padrão, a classe comum define a sequência de passos, e cada fonte preenche apenas quatro lacunas: qual é a primeira página, onde estão os registros, qual é o total anunciado e qual é a próxima página. O laço começa pela primeira página e, para cada página, obtém o JSON, conta os registros, confere o total e pede à fonte o endereço da página seguinte. O laço para quando a fonte não indica próxima página. Há também um limite de 1.000 páginas, e ultrapassá-lo levanta `ErroColeta`, porque um servidor defeituoso que devolvesse sempre um link para a página seguinte, ou para si mesmo, prenderia o pipeline num laço sem fim. O laço de paginação, assim, concentra numa única implementação a lógica que seria repetida e garante que ela sempre termine.

A completude da coleta

A completude da coleta é decidida ao fim do laço, comparando o total anunciado pela fonte com o total de registros efetivamente obtidos. Se os dois forem iguais, a coleta é completa; se forem diferentes, a coleta fica registrada como incompleta, e a transformação não a usará. Se a fonte não anuncia total, como acontece com o IPCA do Banco Central, a coleta é considerada completa nesta etapa, e a completude é conferida depois, na limpeza, pela ausência de meses faltantes. O resumo registra também o número de páginas lidas. A completude da coleta, por conseguinte, impede que uma coleta parcial seja tratada como se fosse o retrato inteiro da fonte.

O total estável

A regra do total estável, aplicada pela função `_total_estavel`, exige que o total anunciado pela fonte seja o mesmo em todas as páginas da coleta. Essa regra existe porque, se a fonte receber novos registros enquanto o pipeline percorre as páginas, a paginação por posição pode pular ou repetir registros: um registro inserido no início empurra todos os outros uma posição para frente, e o último de uma página reaparece no início da seguinte. Quando o total muda, a fonte mudou no meio da coleta, e a fotografia obtida misturaria dois momentos diferentes. A função então recusa a coleta com `ErroColeta` e a mensagem "recolete". A regra do total estável, assim, garante que cada coleta represente um único instante da fonte.

O reaproveitamento das páginas

O reaproveitamento das páginas é feito pela função `obter_json`, que, antes de pedir uma página ao servidor, verifica se ela já foi gravada no mesmo dia. Se a página existe, é relida do disco em vez de pedida de novo. Essa regra tem dois motivos: a cortesia, porque pedir de novo a mesma página no mesmo dia sobrecarrega o servidor sem trazer informação nova, e a retomada, porque uma coleta interrompida na página 8 recomeça da página 8, sem repetir as sete anteriores. A busca é feita pela função `gravado_hoje`, que procura no manifesto um registro da mesma fonte, assunto, dia, página e extensão, e devolve o mais recente; com a opção `--recoletar`, essa busca é desligada. Quando a página é nova, ela passa pelo cliente com a política completa, é lida como JSON e só então gravada, com a URL final, o status e o número de registros. O reaproveitamento das páginas, por conseguinte, faz com que interromper e retomar uma coleta não custe nada ao servidor.

CONCLUSÃO — SÍNTESE

A síntese da interface e do laço de paginação mostra que suas partes separam com precisão o que é comum do que é específico. O comum, a sequência de páginas, o limite, a completude, o total estável e o reaproveitamento, é escrito uma vez e vale para todas as fontes; o específico, onde estão os registros e como se chega à página seguinte, fica em cada adaptador. Por causa dessa separação, cada fonte escreve apenas a parte que é dela, e uma correção no laço comum beneficia todas ao mesmo tempo.

18. Adaptadores das APIs paginadas

INTRODUÇÃO — O TODO

Os adaptadores das APIs paginadas, no módulo `pipeline/coleta/fontes_api.py`, são as três classes que traduzem formas diferentes de paginação para o laço comum. Cada servidor público divide seus dados em páginas de um jeito próprio: a API da Prefeitura usa links para a página seguinte, o serviço de mapas do IBGE usa deslocamento numérico, e o Banco Central limita cada consulta a um período de tempo. Os adaptadores existem para que essas diferenças fiquem confinadas a poucas linhas cada, e o restante do pipeline veja as três fontes da mesma forma. O desenvolvimento a seguir analisa cada adaptador e as funções auxiliares que eles compartilham.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A arrecadação da Prefeitura de Palmas

O adaptador `ArrecadacaoPalmas` coleta a arrecadação histórica da Prefeitura pela API em `integracao.palmas.to.gov.br/apirest/arrecadacao/`. Essa API foi construída com o Django REST Framework e pagina por link: cada página traz o total no campo `count`, os registros na lista `results` e o endereço da página seguinte no campo `next`. O adaptador lê esses três campos e, como o link `next` chega em HTTP, promove-o a HTTPS no mesmo host antes de segui-lo. A coleta para quando o campo `next` vem vazio, que é a forma com que essa API indica a última página. Na execução de referência, foram 36 páginas e 3.542 registros, que cobrem os exercícios de 2010 a 2018; o histórico da API foi atualizado pela última vez em 01/06/2021. O adaptador da arrecadação, assim, traduz a paginação por link numa sequência segura de pedidos.

Os setores censitários do IBGE

O adaptador `SetoresIbge` coleta a malha dos setores censitários de 2022 pelo serviço WFS do IBGE, um padrão internacional para servir dados geográficos. Esse serviço pagina por deslocamento: o pedido informa quantas feições quer, no parâmetro `count`, e a partir de qual posição, no parâmetro `startIndex`. O adaptador pede 200 feições por página e acrescenta dois parâmetros decisivos. O primeiro é a ordenação `sortBy=cd_setor`, que existe porque, sem uma ordem estável, o servidor pode devolver as feições em ordem diferente a cada pedido, e a paginação por posição repetiria alguns setores e pularia outros. O segundo é o filtro `CQL_FILTER`, que restringe a resposta aos setores de Palmas, porque a camada contém o Brasil inteiro. As coordenadas são pedidas em SIRGAS 2000 (EPSG:4674), o sistema de referência oficial do Brasil. O adaptador dos setores, por conseguinte, transforma uma paginação frágil numa sequência determinística.

O critério de parada dos setores

O critério de parada dos setores combina duas condições, e a coleta termina quando qualquer uma delas é verdadeira. A primeira é que o número de feições obtidas alcance o total anunciado no campo `numberMatched`; a segunda é que uma página venha vazia. A segunda condição parece desnecessária, mas protege contra um servidor que anuncie um total maior do que consegue entregar: sem ela, o pipeline pediria páginas vazias até atingir o limite de 1.000. Na execução de referência, a coleta levou 4 páginas e trouxe os 733 setores de Palmas. O critério de parada, assim, garante que a coleta termine tanto no caso normal quanto no caso defeituoso.

O IPCA do Banco Central

O adaptador `IpcaBcb` coleta a série 433 do Sistema Gerenciador de Séries Temporais do Banco Central, que traz a variação mensal do IPCA. Essa API não pagina por páginas, e sim por período: cada consulta informa uma data inicial e uma data final, e o Banco Central aceita no máximo dez anos por consulta. O adaptador, por isso, divide o período de janeiro de 2000 até a data de hoje em janelas de até dez anos e trata cada janela como uma página. O período começa em janeiro de 2000, uma década antes da receita mais antiga do projeto, de 2010, de modo que todo mês da receita tem índice com folga. O período termina na data de hoje porque o IPCA mais recente é o mês de referência da deflação. O adaptador do IPCA, por conseguinte, transforma um limite de período numa paginação comum.

As janelas decenais

A função `janelas_decenais` divide um período em janelas de no máximo dez anos, sem sobreposição e sem lacuna. Cada janela começa no dia seguinte ao fim da anterior e termina na véspera do dia que fica dez anos depois do seu início, ou na data final, o que vier primeiro. A ausência de sobreposição importa porque um mês repetido apareceria duas vezes na série; a ausência de lacuna importa porque um mês faltante impediria deflacionar a receita daquele mês. O 29 de fevereiro tem tratamento próprio: somar dez anos a essa data pode cair num ano que não é bissexto, onde o dia 29 não existe, e nesse caso a função usa o dia 28. As janelas decenais, assim, cobrem o período inteiro exatamente uma vez.

O avanço e a completude do IPCA

O avanço entre as janelas do IPCA é decidido pelo método `proxima`, que identifica a janela atual pela data inicial do pedido e devolve a janela seguinte, ou nada, quando a última foi lida. O Banco Central não informa quantas observações a série tem, e por isso o método `total` devolve nada e a coleta é considerada completa ao fim das janelas. A completude real do IPCA é conferida depois, na limpeza, que verifica se não falta nenhum mês entre o primeiro e o último. Na execução de referência, três janelas trouxeram 321 meses, de janeiro de 2000 a setembro de 2026. O avanço do IPCA, por conseguinte, delega à limpeza a conferência que a fonte não permite fazer na coleta.

As funções auxiliares

As funções auxiliares `_lista` e `_inteiro` extraem da resposta os campos de que os adaptadores precisam e verificam o tipo de cada um. A função `_lista` exige que o campo dos registros seja uma lista e levanta `ErroColeta` se ele faltar ou tiver outro tipo, porque uma resposta sem a lista esperada indica que a API mudou de formato. A função `_inteiro` lê o total apenas se ele for um número inteiro, e devolve nada em qualquer outro caso, porque um total de tipo errado não

merece confiança. As funções auxiliares, assim, fazem a coleta falhar de forma clara quando a fonte muda de formato, em vez de continuar com dados mal interpretados.

CONCLUSÃO — SÍNTESE

A síntese dos adaptadores das APIs mostra que três formas de paginação muito diferentes, por link, por deslocamento e por período, foram reduzidas a um mesmo padrão: cada adaptador diz como começar, onde estão os registros, qual é o total e como avançar. O padrão comum é acompanhado de um critério de parada explícito em cada caso, o link vazio, o total alcançado ou a última janela, e de uma proteção contra o defeito típico de cada forma. É essa combinação que permite ao laço comum percorrer as três fontes com a mesma confiança.

19. Adaptadores de arquivos

INTRODUÇÃO — O TODO

Os adaptadores de arquivos, no módulo `pipeline/coleta/fontes_arquivos.py`**, tratam as três fontes que não são paginadas: o GeoPalmas, os arquivos do Censo 2022 e a extração NUCLEOGOV.** Essas fontes exigem adaptadores próprios porque cada uma tem um protocolo particular: o GeoPalmas exige dois níveis de consulta, os arquivos do Censo mudam de nome a cada revisão, e a extração NUCLEOGOV não pode ser obtida do portal. O ponto comum entre elas é que todas entregam arquivos ou respostas que precisam ser conferidos antes de virar bruto. O desenvolvimento a seguir analisa cada fonte e os mecanismos de download e conferência que elas usam.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O GeoPalmas

O adaptador `GeoPalmas` **coleta as quadras do portal de mapas da Prefeitura em dois passos.** O primeiro pedido, ao endereço `getshape/`, traz a lista das regiões de planejamento da cidade; o segundo passo faz um pedido por região, ao endereço `getplacemarker/<região>/`, e traz as quadras daquela região com seus polígonos. A consulta em dois níveis existe porque é assim que o próprio portal organiza os dados, e não há um endereço que devolva todas as quadras de uma vez. As regiões são registradas como uma coleta própria, porque também são usadas depois, para agregar indicadores por região, e cada região vira uma página no bruto. O adaptador do GeoPalmas, assim, segue a organização do portal sem impor a ele uma forma que ele não oferece.

A completude do GeoPalmas

A completude do GeoPalmas é decidida pelo número de arquivos, e não pelo número de quadras. A coleta é considerada completa se houver um arquivo de quadras para cada região listada no primeiro pedido. Esse critério foi adotado porque o portal não anuncia quantas quadras existem, e a única verificação possível é a de que todas as regiões foram consultadas. Na execução de referência, foram seis regiões e 284 quadras. A completude do GeoPalmas, por conseguinte, confere tudo o que a fonte permite conferir, e declara o limite do que não permite.

A descoberta dos arquivos do Censo

A descoberta dos arquivos do Censo é necessária porque o nome de cada arquivo muda a cada revisão do IBGE: o nome termina com a data da revisão, no formato AAAAMMDD. Se o pipeline usasse um nome fixo, deixaria de encontrar o arquivo na revisão seguinte ou, pior, continuaria usando uma revisão antiga sem perceber. Por isso, o adaptador `ArquivoCenso` baixa primeiro o índice da pasta no servidor do IBGE, uma página HTML que lista os arquivos, e grava esse índice no bruto como evidência de quais arquivos existiam no dia. Em seguida, uma expressão regular, chamada seletor, procura no índice os nomes que seguem o padrão esperado e captura a data de cada um, e a função `arquivo_mais_recente` escolhe o nome com a maior data. Essa técnica é uma raspagem do índice, documentada no próprio código. A descoberta dos arquivos, assim, acompanha as revisões do IBGE sem intervenção humana.

Os dois arquivos do Censo

Os dois arquivos do Censo coletados pelo pipeline são o ZIP com a renda do responsável por setor e o dicionário de dados em formato XLSX. O ZIP contém o CSV nacional com as variáveis de renda, moradores e domicílios, que alimentam o enriquecimento territorial. O dicionário descreve o significado de cada variável e é guardado como documentação da fonte, embora a transformação não o leia. Os dois arquivos passam pela mesma conferência de integridade do ZIP, porque um arquivo XLSX é, internamente, um contêiner ZIP. Os dois arquivos do Censo, por conseguinte, trazem juntos o dado e a sua definição.

O download condicional

O download condicional, feito pela função `baixar`, evita baixar de novo um arquivo grande que não mudou. Antes do pedido, a função procura no manifesto o último download da mesma URL e, se ele existir e o arquivo ainda estiver no disco, envia ao servidor dois cabeçalhos: `If-None-Match`, com a etiqueta de versão do arquivo anterior, e `If-Modified-Since`, com a data de modificação anterior. Se o servidor responde 304, ele está informando que o arquivo não mudou, e nenhum byte do conteúdo é transferido; o pipeline então reaproveita o bruto anterior (Kurose; Ross, 2013, §2.2.6). Se o arquivo anterior não está mais no disco, a função não envia os cabeçalhos condicionais, porque uma resposta 304 seria inútil sem a cópia local. Se o servidor responder 304 sem que haja cópia anterior, a função levanta `ErroColeta`, porque essa resposta seria incoerente. O download condicional, assim, poupa dezenas de megabytes por execução sem arriscar a atualidade do dado.

A conferência antes da promoção

A conferência antes da promoção é o passo que decide se um download com status 200 pode virar bruto. O arquivo é baixado primeiro para uma pasta temporária, `data/bruto/.parcial`, e só depois de conferido pela função que a fonte informou é promovido ao nome definitivo. Junto com o arquivo, o manifesto registra a etiqueta de versão, a data de modificação, o tipo de conteúdo e a URL pedida, porque esses detalhes são exatamente os que o próximo download condicional vai precisar. A conferência antes da promoção, por conseguinte, garante que nenhum arquivo defeituoso chegue a ocupar um lugar no bruto.

A integridade do ZIP

A função `zip_integro` confere a integridade de um arquivo ZIP abrindo-o e testando cada membro interno. O teste recalcula o CRC de cada arquivo contido no ZIP, um código de verificação que o próprio formato guarda, e compara com o valor registrado. Um ZIP que não abre ou que tem um membro com CRC errado levanta `ErroColeta`. Essa conferência existe porque um download pode chegar com o tamanho certo e, ainda assim, com bytes corrompidos no meio, e só a verificação do conteúdo detecta esse caso. A integridade do ZIP, assim, completa a conferência de tamanho feita pelo transporte.

A tentativa ao portal NUCLEOGOV

A coleta do NUCLEOGOV começa sempre por uma tentativa ao portal, feita pelo método `_tentar_portal`. O método faz um único pedido à página do portal e registra no manifesto o resultado, qualquer que seja: recusa, falha de rede ou sucesso. Na execução de referência, o portal respondeu 403, e o registro de tentativa guardou a observação de que não houve troca de User-Agent, proxy ou VPN. A tentativa é feita mesmo sabendo que o portal costuma recusar, por duas razões: a recusa precisa ser comprovada a cada execução, e não apenas afirmada, e, se um dia o portal passar a aceitar o acesso, o registro mostrará essa mudança. A tentativa ao portal, por conseguinte, transforma a recusa num fato documentado, e não numa suposição.

A cópia local da extração

A cópia local da extração NUCLEOGOV é a forma normal de o dado entrar no pipeline depois da recusa do portal. O caminho da cópia vem da opção `--nucleogov-arquivo` ou da variável `PI2_NUCLEOGOV_ARQUIVO`; sem cópia e sem a opção do GitHub, a coleta falha com uma mensagem que explica como resolver. A cópia só é aceita se for idêntica ao arquivo publicado pela coordenação: o tamanho precisa ser de 28.136.842 bytes, e o hash de objeto do Git precisa ser `449ca775…`. O tamanho é comparado primeiro porque é uma verificação instantânea, e só se ele conferir o pipeline calcula o hash, que exige ler os 28 MB do arquivo. A cópia local, assim, é aceita não por confiança em quem a forneceu, mas por prova de que é o arquivo oficial.

O hash de objeto do Git

O hash de objeto do Git é o identificador que o Git e o GitHub atribuem a cada arquivo, e o pipeline o calcula por conta própria. Esse hash é o SHA-1 de uma sequência formada pela palavra "blob", pelo tamanho do arquivo, por um byte nulo e pelo conteúdo. O pipeline usa esse hash, e não o SHA-256 que usa em outros lugares, porque ele é o único que a fonte publica: a coordenação disponibilizou a extração num repositório do GitHub, e o GitHub mostra, para cada arquivo, exatamente esse hash. Assim, a comparação é feita contra um valor que a própria fonte tornou público. O SHA-1 é considerado fraco contra quem fabrica colisões de propósito, e o código declara isso ao calcular o hash com a indicação de que não é usado para segurança; aqui, ele serve para identificar a versão publicada do arquivo, e não para resistir a um ataque. O hash de objeto do Git, por conseguinte, liga a cópia local ao arquivo publicado com o único identificador que a fonte oferece.

O reaproveitamento da extração

O reaproveitamento da extração evita que o pipeline copie o mesmo arquivo para o bruto a cada execução. Antes de copiar, o adaptador procura no manifesto um arquivo da mesma fonte com o mesmo SHA-256, em qualquer dia; se o encontrar e ele ainda existir, reaproveita o registro. Essa regra existe porque a extração NUCLEOGOV é uma fotografia fixa, feita em 07/07/2026, e copiá-la todos os dias encheria o bruto de cópias idênticas com datas diferentes. Quando a cópia é necessária, o manifesto registra como URL apenas `arquivo-local:` seguido do nome do arquivo, e não o caminho completo do computador, que identificaria o usuário. O reaproveitamento da extração, assim, mantém o bruto sem redundância e sem dado pessoal.

O download opcional pelo GitHub

O download pelo GitHub é a alternativa à cópia local, e só ocorre com a opção `--nucleogov-github`. Nesse caminho, o pipeline consulta o último commit da branch principal do repositório da coordenação, lista a pasta `dados/` exatamente naquele commit e baixa o arquivo da extração, conferindo-o contra o hash publicado na listagem. A listagem é feita no commit específico, e não na branch, porque a branch pode avançar entre a consulta e o download, e então a listagem e o arquivo poderiam pertencer a versões diferentes. O caminho é opcional porque a equipe decidiu que nenhum acesso ao GitHub acontece sem pedido explícito, e porque depender de um serviço de terceiros a cada execução tornaria o pipeline mais frágil. O download pelo GitHub, por conseguinte, existe como recurso consciente, e não como comportamento padrão.

CONCLUSÃO — SÍNTESE

A síntese dos adaptadores de arquivos revela que, apesar dos protocolos diferentes, todos obedecem ao mesmo princípio: nada entra no bruto sem ter sido conferido contra uma referência externa. O GeoPalmas é conferido pela lista de regiões, o Censo pela data no índice, pelo tamanho e pelo CRC do ZIP, e a extração NUCLEOGOV pelo hash publicado no GitHub. A tentativa ao portal, por sua vez, garante que a recusa também seja um fato registrado. É esse princípio que permite afirmar que todo arquivo do bruto chegou íntegro e é o que diz ser.

20. Catálogo e execução da coleta

INTRODUÇÃO — O TODO

O catálogo e a execução da coleta, no módulo `pipeline/coleta/catalogo.py`, são a parte que lista as fontes do pipeline e as percorre uma a uma. Esse módulo é o ponto em que a coleta deixa de ser um conjunto de adaptadores independentes e se torna uma etapa do pipeline, com relatório, log e evidências. A característica central do módulo é o isolamento das falhas: uma fonte que falha não impede a coleta das outras. O desenvolvimento a seguir analisa o catálogo das fontes, o relatório da coleta, a execução com isolamento de falhas e a gravação das evidências do robots.txt.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O catálogo das fontes

O catálogo das fontes é a função `fontes_padrao`, que devolve as sete fontes na ordem em que são coletadas: a arrecadação da Prefeitura, a extração NUCLEOGOV, as quadras do GeoPalmas, os setores do IBGE, a renda do Censo, o dicionário do Censo e o IPCA. A função recebe a data de hoje, porque o IPCA precisa saber até quando pedir a série, e as opções do NUCLEOGOV, porque o adaptador dessa fonte depende da cópia local ou da autorização para o GitHub. Concentrar a lista numa única função permite ver, num só lugar, tudo o que o pipeline

coleta. O catálogo das fontes, assim, é o índice da coleta.

O relatório da coleta

O relatório da coleta, o objeto `RelatorioColeta`, guarda os resumos das coletas bem-sucedidas e a lista das falhas. Cada falha registra a fonte, o assunto e a mensagem do erro, porque essas três informações bastam para saber o que falhou e por quê. A propriedade `registros` soma os registros obtidos em todas as fontes. O relatório é devolvido ao orquestrador, que o resume no registro da execução. O relatório da coleta, por conseguinte, separa claramente o que deu certo do que deu errado.

A execução com isolamento de falhas

A execução da coleta, na função `coletar_tudo`, percorre as fontes uma a uma e isola as falhas de cada uma. Se uma fonte levanta `ErroColeta`, a falha é registrada no relatório e no log, e a função segue para a próxima fonte. Esse isolamento existe porque uma fonte fora do ar não deveria impedir a coleta das outras seis: o servidor do IBGE pode estar em manutenção num dia em que a API da Prefeitura funciona normalmente. A função, porém, só isola os erros de coleta; um erro de outro tipo, como um defeito de programação, não é capturado e interrompe a execução, porque um defeito no código não é um problema da fonte e não deve ser disfarçado como tal. Cada coleta bem-sucedida gera uma linha no log com os registros, os arquivos, o total esperado e a situação, e uma coleta incompleta aparece com a palavra INCOMPLETA em maiúsculas, para se destacar na leitura. A execução com isolamento de falhas, assim, aproveita tudo o que pode ser coletado sem esconder o que não pôde.

A evidência do robots.txt

A evidência do robots.txt é gravada pela função `registrar_robots` ao fim da coleta. Para cada host consultado, a função grava no bruto a resposta recebida ao pedido do robots.txt, seja o arquivo de regras, seja a página de erro. O arquivo recebe a fonte "robots" e, como assunto, o nome do host com os pontos trocados por hífens, porque o padrão de nomes do bruto não aceita pontos no assunto. O manifesto registra também o status da resposta e a situação interpretada: REGRAS, INDISPONIVEL ou INALCANCAVEL. Essa gravação existe porque o robots.txt de um servidor pode mudar, e só uma cópia datada prova o que ele dizia no dia da coleta. A evidência do robots.txt, por conseguinte, documenta que a coleta respeitou as regras vigentes naquele dia.

CONCLUSÃO — SÍNTESE

A síntese do catálogo e da execução mostra que a coleta foi desenhada para ser tolerante a falhas por fonte, mas intolerante a defeitos de código. O catálogo diz o que coletar, o relatório separa sucesso de falha, a execução aproveita cada fonte disponível sem esconder as indisponíveis, e o robots.txt fica gravado como prova de conduta. A decisão sobre se há dado suficiente para seguir não cabe à coleta: ela cabe à transformação, que confere, pela validação V01, se cada fonte tem uma coleta completa.

21. Leitura do bruto

INTRODUÇÃO — O TODO

A leitura do bruto, no módulo `pipeline/transformacao/leitura.py`, é o primeiro passo da transformação e decide quais arquivos serão processados. A regra central desse módulo é que a transformação nunca lê "o que estiver na pasta": ela lê apenas os arquivos que o manifesto atribui à coleta completa mais recente de cada fonte. Essa regra existe porque a pasta do bruto acumula coletas de vários dias, versões alternativas e coletas incompletas, e ler tudo o que há nela misturaria momentos diferentes da fonte. O desenvolvimento a seguir analisa a lista das fontes lidas, a conferência de cada arquivo, a leitura segura do ZIP do Censo, a junção das páginas e o objeto que reúne o resultado.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

As fontes lidas

As fontes lidas pela transformação estão listadas no dicionário `FONTES`, que liga cada nome interno a uma fonte e a um assunto do manifesto. São sete entradas: a arrecadação, a extração NUCLEOGOV, as regiões, as quadras, os setores, a renda do Censo e o IPCA. O dicionário de dados do Censo não está nessa lista, porque ele documenta o significado das variáveis para as pessoas, mas não contém dados que a transformação precise processar. Para cada entrada, a função `ler_brutos` pede ao depósito a coleta completa mais recente, e é essa coleta, registrada no manifesto, que define quais arquivos serão abertos. As fontes lidas, assim, são escolhidas pelo registro, e não pelo conteúdo da pasta.

A conferência de cada arquivo

A conferência de cada arquivo, feita pela função `_conferidos`, verifica três condições antes de qualquer leitura: o arquivo precisa ter registro no manifesto, precisa existir no disco e precisa ter exatamente o SHA-256 registrado. Se qualquer condição falhar, a função levanta `ErroColeta`, e a transformação para. Essa conferência repete, para os arquivos que serão lidos, o que a validação V12 já fez para o bruto inteiro; a repetição existe porque a leitura não deve depender de que outra etapa tenha rodado antes, e, se a função for chamada sozinha, por exemplo num teste, ela continua protegida. A conferência de cada arquivo, por conseguinte, garante que a transformação nunca use um arquivo trocado, apagado ou alterado.

A leitura da extração NUCLEOGOV

A leitura da extração NUCLEOGOV usa o leitor aprovado na Sprint 2, a função `ler_fonte` do módulo `etl.parser`. O arquivo é lido como texto na codificação UTF-8 com marca de ordem de bytes, porque arquivos gerados por planilhas costumam começar com essa marca invisível, e, se ela não fosse removida, o nome da primeira coluna ficaria com um caractere a mais e deixaria de corresponder ao contrato. O texto é entregue ao leitor de CSV com a opção que preserva as quebras de linha originais, porque um campo entre aspas pode conter uma quebra de linha, e só essa opção permite ao leitor reconhecê-la como parte do campo. O leitor devolve o cabeçalho e os registros numerados pela linha de origem. A leitura da extração, assim, reaproveita o contrato já auditado e preserva o número da linha para a quarentena.

A leitura segura do ZIP do Censo

A leitura do ZIP do Censo, feita pela função `_csv_do_zip`, extrai o único CSV contido no arquivo com dois limites de segurança. O ZIP precisa conter exatamente um CSV, porque mais de um tornaria ambíguo qual ler, e nenhum indicaria um arquivo errado. O primeiro limite é o tamanho descompactado, que não pode passar de 256 MiB; o segundo é a razão entre o tamanho descompactado e o compactado, que não pode passar de 50 para 1. Esses limites existem para recusar uma bomba de descompressão, um arquivo pequeno construído para se expandir até esgotar a memória da máquina. O CSV real tem 30,6 MB descompactado e uma razão de cerca de 3,3 para 1, de modo que os limites deixam uma folga de oito vezes para o crescimento legítimo do arquivo e, ainda assim, recusam as razões típicas de um ataque. A leitura segura do ZIP, por conseguinte, trata um arquivo vindo da internet como potencialmente hostil.

A desconfiança do tamanho declarado

A desconfiança do tamanho declarado é a razão pela qual a leitura do ZIP confere duas vezes o tamanho. O cabeçalho do ZIP declara o tamanho descompactado de cada membro, e a função primeiro compara essa declaração com os limites, porque é uma verificação imediata que recusa os casos óbvios sem ler nada. A declaração, porém, pode mentir: um arquivo malicioso pode declarar um tamanho pequeno e conter muito mais. Por isso, a função lê o CSV em fluxo até o limite mais um byte e confere o que de fato leu; se leu mais que o limite, recusa o arquivo. A desconfiança do tamanho declarado, assim, combina a rapidez da verificação pelo cabeçalho com a certeza da verificação pelo conteúdo.

A junção das páginas

A junção das páginas, feita pela função `_achatar`, reúne numa única lista os registros que a coleta gravou em vários arquivos. Para a arrecadação, a função junta a lista `results` de cada página; para os setores, junta a lista `features`; para as quadras e o IPCA, cuja página já é uma lista, junta as listas diretamente. As regiões vêm num único arquivo, e por isso dispensam a junção. Essa etapa existe porque a divisão em páginas é uma conveniência do servidor, e não uma propriedade do dado; a limpeza precisa ver a fonte inteira, por exemplo para detectar um identificador repetido em páginas diferentes. A junção das páginas, por conseguinte, desfaz a fragmentação imposta pela coleta.

O objeto Brutos

O objeto `Brutos` é o resultado da leitura e reúne, para cada fonte, os dados prontos para a limpeza. Ele guarda as coletas usadas, a lista de registros da arrecadação, o cabeçalho e os registros numerados do NUCLEOGOV, as regiões, as quadras, as feições dos setores, o texto do CSV da renda e as observações do IPCA. Guardar as coletas usadas junto com os dados permite que a validação V01 confira a completude de cada fonte e que o relatório de qualidade informe de que coleta veio cada dado. O objeto `Brutos`, assim, entrega à limpeza não só os dados, mas também a prova de sua origem.

CONCLUSÃO — SÍNTESE

A síntese da leitura do bruto mostra que todas as suas partes servem para que a transformação parta de um ponto inequívoco e íntegro. O manifesto escolhe os arquivos, a conferência garante que eles não mudaram, os limites do ZIP impedem um ataque, e a junção entrega cada fonte inteira. Como a escolha depende do manifesto, e não da pasta, duas pessoas que transformarem o mesmo bruto em máquinas diferentes processarão exatamente os mesmos arquivos.

22. Fundamentos da limpeza

INTRODUÇÃO — O TODO

Os fundamentos da limpeza, no módulo `pipeline/limpeza/comum.py`, são as peças que todos os filtros de limpeza compartilham: o formato do resultado, a quarentena, o erro de contrato, a conversão de dinheiro e a normalização de texto. Essas peças foram reunidas num módulo próprio porque a limpeza de cada fonte precisa obedecer às mesmas regras de contagem e de exatidão; se cada filtro inventasse a sua forma de contar ou de converter dinheiro, os resultados de fontes diferentes não poderiam ser comparados. Nenhuma função do módulo faz entrada ou saída, o que permite testá-las isoladamente. O desenvolvimento a seguir analisa cada uma dessas peças.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O resultado de um filtro

O resultado de um filtro é o objeto `ResultadoLimpeza`, que guarda as linhas limpas, a quarentena e três contagens: os registros lidos, os aceitos e os descartados. Os rejeitados não são uma contagem independente: correspondem ao tamanho da quarentena, de modo que nunca pode haver um registro rejeitado sem o motivo correspondente. As três contagens existem para que se possa conferir que nenhum registro desapareceu durante a limpeza. O resultado de um filtro, assim, carrega a prova de que a limpeza contou tudo o que recebeu.

A identidade da reconciliação

A identidade da reconciliação é a igualdade lidos = aceitos + rejeitados + descartados, que todo filtro precisa satisfazer. O método `reconcilia` confere essa igualdade, e a validação V02 a exige para cada fonte. A identidade importa porque um registro pode se perder de forma silenciosa, por exemplo quando uma exceção é capturada sem que o registro seja contado em lugar algum; se isso acontecer, a soma deixa de fechar, e o pipeline para. A identidade conta registros, e não linhas, porque um registro aceito pode gerar várias linhas: um registro mensal da API, por exemplo, gera doze linhas, uma por mês. A identidade da reconciliação, por conseguinte, transforma a pergunta "algum registro sumiu?" numa conta que a máquina faz sozinha.

A quarentena

A quarentena é a lista dos registros rejeitados, cada um descrito por um objeto `Rejeicao` com três campos: a origem, a chave e o motivo. A origem diz de qual arquivo o registro veio, a chave o identifica dentro desse arquivo, e o motivo explica por que ele não foi aceito. Esses três campos bastam para que alguém encontre o registro, entenda o problema, corrija a regra ou a fonte e reprocesse. O registro rejeitado vai para a quarentena, e não para o lixo, porque descartar em silêncio um registro esconderia do leitor o tamanho do problema e impediria corrigi-lo. A quarentena, assim, guarda o que não pôde ser aceito de forma que possa ser examinado.

O erro de contrato

O erro de contrato, a classe `ErroContrato`, indica que a entrada inteira não tem a forma combinada, por exemplo quando falta uma coluna ou quando a chave não existe. Esse erro é diferente de uma rejeição: a rejeição afeta um registro e permite continuar com os demais, enquanto o erro de contrato afeta o arquivo inteiro e impede qualquer continuação. A distinção existe porque um arquivo sem a coluna da renda, por exemplo, não tem registros "ruins" e registros "bons"; todos os registros estão igualmente incompletos, e mandá-los um a um para a quarentena só esconderia que o problema é do arquivo. O erro de contrato, por conseguinte, interrompe a etapa quando a falha é estrutural, e não pontual.

O dinheiro em centavos

O dinheiro é convertido pela função `centavos` em números inteiros de centavos, e não em números de ponto flutuante. Essa escolha existe porque o ponto flutuante, a representação comum de números decimais nos computadores, não consegue representar exatamente valores como 0,10; somar milhares desses valores acumula pequenos erros, e o total deixa de bater ao centavo. Os inteiros, ao contrário, são somados sem erro algum. A conversão passa pelo tipo `Decimal`, que representa decimais de forma exata, e o valor é antes convertido em texto, porque um `Decimal` criado diretamente de um número de ponto flutuante herdaria o erro de representação. A função rejeita quatro casos, cada um com o nome do campo na mensagem: texto vazio, texto não numérico, valor infinito e valor com fração de centavo, porque uma fração de centavo indica um dado que não é dinheiro ou que foi mal formatado. O dinheiro em centavos, assim, garante que as somas do pipeline sejam exatas.

A normalização de texto

A normalização de texto, feita pela função `normalizar`, prepara uma descrição para ser comparada com as regras de classificação. A função remove os acentos, decompondo cada letra acentuada na letra simples e no sinal e descartando o sinal; troca qualquer sequência de espaços, inclusive o espaço não separável que algumas fontes usam, por um espaço simples; e passa tudo para maiúsculas. Essa preparação existe porque a mesma conta aparece na fonte com grafias diferentes, como "Dívida Ativa" e "DIVIDA ATIVA", e as regras precisam reconhecê-las como iguais. O texto original, porém, é preservado na saída, e a versão normalizada serve apenas para casar as regras, porque quem lê o arquivo limpo precisa ver a descrição exatamente como a fonte a escreveu. A normalização de texto, por conseguinte, torna a comparação robusta sem alterar o dado.

CONCLUSÃO — SÍNTESE

A síntese dos fundamentos da limpeza revela que todas as peças servem a duas garantias: nada entra ou sai sem registro, e nenhum valor perde exatidão. O resultado e a identidade garantem a contagem; a quarentena e o erro de contrato garantem que toda rejeição tenha motivo e que toda falha estrutural interrompa; os centavos garantem a exatidão das somas; e a normalização garante que a comparação não dependa da grafia. Por isso, cada filtro de limpeza que usa esses fundamentos herda essas garantias sem precisar reescrevê-las.

23. Limpeza da arrecadação da API

INTRODUÇÃO — O TODO

A limpeza da arrecadação da API, no módulo `pipeline/limpeza/receita_api.py`**, transforma os registros brutos da API antiga da Prefeitura numa tabela tipada e classificada.** A API cobre os exercícios de 2010 a 2018, e cada registro traz uma conta contábil, a sua descrição, o valor anual e os doze valores mensais. Essa fonte exige uma limpeza cuidadosa porque mistura formatos sem avisar, trunca descrições em alguns anos e descreve a mesma espécie de receita com grafias diferentes. O desenvolvimento a seguir analisa os campos exigidos, o tratamento dos identificadores repetidos, a decisão de granularidade, o ano, a classificação tributária, as descrições truncadas, a fonte de recurso, o arredondamento e a linha limpa resultante; as decisões são identificadas pelos códigos D-API-01 a D-API-07.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os campos exigidos

Cada registro da API precisa ter 17 campos preenchidos: `id`**,** `ano`**,** `receita_codigo`**,** `receita_descricao`**,** `valor_ano` **e os doze valores mensais.** Um registro com qualquer campo vazio vai para a quarentena, e o motivo lista os campos ausentes. Essa exigência existe porque cada campo participa de alguma regra da limpeza: sem o ano não há competência, sem a descrição não há classificação, e sem um dos meses não é possível decidir a granularidade. Os campos exigidos, assim, são exatamente os que a limpeza precisa para interpretar o registro.

Os identificadores repetidos

Os identificadores repetidos são tratados pela decisão D-API-05, segundo a qual um identificador que aparece mais de uma vez rejeita todas as suas ocorrências. A função conta os identificadores de todas as páginas antes de examinar qualquer registro, porque só com a contagem completa se sabe quais estão repetidos. Todas as ocorrências são rejeitadas, e não apenas a segunda, porque não há como saber qual das cópias é a correta; manter a primeira seria uma escolha arbitrária apresentada como se fosse certa. Na execução de referência, nenhum identificador estava repetido. Os identificadores repetidos, por conseguinte, são tratados com a cautela de quem prefere declarar a dúvida a escondê-la.

A decisão de granularidade

A decisão de granularidade, D-API-01, determina se um registro descreve doze meses ou um ano inteiro, e é tomada pelos próprios números, e não pelo ano do registro. Essa decisão é necessária porque a API mistura dois formatos: em alguns registros, os doze meses somam o valor anual, como se espera; em outros, sobretudo de 2016 a 2018, cada mês repete o valor anual inteiro. Se a soma dos meses é igual ao valor anual, o registro é mensal e vira doze linhas, uma por mês. Se todos os meses repetem o valor anual, que não pode ser zero, o registro é anual e vira uma única linha, sem mês. Somar os meses no segundo caso multiplicaria a arrecadação por doze, e tratá-los como mensais criaria uma sazonalidade falsa. As duas condições não podem valer ao mesmo tempo, exceto quando tudo é zero, e é por isso que o valor anual do caso anual precisa ser diferente de zero. Qualquer outro padrão vai para a quarentena, porque não há como interpretar o registro com segurança. Na execução de referência, 1.964 registros eram mensais e 1.578 anuais, e nenhum foi para a quarentena. A decisão de granularidade, assim, lê o formato de cada registro a partir dos seus próprios números.

O ano do registro

O ano do registro precisa estar entre 2000 e 2100, e um ano fora desse intervalo leva o registro para a quarentena. Essa regra é larga de propósito: ela não tenta adivinhar quais anos a API deveria ter, apenas barra valores que só podem ser erro de digitação ou de conversão, como 20017 ou 207. A regra existe porque um ano absurdo passaria por todas as etapas seguintes e só seria percebido num gráfico, depois de contaminar somas e indicadores. O ano do registro, por conseguinte, é filtrado por um critério que recusa o impossível sem rejeitar o plausível.

A tabela de classificação

A classificação tributária, D-API-02, usa uma tabela explícita de regras, em que cada regra é uma expressão regular associada a um componente da receita, e a primeira regra que casa com a descrição vale. A tabela foi escolhida em vez de uma classificação por semelhança porque adivinhar o tributo pela aparência da descrição produziria erros silenciosos justamente nas contas raras, que são as menos conferidas. Uma conta que não casa com nenhuma regra fica fora do escopo, com o tributo vazio, e nunca é adivinhada; na execução de referência, 3.135 registros ficaram assim, entre taxas, contribuições e transferências, e 407 foram classificados como tributários. Cada linha classificada guarda o identificador da regra que a classificou, de modo que qualquer classificação pode ser auditada. A tabela de classificação, assim, troca a conveniência da adivinhação pela possibilidade de conferência.

A preparação da descrição

A preparação da descrição acontece antes da classificação e consiste em normalizar o texto e retirar dele três marcas que não dizem respeito ao tributo. A primeira é a marca de estorno, que indica a anulação de um lançamento; a segunda é a fonte de recurso, como PRÓPRIO, MDE ou ASPS; a terceira é a marca de arredondamento. Essas marcas são retiradas porque, se ficassem, cada regra precisaria prever todas as combinações possíveis delas, e a tabela se multiplicaria. Retiradas as marcas, a regra casa só com o que sobra, que é a descrição da espécie de receita. A informação das marcas, porém, não se perde: o estorno define o componente, e a fonte de recurso é guardada numa coluna própria. A preparação da descrição, por conseguinte, simplifica as regras sem descartar informação.

As regras da tabela

As regras da tabela têm identificadores que indicam a sua família. As regras A1 e A2 tratam deduções e restituições, que são ajustes; D1 e D2 tratam a dação em pagamento, em que o contribuinte quita a dívida com bens, classificada como dívida ativa ou principal conforme a descrição. As regras M1 e M2 tratam multas e juros da dívida ativa, V1 e V2 tratam a receita da dívida ativa, e J1, J2 e J3 tratam multas e juros do exercício corrente. A regra P1 trata o principal e casa com o nome do tributo sozinho ou seguido de "PRINCIPAL" ou "REVISADO". O nome do tributo é reconhecido por prefixos que incluem formas truncadas, como IPT para IPTU, ITB para ITBI e ISSQ para ISSQN, porque o sistema de origem cortou nomes em vários registros. As regras da tabela, assim, descrevem uma a uma as espécies de receita que o projeto precisa separar.

Os componentes

Cada regra leva a um dos cinco componentes da receita: PRINCIPAL, MULTAS_JUROS, DIVIDA_ATIVA, DIVIDA_ATIVA_MULTAS_JUROS e AJUSTE. Esses componentes existem porque a receita de um tributo não é homogênea: o principal é o imposto pago no próprio exercício, as multas e juros são acréscimos pelo atraso, e a dívida ativa é o que foi pago depois de o crédito ser inscrito para cobrança. Separar esses componentes é o que permite, depois, medir quanto da arrecadação vem da recuperação de dívidas. O estorno sempre vira AJUSTE, qualquer que seja a regra que casou, segundo a decisão D-API-04, e o ajuste mantém o sinal da fonte, normalmente negativo. Os componentes, por conseguinte, transformam descrições contábeis em categorias com significado para a gestão tributária.

As descrições truncadas

As descrições truncadas são o problema tratado pela decisão D-API-06: em 2016 e 2017, o sistema de origem cortou descrições a ponto de perder o nome do tributo, como em "MULTA JUROS E MORA DA DIVIDA A233". O que sobrou foi um sufixo numérico, e a análise mostrou que esses números vêm em trincas consecutivas por tributo, na ordem PRÓPRIO, MDE e ASPS. A ordem da trinca foi deduzida das linhas de estorno, que trazem a fonte por extenso, como "DIVI235 ASPS - ESTORNO": como 235 é o terceiro número da trinca que começa em 233, a terceira posição corresponde a ASPS. A tabela `TRINCAS_2016_2017` registra o primeiro número de cada trinca e o tributo correspondente, separadas por espécie de receita, e a regra M3 usa essa tabela para classificar as multas e juros da dívida ativa com descrição truncada. Na execução de referência, 21 registros foram classificados pela regra M3. As descrições truncadas, assim, foram recuperadas por uma convenção demonstrada nos próprios dados, e não por suposição.

A fonte de recurso

A fonte de recurso, tratada pela decisão D-API-03, indica a parcela contábil a que cada lançamento foi atribuído: PRÓPRIO, MDE ou ASPS. As siglas MDE e ASPS designam, respectivamente, a manutenção e desenvolvimento do ensino e as ações e serviços públicos de saúde, parcelas que a contabilidade pública separa para acompanhar os mínimos constitucionais aplicados a essas áreas. A função `fonte_recurso` procura primeiro o nome da fonte na descrição e, se não o encontra, usa a posição do sufixo numérico na trinca; sem nome e sem trinca, a fonte é "NAO_INFORMADA". O total do tributo soma as três fontes, porque elas são partes de uma mesma arrecadação, e nenhuma é descartada. Na execução de referência, foram 130 registros PRÓPRIO, 128 MDE, 135 ASPS e 14 sem fonte informada. A fonte de recurso, por conseguinte, é preservada como detalhe sem fragmentar o total do tributo.

O arredondamento

O arredondamento, tratado pela decisão D-API-07, refere-se a lançamentos marcados com "(ARREDONDAMENTO)", que são ajustes de centavos do próprio componente. A marca é retirada da descrição para a classificação, e o valor permanece no componente a que pertence, em vez de virar uma linha separada. A grafia errada "ARRENDONDAMENTO", que aparece na fonte, também é reconhecida, porque uma regra que só aceitasse a grafia correta deixaria esses lançamentos fora do escopo. O sufixo "[E]", que aparece em 2018 sem significado documentado e soma centavos nas contas tributárias, recebe o mesmo tratamento. O arredondamento, assim, é absorvido pelo componente correto, sem criar uma categoria artificial.

A linha limpa

A linha limpa é o resultado da limpeza de cada registro e guarda tudo o que as etapas seguintes precisam. Ela traz o identificador de origem, o ano, o mês, que fica vazio nas linhas anuais, a granularidade, o código e a descrição originais, o tributo, o componente, a fonte de recurso, a regra que a classificou e o valor em centavos. O identificador de origem e a descrição original permitem voltar do resultado ao registro bruto; a regra permite auditar a classificação; e o valor em centavos garante a exatidão das somas. A linha limpa, por conseguinte, é ao mesmo tempo um dado pronto para uso e um rastro até a sua origem.

CONCLUSÃO — SÍNTESE

A síntese da limpeza da API revela um padrão que se repete em todas as suas regras: diante de uma irregularidade da fonte, o pipeline escolhe uma interpretação demonstrável nos próprios dados ou declara a dúvida, mas nunca adivinha. A granularidade é lida pelos números, a classificação é feita por regras explícitas, as descrições truncadas são resolvidas por uma convenção comprovada, e o que não se encaixa vai para a quarentena ou fica fora do escopo com o motivo registrado. Por isso, cada linha limpa diz qual regra a classificou, e qualquer classificação pode ser refeita e conferida.

24. Limpeza da extração NUCLEOGOV

INTRODUÇÃO — O TODO

A limpeza da extração NUCLEOGOV, no módulo `pipeline/limpeza/receita_nucleogov.py`**, transforma o arquivo oficial da receita de 2019 a julho de 2026 numa tabela classificada por tributo e componente.** A característica central dessa limpeza é que ela reaproveita o contrato e o mapeamento aprovados na Sprint 2, implementados no módulo `etl/parser.py`. Esse reaproveitamento foi deliberado: o arquivo já tinha sido analisado e auditado na Sprint 2, e escrever uma segunda leitura do mesmo arquivo criaria o risco de duas interpretações diferentes do mesmo dado. O desenvolvimento a seguir analisa o contrato herdado, o mapeamento por vigência, a exclusão de 2018, o filtro e a conciliação.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O contrato herdado da Sprint 2

O contrato herdado da Sprint 2 define as condições que o arquivo precisa cumprir para ser interpretado. O arquivo precisa ter as 14 colunas do layout da fonte, e cada linha precisa ter um valor por coluna. Os códigos das contas são tratados como texto, e nunca como números, porque zeros à esquerda e o comprimento do código têm significado contábil, e uma conversão numérica os destruiria. O dinheiro é convertido em `Decimal` exato, e uma fração de centavo é rejeitada. O contrato também registra uma particularidade desta extração: o valor arrecadado no período é igual ao valor arrecadado no mês, e somar os dois duplicaria a receita. O contrato herdado, assim, transporta para a Sprint 3 todo o conhecimento sobre o arquivo que a Sprint 2 acumulou.

A conta-pai e os componentes

A conta-pai e os componentes são os dois papéis que uma linha do NUCLEOGOV pode ter. A conta-pai é a conta que totaliza um tributo, e os componentes são as contas que a detalham em principal, multas e juros, dívida ativa e multas e juros da dívida ativa. A medida que se soma são os quatro componentes, e a conta-pai serve apenas para conferência, porque somar a conta-pai e os componentes contaria o mesmo dinheiro duas vezes. O contrato exige ainda que todo componente

tenha a sua conta-pai no mesmo órgão e no mesmo ano, porque um componente sem pai indicaria um arquivo incompleto ou um código mal formado. A conta-pai e os componentes, por conseguinte, permitem ao mesmo tempo somar sem duplicar e conferir a soma.

O mapeamento por vigência

O mapeamento aprovado liga cada tributo à sua conta-pai, e esse mapeamento muda conforme o período, que o código chama de vigência. Há duas vigências, de 2019 a 2021 e de 2022 a 2026, porque, a partir de 2022, a mesma receita passou a ser registrada com outros códigos de conta. O IPTU, por exemplo, tem a conta-pai `1118011` na primeira vigência e `1112500` na segunda. O componente é identificado pela conta-pai seguida de um dígito: 1 para o principal, 2 para multas e juros, 3 para a dívida ativa e 4 para multas e juros da dívida ativa. Uma conta fora do mapeamento não é classificada, pela mesma regra de não adivinhar que vale para a API. O mapeamento por vigência, assim, acompanha a mudança do plano de contas sem confundir códigos de épocas diferentes.

A exclusão do exercício de 2018

A exclusão do exercício de 2018, decisão D-NUC-01, retira esse ano da limpeza do NUCLEOGOV. O ano de 2018 não pertence a nenhuma vigência do mapeamento aprovado, e, além disso, o arquivo repete a mesma descrição em vários níveis do código naquele ano, como em 1931, 19311 e assim por diante até 19311100010000000. Somar esses níveis multiplicaria o valor, porque cada nível repete o total do nível abaixo. Como a API antiga também tem 2018 sob suspeita, pela decisão D-REC-05, não há fonte confiável para esse ano, e 2018 vira uma lacuna declarada, sem substituto. A exclusão de 2018, por conseguinte, prefere uma lacuna visível a um valor que se sabe errado.

O filtro

O filtro, a função `limpar_nucleogov`, aplica o contrato ao arquivo inteiro e separa as linhas em três grupos. O primeiro grupo é o das linhas que não puderam ser interpretadas, que vão para a quarentena com o motivo apontado pelo contrato; o segundo é o das linhas interpretadas, que contam como aceitas; e o terceiro, dentro das aceitas, é o das linhas mapeadas, que viram linhas limpas. Uma linha interpretada mas fora do mapeamento conta como aceita sem produzir linha limpa, porque ela foi lida corretamente e apenas não pertence ao escopo tributário. Erros que afetam o arquivo inteiro, como um cabeçalho errado ou um componente sem conta-pai, não vão para a quarentena: eles reprovam a validação V05 e interrompem a execução, porque afetam todas as linhas igualmente. Cada linha limpa guarda o ano, o mês, o órgão, o tributo, o papel, o componente, o código original, o valor arrecadado e o valor orçado, ambos em centavos, e a linha de origem. O filtro, assim, distingue o que é ilegível, o que está fora do escopo e o que é dado tributário.

A conciliação

A conciliação compara cada conta-pai com a soma dos seus componentes. A chave da comparação é formada pelo órgão, pelo ano, pelo mês e pela conta-pai, e há divergência se a soma dos componentes difere do total da conta-pai ou se faltam componentes. A conciliação existe porque a soma dos componentes é a medida usada pelo pipeline, e só a concordância com o total registrado pela própria fonte prova que nenhum componente foi esquecido ou duplicado. A validação V06 exige que não haja nenhuma divergência. Na execução de referência, as 176.993 linhas foram lidas sem rejeição, resultaram em 1.092 componentes e 273 totais tributários de 2019 a julho de 2026, e não

houve nenhuma divergência. A conciliação, por conseguinte, confirma pelos números da própria fonte que a classificação está completa.

CONCLUSÃO — SÍNTESE

A síntese da limpeza do NUCLEOGOV mostra que suas partes têm um propósito comum: preservar as regras que a Sprint 2 já validou, em vez de reinventá-las. O contrato garante a leitura correta, o mapeamento garante a classificação por vigência, a exclusão de 2018 evita uma soma multiplicada, o filtro separa o ilegível do fora de escopo, e a conciliação confere o resultado contra os totais da própria fonte. Por isso, a receita de 2019 em diante entra no pipeline com o mesmo grau de confiança que tinha ao sair da auditoria da Sprint 2.

25. Limpeza das geometrias

INTRODUÇÃO — O TODO

A limpeza das geometrias, no módulo `pipeline/limpeza/territorio.py`, prepara os polígonos das quadras do GeoPalmas e dos setores censitários do IBGE para o cruzamento territorial. As duas fontes descrevem áreas da cidade por meio de polígonos, mas em formatos diferentes, e cada polígono precisa ser geometricamente válido antes de ser medido ou cruzado. Essa exigência existe porque um polígono inválido, por exemplo um que cruza a si mesmo, produz áreas e interseções erradas sem gerar nenhum erro visível. O desenvolvimento a seguir analisa a correção dos polígonos, a leitura do formato do GeoPalmas, a limpeza das quadras, a limpeza dos setores e o resultado obtido.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A correção dos polígonos

A correção dos polígonos, feita pela função `corrigir`, devolve uma geometria válida e informa se foi preciso corrigi-la. Uma geometria já válida passa sem mudança. Uma geometria inválida passa pela função `make_valid`, que a reconstrói sem descartar nenhum de seus vértices, segundo a decisão D-TER-02. A reconstrução pode produzir, além de polígonos, pedaços degenerados, como linhas e pontos, e só as partes poligonais são mantidas, porque o cruzamento mede áreas, e linhas e pontos não têm área. Se a correção não deixar nenhum polígono, a geometria é rejeitada. Cada geometria corrigida é marcada, para que se saiba quais polígonos não vieram válidos da fonte. A correção dos polígonos, assim, recupera o que é recuperável e declara o que foi alterado.

O formato do GeoPalmas

O formato do GeoPalmas guarda as coordenadas de cada quadra como texto, na forma "longitude,latitude,altitude", com os pontos separados por espaços e, nas quadras com mais de uma parte, os anéis separados pelo caractere "#". A função `poligono_geopalmas` converte esse texto em polígono. De cada ponto, ela usa apenas a longitude e a latitude e descarta a altitude, porque a análise é feita sobre o mapa, em duas dimensões, e a altitude não interfere na área projetada. Cada anel precisa ter pelo menos quatro pontos, porque o menor polígono possível, um triângulo, tem três vértices, e o anel é fechado repetindo o primeiro ponto no fim. Um anel com menos pontos levanta erro. O formato do GeoPalmas, por conseguinte, é traduzido para um polígono padrão com uma verificação mínima de que ele pode existir.

A região da quadra

A região da quadra é a região de planejamento a que ela pertence, informada no campo `shape`, e a limpeza exige que essa região exista na lista de regiões coletada. Uma quadra que aponta para uma região inexistente vai para a quarentena. Essa exigência existe porque a região é usada depois para agregar indicadores, e uma quadra sem região válida ficaria fora dessas agregações sem que ninguém percebesse. A região da quadra, assim, é conferida contra a própria fonte antes de ser usada.

Os atributos da quadra

Os atributos da quadra são normalizados segundo a decisão D-TER-03, para que o mesmo significado tenha sempre a mesma grafia. O uso da quadra, quando vazio ou marcado com "*", vira "NAO_INFORMADO", porque o asterisco não tem significado documentado e equivale, na prática, à falta de informação. A situação é normalizada para três valores: APROVADA, PENDENTE, que reúne "APROVACAO PENDENTE" e "PENDENTE", e NAO_INFORMADA. Os códigos de uso sem legenda oficial no serviço são mantidos como vêm, porque traduzi-los exigiria adivinhar o seu significado. A referência entre parênteses no nome da quadra é extraída para uma coluna própria. Na execução de referência, havia 202 quadras aprovadas, 16 pendentes e 65 sem situação informada. Os atributos da quadra, por conseguinte, são padronizados apenas até onde a fonte permite.

As medidas da quadra

As medidas da quadra são calculadas a partir do polígono já válido. A área é medida em metros quadrados depois de projetar o polígono para o sistema UTM, porque coordenadas em graus não servem para medir área. Além da área, a limpeza calcula um ponto representativo, que é garantidamente interior ao polígono; esse ponto é usado em vez do centro geométrico porque o centro de uma quadra côncava ou com várias partes pode cair fora dela. A geometria é guardada como texto no formato WKT, com sete casas decimais, o que corresponde a cerca de um centímetro; esse arredondamento torna a saída determinística, porque pequenas diferenças de cálculo entre máquinas desaparecem abaixo dessa precisão. As medidas da quadra, assim, são feitas no sistema certo e gravadas de forma reproduzível.

O isolamento dos erros por quadra

O isolamento dos erros por quadra faz com que um problema numa quadra não interrompa a limpeza das demais. A limpeza captura três tipos de erro: o erro de valor, como um anel com poucos pontos, o erro de chave, quando falta um campo, e o erro de tipo, quando um campo tem o tipo errado. Em todos os casos, a quadra vai para a quarentena com o motivo, e a limpeza segue. Esse isolamento existe porque o GeoPalmas é uma fonte de cadastro, em que erros pontuais de digitação são esperados, e uma única quadra mal cadastrada não deveria impedir o uso das outras centenas. O isolamento dos erros, por conseguinte, trata a falha pontual como pontual.

A limpeza dos setores

A limpeza dos setores aplica regras próprias às feições geográficas do IBGE. Cada setor precisa pertencer ao município de Palmas, e um setor de outro município é rejeitado, porque o filtro da coleta deveria tê-lo excluído, e a sua presença indica uma resposta anômala do servidor. Uma feição sem geometria também é rejeitada, porque um setor sem polígono não pode ser cruzado nem mapeado. Um código de setor repetido rejeita todas as ocorrências, pelo mesmo motivo que vale para

a API: não há como saber qual geometria é a correta. O setor limpo guarda os códigos, a situação, o tipo, o distrito e duas áreas: a informada pelo IBGE e a calculada pelo pipeline na projeção UTM. As duas áreas são guardadas porque a comparação entre elas serve de conferência da projeção. A limpeza dos setores, assim, garante uma malha completa, sem duplicação e restrita ao município.

O resultado real

O resultado real da limpeza das geometrias, na execução de referência, mostrou fontes em bom estado. Apenas uma quadra foi rejeitada, a "Arse 152", porque um de seus anéis não tinha pontos suficientes para formar um polígono. Nenhuma geometria precisou de correção, e nenhum dos 733 setores foi rejeitado. Esse resultado não torna as regras desnecessárias, porque elas protegem contra o que pode chegar numa coleta futura. O resultado real, por conseguinte, confirma que as regras funcionam sem rejeitar dados bons.

CONCLUSÃO — SÍNTESE

A síntese da limpeza das geometrias revela que todas as suas regras convergem para um mesmo resultado: polígonos válidos, medidos no mesmo sistema de projeção e gravados com a mesma precisão. A correção garante a validade, a tradução do formato garante a uniformidade, as regras de região e de código garantem a coerência com a fonte, e o isolamento dos erros garante que um defeito pontual não contamine o conjunto. O cruzamento territorial, etapa seguinte, depende inteiramente dessas garantias, porque a interseção de dois polígonos só é confiável se ambos forem válidos.

26. Limpeza do Censo e do IPCA

INTRODUÇÃO — O TODO

A limpeza do Censo e do IPCA, no módulo `pipeline/limpeza/censo_ipca.py`**, trata duas fontes que, embora diferentes, compartilham um problema: ambas têm regras próprias para valores ausentes.** No Censo, a ausência é o sigilo imposto pelo IBGE; no IPCA, a ausência seria um mês faltante na série. Nas duas fontes, a regra do pipeline é a mesma: ausência nunca vira zero, porque um setor sob sigilo não tem renda zero, e um mês sem dado não teve inflação zero. O desenvolvimento a seguir analisa, no Censo, a seleção dos setores de Palmas, as variáveis, o contrato e o sigilo; e, no IPCA, a competência, o número-índice, as duplicidades e as lacunas.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A seleção dos setores de Palmas

A seleção dos setores de Palmas é necessária porque o arquivo do Censo é nacional e contém todos os setores do país. A limpeza fica apenas com os setores cujo código começa por `1721000`, o código de Palmas. As outras linhas nem entram na contagem de lidos, porque contar centenas de milhares de setores de outros municípios faria a proporção de rejeição de Palmas parecer insignificante mesmo que todos os setores da cidade fossem rejeitados. A seleção dos setores, assim, faz com que as contagens da limpeza descrevam apenas o recorte que importa ao projeto.

As variáveis do Censo

As variáveis do Censo usadas pelo pipeline são seis, todas do arquivo de renda do responsável pelo domicílio. A V06001 é o número de responsáveis em domicílios particulares permanentes ocupados, que se aproxima do número de domicílios, porque cada domicílio tem um responsável; a V06002 é o número de moradores; a V06003 é a variância do número de moradores. A V06004 é o rendimento nominal médio mensal dos responsáveis com rendimento, a V06005 é a sua variância, e a V06006 é a sua mediana. Essas variáveis foram escolhidas porque descrevem, por setor, o tamanho da população e o nível de renda, que são as duas informações de que o enriquecimento territorial e a simulação precisam. As variáveis do Censo, por conseguinte, são o elo entre o território e a condição socioeconômica.

O contrato do Censo

O contrato do Censo exige que o arquivo tenha a coluna `CD_SETOR` e as seis variáveis; se faltar alguma, a limpeza levanta `ErroContrato`. Esse contrato existe porque o IBGE revisa os arquivos, e uma revisão pode renomear ou remover colunas; sem o contrato, a falta de uma coluna produziria um erro confuso em alguma etapa posterior, ou, pior, valores vazios tratados como ausência legítima. Quando o contrato é violado, o encadeamento da transformação registra a validação V05 como reprovada antes de interromper, para que o motivo apareça no log e no banco. O contrato do Censo, assim, transforma uma mudança silenciosa da fonte num erro explícito.

O sigilo do IBGE

O sigilo do IBGE é a omissão de um valor que o instituto pratica quando um setor tem poucos domicílios, para impedir que os moradores sejam identificados. No arquivo, o valor omitido aparece como "X", e a decisão D-CEN-01 determina que ele vire ausente, nunca zero. Um setor com algum valor ausente recebe a marca de sigilo, continua na base, mas fica sem renda, e nenhum valor é imputado no seu lugar. A razão é dupla: imputar a renda criaria justamente o dado que o próprio produtor decidiu omitir, e um zero faria o setor parecer o mais pobre da cidade. Um valor não numérico ou negativo, diferente de "X", é tratado como erro e leva o setor à quarentena. Na execução de referência, foram lidos 683 setores de Palmas, nenhum foi rejeitado, e 23 estavam sob sigilo. O sigilo do IBGE, por conseguinte, é respeitado como ausência declarada, e não como valor.

A competência do IPCA

A competência do IPCA é o mês a que cada observação se refere. Cada observação do Banco Central traz uma data no formato dia/mês/ano e uma variação percentual, e a limpeza exige que a data seja sempre o primeiro dia do mês, porque é assim que o Banco Central representa uma série mensal. Uma data em outro dia indicaria uma série diferente da esperada, talvez diária, e levaria a observação à quarentena. A variação é convertida em `Decimal` e precisa ser um número finito. A competência do IPCA, assim, garante que cada observação represente exatamente um mês.

O número-índice

O número-índice é calculado pela limpeza a partir das variações mensais, segundo a decisão D-IPC-01. O índice é encadeado a partir de uma base igual a 100 em dezembro de 1999, o mês anterior ao início da série, e cada mês multiplica o índice anterior por um mais a variação dividida por cem. O índice é necessário porque a deflação precisa do nível de preços, e não da variação: a razão entre o índice de dois meses dá a inflação acumulada entre eles, o que permite converter um valor de qualquer mês para a moeda de qualquer outro. O cálculo usa `Decimal` em toda

a cadeia, porque um erro de arredondamento repetido ao longo de 321 multiplicações se acumularia. O número-índice, por conseguinte, transforma uma lista de variações numa régua de preços.

As duplicidades e os conflitos

As duplicidades e os conflitos são tratados de forma diferente, conforme as observações repetidas de um mesmo mês sejam iguais ou não. Uma cópia idêntica do mesmo mês é descartada e entra na contagem de descartados, porque ela não acrescenta nem contradiz informação. Valores diferentes para o mesmo mês, ao contrário, vão todos para a quarentena, porque não há como saber qual é o correto. Um mês em conflito deixa, portanto, uma lacuna na série. Essa lacuna precisa interromper o pipeline, e não apenas ser registrada, porque o índice dos meses seguintes seria calculado a partir do índice anterior à lacuna e ficaria errado. As duplicidades e os conflitos, assim, distinguem a repetição inofensiva da contradição perigosa.

As lacunas

As lacunas são identificadas pela função `lacunas`, que lista os meses ausentes entre o primeiro e o último mês da série. A função percorre os meses um a um e anota cada mês que não está presente. O resultado alimenta a validação V07, que reprova a execução se houver qualquer lacuna e também se faltar o índice de algum mês da receita. Na execução de referência, a série teve 321 meses, de janeiro de 2000 a setembro de 2026, sem nenhuma lacuna e sem nenhum conflito. As lacunas, por conseguinte, são detectadas antes que a deflação possa usá-las.

CONCLUSÃO — SÍNTESE

A síntese da limpeza do Censo e do IPCA mostra que as duas fontes, tão diferentes, foram tratadas pelo mesmo princípio: a ausência é uma informação, e não um valor. No Censo, o sigilo vira ausente e marca o setor; no IPCA, o mês faltante vira lacuna e interrompe a execução. Essa diferença entre ausência e zero protege os dois cálculos que dependem dessas fontes: a renda dos setores, que não pode ser rebaixada a zero por um sigilo, e a inflação acumulada, que não pode ser calculada sobre uma série incompleta.

27. Enriquecimento da receita

INTRODUÇÃO — O TODO

O enriquecimento da receita, no módulo `pipeline/enriquecimento/receita.py`, transforma as duas tabelas limpas de arrecadação, a da API e a do NUCLEOGOV, numa única série analítica em valores comparáveis entre anos. Esse módulo é necessário porque as duas fontes cobrem períodos diferentes, usam granularidades diferentes e estão em moedas de poder de compra diferente: um real de 2010 não compra o mesmo que um real de 2026. O módulo realiza quatro operações: unifica as fontes, deflaciona os valores pelo IPCA, calcula indicadores anuais e calcula a sazonalidade. O desenvolvimento a seguir analisa cada operação e as regras que a governam, identificadas pelos códigos D-REC-01 a D-REC-05.

A unificação das fontes

A unificação das fontes, feita pela função `unificar` segundo a decisão D-REC-01, junta a API e o NUCLEOGOV numa só série, sem sobreposição entre elas. A API cobre os anos até 2017, depois da exclusão de 2018, e o NUCLEOGOV cobre de 2019 em diante; se um ano aparecesse nas duas fontes, ficaria com o NUCLEOGOV, porque ela é a fonte atual e auditada. A regra da não sobreposição existe porque somar as duas fontes no mesmo ano contaria duas vezes a mesma arrecadação. A unificação soma os valores por fonte, ano, mês, granularidade, tributo e componente, e só entram as linhas com tributo, porque as taxas e transferências fora do escopo não fazem parte da análise tributária. Do NUCLEOGOV, só entram os componentes, e nunca a conta-pai, pela regra que evita contar o mesmo dinheiro duas vezes. A unificação das fontes, assim, produz uma série contínua em que cada real aparece uma única vez.

A exclusão do exercício de 2018 da API

A exclusão do exercício de 2018 da API, decisão D-REC-05, retira da série um ano que a comparação com a fonte atual mostrou ser inconfiável. Os totais de 2018 na API são cerca de 3,3 vezes os das contas-folha do NUCLEOGOV para o mesmo ano: o IPTU, por exemplo, soma R$ 297 milhões na API contra R$ 90,7 milhões nominais no NUCLEOGOV. Além disso, todos os registros de 2018 foram atualizados em 01/06/2021, a data em que o histórico da API terminou, o que indica que lançamentos posteriores se acumularam nesse ano. Em vez de corrigir o valor por suposição, a decisão o exclui e conta as linhas excluídas; na execução de referência, 86 linhas tributárias da API ficaram fora da série. A exclusão de 2018, por conseguinte, transforma um valor suspeito numa lacuna declarada e medida.

A composição do total do tributo

A composição do total do tributo, decisão D-REC-02, define que o total soma as três fontes de recurso e inclui os ajustes com o sinal da fonte. As fontes de recurso PRÓPRIO, MDE e ASPS são somadas porque representam divisões contábeis de uma mesma arrecadação, e não receitas distintas. Os ajustes, como estornos, restituições e deduções, entram com o sinal negativo que a fonte lhes atribui, porque eles representam dinheiro que deixou de pertencer ao município, e ignorá-los superestimaria a arrecadação. Na execução de referência, a série unificada ficou com 2.055 linhas. A composição do total, assim, faz com que a série represente o que o município efetivamente arrecadou.

A deflação

A deflação, feita pela função `deflacionar` segundo a decisão D-REC-03, converte cada valor nominal em reais do último mês publicado do IPCA, que na execução de referência foi setembro de 2026. O fator de deflação é o índice do mês de referência dividido pelo índice do período do valor: se os preços subiram 50% desde o período, o fator é 1,5, e cada real daquele período vale 1,5 real de hoje. Para um valor mensal, o período é o próprio mês. Para um valor anual, o período é a média dos doze índices do ano, porque o valor anual é a soma de pagamentos distribuídos ao longo do ano, e cada um deles foi feito a um nível de preços diferente; a média aproxima o nível de preços típico desse conjunto. A deflação, por conseguinte, coloca todos os anos na mesma moeda, condição para que uma comparação de crescimento não confunda inflação com aumento de arrecadação.

A precisão do fator

A precisão do fator de deflação é fixada em dez casas decimais, e o mesmo fator fixado é usado no cálculo e gravado no banco. Essa regra foi introduzida para corrigir um defeito real: antes, o Python calculava com o fator em precisão total, enquanto o banco recebia o fator arredondado em seis casas, e a view do banco divergia dos arquivos em até R$ 15. Com o mesmo fator nos dois lugares, a view `v_receita_real` chega ao mesmo centavo que os arquivos. O valor real é arredondado para o centavo apenas no fim do cálculo, com o critério de arredondar a metade para longe do zero, que é o mesmo da função `ROUND` do MySQL. Arredondar só no fim evita que pequenos erros intermediários se acumulem, e usar o mesmo critério do banco evita uma nova divergência de um centavo. A precisão do fator, assim, garante que duas implementações do mesmo cálculo cheguem ao mesmo número.

A codificação one-hot

A codificação one-hot, feita pela função `com_one_hot`, cria uma coluna de valor 0 ou 1 para cada tributo: `tributo_IPTU`, `tributo_ISSQN` e `tributo_ITBI`. Em cada linha, a coluna do tributo da linha vale 1, e as outras valem 0. Essa codificação existe porque os modelos estatísticos trabalham com números, e uma coluna de texto como "IPTU" não pode entrar diretamente num cálculo; atribuir números arbitrários aos tributos, como 1, 2 e 3, criaria uma ordem falsa entre eles. A codificação one-hot, por conseguinte, prepara a categoria para o modelo da Sprint 4 sem lhe impor uma ordem que ela não tem.

Os indicadores anuais

Os indicadores anuais, calculados pela função `indicadores_anuais`, resumem a receita por ano e por tributo. Para cada par de ano e tributo, a função soma o valor nominal, o valor real e a parcela da dívida ativa, e conta os meses que têm dado; uma linha anual conta como os doze meses. Sobre essas somas, a função calcula quatro indicadores: a variação real, a quebra de fonte, a participação da dívida ativa e o desvio orçamentário. A contagem de meses é guardada porque vários indicadores dependem de saber se o ano está completo. Os indicadores anuais, assim, transformam milhares de linhas mensais em poucas medidas interpretáveis por um gestor.

A variação real

A variação real é o crescimento percentual da arrecadação real de um ano em relação ao ano anterior, e só é calculada quando os dois anos têm os doze meses. Essa restrição existe porque o último ano da série, 2026, só tem dados até julho; comparar sete meses de 2026 com doze meses de 2025 faria parecer uma queda que, na verdade, é apenas um ano incompleto. Quando um dos anos é parcial, a variação fica ausente, e não zero. A variação real, por conseguinte, só afirma um crescimento ou uma queda quando a comparação é entre períodos equivalentes.

A quebra de fonte

A quebra de fonte é o indicador que marca o ano em que a série passa de uma fonte para outra. Ele compara a fonte do ano com a fonte do último ano anterior que tem dado, e não simplesmente com o ano imediatamente anterior. Essa forma de comparar foi adotada para corrigir um defeito: como 2018 está fora da série, comparar 2019 com 2018 não encontrava nenhuma fonte, e a troca da API para o NUCLEOGOV não era marcada. Comparando com o último ano com dado, 2017, a quebra aparece corretamente em 2019. A marcação importa porque duas fontes podem medir a mesma receita de formas ligeiramente diferentes, e uma mudança de patamar entre 2017 e 2019

pode ser efeito da troca de fonte, e não da realidade. A quebra de fonte, assim, avisa o leitor exatamente onde a comparação exige cautela.

A participação da dívida ativa

A participação da dívida ativa é a porcentagem da arrecadação real de um tributo que veio da cobrança de créditos inscritos em dívida ativa. O indicador soma os componentes de dívida ativa e de multas e juros da dívida ativa e divide pelo total real do ano. Esse indicador importa para a Opção D porque mede quanto da arrecadação depende da recuperação de créditos atrasados, e não do pagamento em dia: uma participação alta pode indicar inadimplência acumulada ou um esforço de cobrança intenso. A participação da dívida ativa, por conseguinte, é o indicador da receita real mais próximo do tema da inadimplência.

O desvio orçamentário e a previsão

O desvio orçamentário compara o que foi arrecadado com o que estava previsto no orçamento, e a previsão vem da função `orcado_anual`**.** Essa função lê, para cada ano e tributo, o valor orçado da conta-pai no último mês informado, porque a previsão orçamentária é revista ao longo do ano, e o último valor é a previsão atualizada. A comparação usa valores nominais, e não reais, porque a previsão é feita na moeda do próprio ano, e deflacionar só um dos lados criaria uma diferença artificial. O desvio só existe nos anos do NUCLEOGOV, porque a API não traz previsão orçamentária. O desvio orçamentário, assim, mede a qualidade do planejamento na mesma moeda em que ele foi feito.

A sazonalidade

A sazonalidade, calculada pela função `sazonalidade` **segundo a decisão D-REC-04, mede como a arrecadação de cada tributo se distribui ao longo dos meses do ano.** O procedimento é o clássico de Persons (1919): para cada ano, cada mês é dividido pela média mensal do ano, e o índice sazonal do mês é a média dessas razões entre os anos. Um índice igual a 1 significa um mês típico; um índice de 3,5, como o de março no IPTU, significa que aquele mês arrecada três vezes e meia a média. Só entram anos com os doze meses e com total positivo, porque um ano parcial distorceria a média mensal, e as linhas anuais não entram, porque não têm distribuição por mês. O índice é arredondado para quatro casas, e a tabela informa quantos anos foram usados; na execução de referência, foram 13 anos por tributo. A sazonalidade, por conseguinte, revela o calendário da arrecadação, que depois orienta o vencimento da simulação.

CONCLUSÃO — SÍNTESE

A síntese do enriquecimento da receita mostra que todas as suas operações perseguem um mesmo fim: tornar comparável o que, no bruto, não era. A unificação elimina a duplicação entre fontes, a deflação elimina a diferença de moeda, a precisão do fator elimina a diferença entre implementações, e os indicadores só comparam períodos equivalentes. Onde a comparação continua arriscada, como na troca de fonte, o risco é marcado em vez de escondido. Por isso, todo valor real da série está na mesma moeda, e toda quebra está sinalizada.

28. Enriquecimento territorial

INTRODUÇÃO — O TODO

O enriquecimento territorial, no módulo `pipeline/enriquecimento/territorio.py`, liga a renda do Censo ao território da cidade, cruzando setores censitários e quadras. Esse cruzamento é necessário porque as duas fontes descrevem a cidade com recortes diferentes: o IBGE divide Palmas em setores censitários, definidos para a coleta do Censo, e a Prefeitura a divide em quadras, definidas pelo planejamento urbano. Os dados de renda estão nos setores, e as análises da Prefeitura costumam usar quadras e regiões; sem o cruzamento, as duas visões não conversariam. O desenvolvimento a seguir analisa o cruzamento geométrico, a escolha da quadra principal, a padronização da renda, o setor enriquecido e a estimativa por quadra.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O cruzamento setor × quadra

O cruzamento setor × quadra, feito pela função `cruzar_setores_quadras` segundo a decisão D-TER-04, encontra todos os pares de setor e quadra que se sobrepõem e mede a área da sobreposição. As geometrias são antes projetadas no sistema UTM 22S, porque a área de interseção precisa ser medida em metros quadrados, e não em graus. Para cada par, a função guarda a área da interseção e duas frações: a fração do setor, que é a parte do setor que cai dentro da quadra, e a fração da quadra, que é a parte da quadra que cai dentro do setor. As duas frações são necessárias porque respondem a perguntas diferentes: a fração do setor serve para repartir a população do setor entre as quadras, e a fração da quadra serve para saber quanto da quadra é coberto por dados do Censo. O cruzamento setor × quadra, assim, cria a ponte quantitativa entre os dois recortes.

A árvore espacial

A árvore espacial, a estrutura `STRtree`, acelera o cruzamento ao devolver, para cada setor, apenas as quadras que estão próximas dele. Sem essa estrutura, cada um dos 733 setores seria comparado com cada uma das 283 quadras, num total de mais de 200 mil cálculos de interseção, a maioria entre polígonos que estão em lados opostos da cidade. A árvore organiza os retângulos que envolvem as quadras de forma que a busca descarta rapidamente as que não podem se tocar, e só as candidatas passam pelo cálculo exato. Os índices devolvidos pela árvore são ordenados antes de usados, para que a ordem dos pares seja sempre a mesma e a saída seja determinística. A árvore espacial, por conseguinte, torna o cruzamento rápido sem alterar o seu resultado.

O descarte do ruído de borda

O descarte do ruído de borda elimina as interseções menores que 1 m². Essas interseções minúsculas surgem porque as malhas do IBGE e da Prefeitura foram desenhadas por instituições diferentes, com precisões diferentes, e as fronteiras de setores e quadras vizinhos raramente coincidem com exatidão; o resultado são lascas de poucos centímetros quadrados que não representam nenhuma sobreposição real. Manter essas lascas criaria pares falsos de setor e quadra, que distorceriam a escolha da quadra principal e a repartição da população. Na execução de referência, depois do descarte, sobraram 1.635 pares. O descarte do ruído de borda, assim, separa a sobreposição geográfica real do erro de desenho.

A quadra principal

A quadra principal, escolhida pela função `quadra_principal`, é, para cada setor, a quadra que contém a maior fração da sua área. Essa escolha existe porque várias análises precisam associar cada setor a uma única quadra, e por meio dela a uma única região de planejamento, por exemplo para calcular médias regionais. Em caso de empate na fração, vence a quadra com o menor código, uma regra arbitrária mas fixa, que garante que a mesma entrada produza sempre a mesma escolha. A quadra principal, por conseguinte, simplifica a relação de muitos para muitos entre setores e quadras quando a análise precisa de uma relação de um para um.

A renda padronizada

A renda padronizada, calculada pela função `_renda_padronizada`, transforma a renda média de cada setor em três medidas: o logaritmo, o escore-z do logaritmo e o quintil. O logaritmo é usado porque a distribuição da renda é muito assimétrica, com poucos setores muito ricos e muitos setores de renda menor; o logaritmo comprime essa assimetria e faz com que uma diferença de renda seja medida em proporção, e não em valor absoluto. O escore-z mede quantos desvios-padrão o logaritmo da renda de um setor está acima ou abaixo da média dos setores, e é calculado com a média e o desvio da população de setores, e não de uma amostra, porque os setores de Palmas são o universo, e não uma amostra dele. Só entram setores com renda conhecida, de modo que os setores sob sigilo não recebem nenhuma das três medidas. A renda padronizada, assim, prepara a renda para ser usada em modelos sem que a assimetria domine o resultado.

Os quintis de renda

Os quintis de renda, calculados pela função `_quintis`, dividem os setores com renda conhecida em cinco grupos de tamanho igual, pela posição de cada um na ordem crescente de renda. O quintil 1 reúne os 20% de setores com menor renda, e o quintil 5, os 20% com maior renda. A divisão é feita pela posição, e não por faixas de valor, porque o objetivo é comparar grupos do mesmo tamanho, independentemente de quão desigual seja a distribuição. O empate de renda é resolvido pelo código do setor, de novo para garantir uma saída determinística. Os quintis de renda, por conseguinte, oferecem uma classificação simples e estável da posição socioeconômica de cada setor.

O setor enriquecido

O setor enriquecido é o resultado da função `socioeconomia_setores`, que junta, para cada setor da malha, os dados do Censo, as medidas de renda e o resultado do cruzamento. Antes de juntar, a função verifica se algum código de setor aparece repetido no Censo e, se aparecer, interrompe a execução, porque um código repetido faria a junção multiplicar as linhas daquele setor. Cada setor ganha a densidade em moradores por quilômetro quadrado, calculada com a área medida pelo pipeline, a renda média e a mediana, as três medidas padronizadas e a quadra principal. Ganha também a cobertura, que informa quanto da área do setor cai dentro de quadras do GeoPalmas, limitada a 1 para que a soma de frações com pequenas sobreposições não ultrapasse a área total. O setor enriquecido, assim, reúne numa única linha tudo o que se sabe sobre cada setor.

A quadra estimada

A quadra estimada é o resultado da função `socioeconomia_quadras`, que estima indicadores do Censo para cada quadra segundo a decisão D-TER-05. Como o Censo não publica dados por quadra, a função distribui as contagens de cada setor entre as quadras que ele toca, na proporção da área sobreposta; esse método é a interpolação areal (Lam, 1983). Uma quadra que recebe 30% da área de um setor recebe, portanto, 30% dos seus responsáveis e dos seus moradores. A renda da quadra é estimada como uma média das rendas dos setores que a tocam, ponderada pelo número de responsáveis atribuídos a ela por cada setor, e os setores sem renda não entram nessa média. A quadra estimada, por conseguinte, leva a informação do Censo a um recorte que o Censo não publica.

A suposição da interpolação

A suposição da interpolação areal é que a população se distribui de modo uniforme dentro de cada setor. Essa suposição é necessária porque o pipeline não sabe onde, dentro do setor, as pessoas moram; repartir pela área é a forma mais simples e neutra de fazer a estimativa. A suposição, porém, erra onde o setor tem áreas vazias, como parques, lagos ou terrenos não ocupados: uma quadra vazia que ocupa metade do setor receberia metade da sua população. Por isso, a limitação está escrita no próprio código, e as estimativas por quadra ficam numa tabela separada da tabela por setor, que contém dados observados. A suposição da interpolação, assim, é declarada para que ninguém confunda uma estimativa com uma medição.

CONCLUSÃO — SÍNTESE

A síntese do enriquecimento territorial mostra que todas as suas operações servem para levar a renda do Censo até os recortes que a gestão municipal usa, sem fingir uma precisão que os dados não têm. O cruzamento cria a ponte entre setores e quadras, o descarte do ruído evita pares falsos, a padronização prepara a renda para o modelo, e a interpolação estende o dado até a quadra com a sua suposição declarada. Por isso, cada número territorial do pipeline pode ser classificado como observado, quando vem do setor, ou estimado, quando vem da quadra.

29. Parâmetros da simulação

INTRODUÇÃO — O TODO

Os parâmetros da simulação, no módulo `pipeline/simulacao/parametros.py`, são os números que governam a geração da dívida de IPTU sintética. Esse módulo existe separado do gerador porque os parâmetros são o lugar onde as premissas da simulação ficam visíveis; o gerador apenas executa o que os parâmetros determinam. Cada parâmetro tem uma origem declarada, que pode ser um fato, uma premissa ou uma indicação da literatura, porque quem lê a simulação precisa saber o que nela é dado e o que é suposição. O desenvolvimento a seguir apresenta a tabela dos parâmetros, analisa os grupos de parâmetros conforme a sua origem e explica a assinatura que identifica cada versão.

A tabela dos parâmetros

A tabela abaixo lista cada parâmetro com o seu valor e a sua origem. Nenhum dos valores é uma observação de inadimplência real de Palmas, porque os dados públicos não trazem dívida por contribuinte nem por local. Por essa razão, todo padrão territorial da inadimplência simulada vem destes números, em especial do parâmetro `beta_renda`, e não pode ser apresentado como achado sobre a cidade.

Parâmetro	Valor	Origem
`versao`	1.0.0	identificação da versão
`semente`	20261027	fixa: torna a simulação reprodutível
`n_imoveis`	111.252	fato: imóveis tributáveis informados pela Sefin
`exercicios`	2021 a 2025	premissa: cinco exercícios com arrecadação real observada
`data_corte`	31/07/2026	fato: mesma data de corte da extração NUCLEOGOV
`vencimento`	10 de março	premissa coerente com o pico real de março
`taxa_inadimplencia_alvo`	0,33	premissa: 1 − meta de adimplência de 67% do enunciado
`beta_renda`	−0,5	direção da literatura (Alm et al., 2014); intensidade é premissa
`beta_historico`	1,5	premissa: quem não pagou tende a não pagar de novo
`desvio_efeito_setor`	0,3	premissa: diferença entre setores não explicada pela renda
`elasticidade_valor_renda`	1,0	premissa: valor lançado proporcional à renda do setor
`dispersao_valor`	0,6	premissa: dispersão lognormal do valor lançado
`prob_pagar_antes_inscricao`	0,45	premissa de recuperação antes da inscrição
`prob_pagar_por_ano_inscrito`	0,12	premissa de recuperação por ano depois da inscrição
`dias_ate_inscricao`	365	premissa: prazo até a inscrição em dívida ativa
`juros_mes`	0,01	fato normativo: 1% ao mês (CTN, art. 161, §1º)
`tipos_excluidos`	quartéis e prisões	setores sem IPTU residencial

Os parâmetros de fato

Os parâmetros de fato são os que vêm de uma fonte verificável e não dependem de escolha da equipe. O número de imóveis, 111.252, é o total de imóveis tributáveis de Palmas informado no enunciado da Opção D, com base em dados da Secretaria da Fazenda. A data de corte, 31/07/2026, é a mesma da extração NUCLEOGOV, para que a simulação descreva a dívida no mesmo momento em que a receita real termina. Os juros de 1% ao mês vêm do Código Tributário Nacional, que fixa essa taxa quando a lei não dispõe de outro modo (art. 161, §1º). Os parâmetros de fato, assim, ancoram a simulação na realidade do município sempre que a realidade é conhecida.

Os parâmetros de premissa

Os parâmetros de premissa são escolhas da equipe, plausíveis mas não observadas, e cada um tem uma justificativa. Os cinco exercícios, de 2021 a 2025, foram escolhidos porque todos têm arrecadação real observada no NUCLEOGOV, que é usada para calibrar o valor lançado. O vencimento em 10 de março é coerente com o pico de março que o índice sazonal real do IPTU mostra. A taxa-alvo de 33% é o complemento da meta de adimplência de 67% do enunciado, e não

uma taxa observada. O peso do histórico, 1,5, expressa a premissa de que quem não pagou num ano tende a não pagar no seguinte; o desvio do efeito de setor, 0,3, representa diferenças entre setores que a renda não explica. Os parâmetros de valor e de recuperação, por fim, definem a distribuição dos valores lançados e a chance de pagamento antes e depois da inscrição em dívida ativa. Os parâmetros de premissa, por conseguinte, são suposições declaradas, e qualquer um deles pode ser mudado e testado.

O parâmetro da literatura

O parâmetro `beta_renda`, igual a −0,5, é o único que combina a literatura com uma premissa. A literatura fornece a direção do efeito: imóveis de menor valor e áreas mais pobres apresentam mais inadimplência (Alm et al., 2014). O sinal negativo do parâmetro expressa essa direção, porque, no modelo, quanto maior a renda, menor a chance de atraso. A intensidade, porém, o valor 0,5, é premissa, porque nenhuma fonte disponível permite estimá-la para Palmas. Esse é o parâmetro que produz o padrão territorial da simulação, e é por isso que esse padrão não pode ser apresentado como descoberta. O parâmetro da literatura, assim, é ao mesmo tempo o mais fundamentado e o mais circular de todos.

A semente

A semente, o número 20261027, é o valor inicial do gerador de números aleatórios. A semente existe porque uma simulação depende de sorteios, e sorteios diferentes produzem resultados diferentes; com a semente fixa, o gerador produz sempre a mesma sequência de sorteios, e a simulação dá sempre o mesmo resultado. Essa propriedade é o que permite ao gate G14 exigir que duas transformações gerem os mesmos bytes, e é o que permite a qualquer pessoa reproduzir os números do relatório. A semente, por conseguinte, transforma um processo aleatório num processo reproduzível.

Os tipos de setor excluídos

Os tipos de setor excluídos são os quartéis e bases militares e as unidades prisionais, que o IBGE classifica como tipos especiais de setor. Esses setores ficam fora da simulação porque têm moradores contados pelo Censo, mas não têm imóveis residenciais sujeitos a IPTU; distribuir imóveis simulados para eles criaria contribuintes onde não há. Os tipos excluídos, assim, evitam que a distribuição dos imóveis siga a população em lugares onde a população não corresponde a imóveis tributáveis.

A assinatura

A assinatura, calculada pelo método `assinatura`, é o SHA-256 de todos os parâmetros, serializados em JSON com as chaves em ordem alfabética. A ordem alfabética é exigida porque o hash depende dos bytes exatos, e a mesma lista de parâmetros escrita em outra ordem produziria outro hash. Qualquer mudança num parâmetro, por menor que seja, muda a assinatura. A assinatura identifica a simulação nos arquivos e no banco, e o banco nunca grava duas vezes a mesma assinatura: rodar o pipeline de novo com os mesmos parâmetros reaproveita a simulação já gravada, e só parâmetros novos geram uma simulação nova. A assinatura, por conseguinte, liga cada resultado simulado exatamente às premissas que o produziram.

CONCLUSÃO — SÍNTESE

A síntese dos parâmetros mostra que o módulo existe para tornar a simulação auditável: cada número tem uma origem, e o conjunto tem uma identidade. Os fatos ancoram a simulação na realidade conhecida, as premissas preenchem o que não se conhece com escolhas declaradas, a literatura dá a direção do efeito principal, a semente garante a reprodução, e a assinatura liga os resultados às premissas. Por isso, quem lê o arquivo de parâmetros sabe exatamente o que, na simulação, é fato e o que é suposição.

30. Gerador da simulação

INTRODUÇÃO — O TODO

O gerador da simulação, no módulo `pipeline/simulacao/gerador.py`, produz a dívida de IPTU sintética a partir dos parâmetros e dos setores censitários reais. O gerador cria imóveis, distribui-os pelos setores, define o valor lançado de cada um, decide quem atrasa o pagamento, calcula as datas de pagamento e de inscrição em dívida ativa, aplica os juros e classifica cada crédito na data de corte. Essa sequência imita, de forma simplificada, o ciclo real de um crédito tributário, do lançamento à cobrança. O desenvolvimento a seguir analisa cada passo do gerador, na ordem em que ele os executa.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os sorteios

Os sorteios são todos feitos de uma vez, no início, pela função `sortear`, numa ordem fixa a partir da semente. São sete conjuntos: um efeito aleatório por setor, um fator de valor por imóvel e, para cada imóvel em cada exercício, um sorteio de inadimplência, um de antecedência do pagamento, dois de espera e um de folga para a inscrição. Fazer todos os sorteios de uma vez, e não à medida que são necessários, tem duas vantagens. A primeira é a estabilidade: se uma parte do gerador mudar, por exemplo o número de iterações da calibração, os sorteios das outras partes continuam os mesmos. A segunda é a calibração, que, como se verá, precisa reutilizar exatamente os mesmos sorteios várias vezes. Os sorteios, assim, são a matéria-prima fixa sobre a qual o gerador trabalha.

Os setores elegíveis

Os setores elegíveis, escolhidos pela função `setores_elegiveis`, são os setores que podem receber imóveis simulados. O setor precisa ser urbano, porque o IPTU incide sobre imóveis urbanos; precisa ter responsáveis, porque a distribuição dos imóveis segue o número de domicílios; precisa ter renda conhecida, porque a renda define o valor e a inadimplência; e não pode ser quartel ou prisão. A lista é ordenada pelo código do setor, porque a distribuição dos imóveis e a associação dos sorteios dependem da ordem dos setores, e uma ordem fixa garante o mesmo resultado em qualquer máquina. Os setores elegíveis, por conseguinte, restringem a simulação aos lugares onde ela faz sentido.

A distribuição dos imóveis

A distribuição dos imóveis, feita pela função `alocar`, reparte os 111.252 imóveis entre os setores elegíveis na proporção do número de responsáveis de cada um. O problema dessa repartição é que as cotas proporcionais quase nunca são números inteiros, e não existe um terço de imóvel. O método usado é o dos maiores restos, de Hamilton: primeiro, cada setor recebe a parte inteira da sua cota; depois, os imóveis que sobram vão, um a um, para os setores com os maiores

restos fracionários. Esse método garante que a soma seja exatamente 111.252 e que nenhum setor fique a mais de um imóvel da sua cota exata, sem depender de sorteio. Os empates de resto são resolvidos pela ordem dos setores, mais uma vez de forma determinística. A distribuição dos imóveis, assim, respeita a proporção dos domicílios com a maior exatidão possível em números inteiros.

O modelo de inadimplência

O modelo de inadimplência, implementado pela função `inadimplencia`, decide quais créditos não são pagos até o vencimento. O modelo é logístico: para cada imóvel e exercício, calcula-se um número chamado logito, que é a soma do intercepto, do efeito da renda, do efeito do histórico e do efeito do setor, e a probabilidade de atraso é a função logística desse número, que o converte num valor entre 0 e 1. O crédito é inadimplente se o sorteio de inadimplência for menor que essa probabilidade. O modelo logístico foi escolhido porque é a forma usual de modelar um evento sim ou não a partir de vários fatores, e porque cada parâmetro tem uma interpretação direta: um efeito positivo aumenta a chance de atraso, e um negativo a diminui. O modelo de inadimplência, por conseguinte, transforma as premissas em decisões crédito a crédito.

Os efeitos do modelo

Os efeitos do modelo são três, além do intercepto. O efeito da renda multiplica o escore-z do logaritmo da renda do setor pelo parâmetro `beta_renda`, que é negativo; assim, imóveis em setores mais ricos têm logito menor e atrasam menos, que é o gradiente declarado na ADR-003. O efeito do histórico soma o parâmetro `beta_historico` quando o imóvel atrasou no exercício anterior; no primeiro exercício, ninguém tem histórico, e o efeito é zero para todos. O efeito do setor é o sorteio feito para cada setor, que representa diferenças locais não explicadas pela renda, como a atuação da fiscalização ou o perfil dos imóveis. Os exercícios são decididos em sequência, porque o histórico de cada ano depende do resultado do ano anterior. Os efeitos do modelo, assim, combinam a renda, o comportamento passado e o acaso local.

A calibração do intercepto

A calibração do intercepto, feita pela função `calibrar_beta0`, ajusta o intercepto do modelo até que a taxa média de inadimplência dos cinco exercícios seja igual ao alvo de 33%. O método usado é a bisseção: começa-se com o intervalo de −15 a 15, calcula-se a taxa no ponto médio e, se ela ficar abaixo do alvo, o intercepto precisa subir, e a metade inferior do intervalo é descartada; se ficar acima, a metade superior é descartada. Depois de 60 iterações, o intervalo tem largura menor que a precisão dos números do computador, e o ponto médio é o intercepto calibrado. Na execução de referência, o intercepto ficou em −1,259, a taxa média ficou em 33,00%, e a validação V10 conferiu esse valor com tolerância de meio ponto percentual. A calibração do intercepto, por conseguinte, garante que a simulação atinja a taxa-alvo sem que a equipe precise escolher o intercepto à mão.

Os números aleatórios comuns

Os números aleatórios comuns são a técnica que torna a bisseção possível. A cada iteração, a calibração recalcula a inadimplência usando exatamente os mesmos sorteios; só o intercepto muda. Com os sorteios fixos, aumentar o intercepto aumenta a probabilidade de atraso de cada crédito, e um crédito que já era inadimplente continua inadimplente; além disso, mais atrasos num ano aumentam o histórico do ano seguinte, o que também só pode aumentar os atrasos. A taxa média é, portanto, uma função que só cresce com o intercepto, e é essa propriedade que garante que a

bisseção converge para um único valor. Se os sorteios mudassem a cada iteração, a taxa oscilaria por acaso, e a bisseção poderia descartar a metade errada do intervalo. Os números aleatórios comuns, assim, transformam um problema aleatório num problema determinístico.

O valor relativo de cada imóvel

O valor relativo de cada imóvel define quanto o imóvel vale em comparação com os outros, antes de qualquer conversão em reais. Ele é a renda do setor dividida pela mediana das rendas dos setores elegíveis, elevada à elasticidade, e multiplicada por um fator lognormal sorteado para o imóvel. A divisão pela mediana centra o valor em torno de 1; a elasticidade igual a 1 faz o valor proporcional à renda, de modo que um setor com o dobro da renda tem, em média, imóveis com o dobro do valor; e o fator lognormal introduz a variação entre imóveis do mesmo setor, porque nem todos os imóveis de um bairro valem o mesmo. O fator é lognormal, e não normal, porque valores de imóveis são sempre positivos e têm distribuição assimétrica, com poucos imóveis muito caros. O valor relativo, por conseguinte, reproduz a desigualdade de valores entre setores e dentro de cada setor.

O valor lançado

O valor lançado, calculado pela função `lancamentos`, converte o valor relativo em centavos, exercício por exercício. O total lançado de cada exercício é o principal de IPTU realmente arrecadado naquele ano, segundo o NUCLEOGOV, dividido por 0,67; cada imóvel recebe desse total uma parte proporcional ao seu valor relativo. A divisão por 0,67 existe porque, se cerca de 67% dos créditos são pagos, o principal pago na simulação se aproxima do principal arrecadado de verdade; assim, a escala da simulação fica ancorada na arrecadação real. O valor mínimo de cada lançamento é de R$ 1,00, para que nenhum imóvel receba um lançamento nulo por arredondamento. Um exercício sem arrecadação real registrada interrompe a simulação, porque não haveria como calibrar o seu total. O valor lançado, assim, combina a distribuição simulada com o total observado.

As datas

As datas de cada crédito são calculadas pela função `_datas`. O vencimento é 10 de março de cada exercício. O crédito pago em dia é quitado entre 0 e 20 dias antes do vencimento, conforme o sorteio de antecedência. O crédito inadimplente tem um atraso sorteado de uma distribuição exponencial, cuja taxa é escolhida para que 45% dos inadimplentes paguem antes de 365 dias; quem paga nesse prazo fica como pago em atraso. A distribuição exponencial descreve o tempo até um evento que pode acontecer a qualquer momento com chance constante, como a decisão de quitar uma dívida, e por isso serve a esse papel. Quem não paga em 365 dias é inscrito em dívida ativa, com uma folga sorteada de até 60 dias, que representa o tempo administrativo da inscrição; depois da inscrição, a chance de pagar é de 12% por ano, também por uma espera exponencial. As datas, por conseguinte, simulam o percurso de cada crédito do vencimento à cobrança.

O corte

O corte é a regra que descarta os eventos posteriores a 31/07/2026. Um pagamento sorteado para depois dessa data é tratado como não ocorrido, e uma inscrição posterior também. Essa regra existe porque a simulação descreve a situação da dívida no dia do corte, e um evento futuro ainda não aconteceu nesse dia; sem o corte, a simulação conheceria o futuro, o que nenhum dado real permite. O corte, assim, faz da simulação uma fotografia tirada num dia determinado.

A situação no corte

A situação no corte, determinada pela função `_situacao`, classifica cada crédito numa de cinco situações. PAGO_EM_DIA é o crédito que não atrasou; PAGO_EM_ATRASO é o que atrasou e foi pago antes da inscrição; RECUPERADO_DIVIDA_ATIVA é o que foi inscrito e depois pago; DIVIDA_ATIVA é o que foi inscrito e continua sem pagamento. A quinta situação, VENCIDO_NAO_INSCRITO, vale para o crédito que venceu, não foi pago e ainda não foi inscrito. As condições são avaliadas nessa ordem, e a primeira verdadeira define a situação, de modo que cada crédito recebe exatamente uma. Com os parâmetros atuais, a quinta situação nunca ocorre, porque o último vencimento, somado ao prazo de inscrição e à folga máxima, ainda cai antes do corte, como se discute nas observações da releitura. A situação no corte, por conseguinte, traduz as datas numa categoria que a gestão tributária reconhece.

Os juros

Os juros, aplicados pela função `_com_juros`, são simples, de 1% ao mês sobre o principal. Cada mês completo de 30 dias desde o vencimento soma 1% do principal, e o resultado é arredondado para o centavo. Os juros são simples, e não compostos: cada mês acrescenta 1% do principal, e não 1% do valor já acrescido. A taxa de 1% vem do Código Tributário Nacional, que a fixa quando a lei não dispõe de outro modo, e a forma simples é uma premissa da simulação. O valor pago inclui os juros até a data do pagamento, e o saldo no corte inclui os juros até 31/07/2026; um crédito pago tem saldo zero, e um crédito pago em dia não tem juros. Os juros, assim, fazem o valor de cada crédito crescer com o tempo de atraso, como acontece na cobrança real.

O resultado da simulação

O resultado da simulação, devolvido pela função `simular`, reúne a tabela de imóveis, a tabela de créditos, o intercepto, a taxa por exercício, a assinatura e um resumo. Na execução de referência, foram 111.252 imóveis e 556.260 créditos, um por imóvel em cada um dos cinco exercícios. O resumo traz o intercepto, a taxa média, o estoque de dívida ativa no corte, de cerca de R$ 79,9 milhões, e a contagem de créditos por situação: 372.694 pagos em dia, 82.463 pagos em atraso, 22.981 recuperados da dívida ativa e 78.122 em dívida ativa. O resultado da simulação, por conseguinte, é uma dívida completa, com todas as suas datas e valores, pronta para virar base analítica.

CONCLUSÃO — SÍNTESE

A síntese do gerador mostra que cada um de seus passos é uma consequência direta dos parâmetros, e nenhum é uma observação. Os sorteios fixos e a ordem fixa dos setores garantem a reprodução; a distribuição de Hamilton garante a proporção; o modelo logístico e a calibração garantem a taxa-alvo; as datas, o corte e os juros garantem um ciclo de crédito coerente. Por isso, a dívida simulada é plausível e reprodutível, mas o seu padrão territorial é consequência da premissa da renda, e não uma descoberta sobre Palmas.

31. Atributos da base analítica

INTRODUÇÃO — O TODO

A base analítica, montada pelo módulo `pipeline/atributos/base_analitica.py`, é a tabela que o modelo de recuperabilidade da Sprint 4 vai usar. Cada linha é um crédito, isto é, um imóvel em um exercício, e a base tem 556.260 linhas. A base existe porque o modelo precisa receber, para cada crédito, atributos já calculados e prontos, como o tempo de inadimplência, a faixa de valor e o histórico de pagamento; calcular esses atributos dentro do modelo misturaria preparação de dados com estatística. O desenvolvimento a seguir analisa os cruzamentos que formam a base, cada atributo, o cuidado com o vazamento temporal, a marca de origem e o agregado por setor.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os cruzamentos

Os cruzamentos da base juntam cada crédito ao seu imóvel e cada imóvel ao seu setor. As duas junções são feitas à esquerda, isto é, mantendo todos os créditos, e exigem que a chave do lado direito seja única: cada imóvel aparece uma só vez na tabela de imóveis, e cada setor uma só vez na tabela de setores. Essa exigência é conferida pela própria biblioteca no momento da junção e existe porque uma chave repetida do lado direito faria a junção duplicar créditos sem aviso. Depois da junção com os setores, um indicador confere se todos os créditos encontraram o seu setor, e, se algum ficou sem setor, a função levanta erro. Os cruzamentos, assim, são construídos de forma que não podem nem multiplicar nem perder créditos, e a validação V09 confirma isso contando as linhas antes e depois.

O tempo de inadimplência

O tempo de inadimplência conta os dias entre o vencimento e o fim do atraso. O fim é a data de pagamento, quando o crédito foi pago, ou a data de corte, quando ainda não foi; o resultado nunca é negativo. O crédito pago em dia tem tempo zero, porque o pagamento antecipado não é um atraso negativo, e sim ausência de atraso. A partir dos dias, a base calcula os meses de inadimplência, em blocos completos de 30 dias, e uma faixa de tempo, que vai de EM_DIA a MAIS_DE_5_ANOS, passando por até um ano, de um a dois anos e de dois a cinco anos. As faixas existem porque um gestor raciocina em períodos, e não em números exatos de dias. O tempo de inadimplência, por conseguinte, é oferecido em três formas, dias, meses e faixa, para usos diferentes.

A sazonalidade do crédito

A sazonalidade do crédito é descrita por três colunas derivadas da data de vencimento. O mês do vencimento é extraído da data, o trimestre é calculado a partir do mês, e o índice sazonal é copiado do índice real do IPTU para o mês do vencimento. O índice real é usado porque liga o crédito simulado ao calendário real de arrecadação: um crédito que vence em março, mês de pico, carrega o índice de cerca de 3,5. Na simulação atual, todos os créditos vencem em março, e por isso essas colunas são constantes; elas existem para que a mesma base funcione com os dados reais da Sefin, em que os vencimentos podem variar. A sazonalidade do crédito, assim, prepara a base para uma variação que só os dados reais vão trazer.

A faixa de valor

A faixa de valor classifica o principal de cada crédito em seis faixas com limites fixos em reais: até 500, de 500 a 1.000, de 1.000 a 2.500, de 2.500 a 5.000, de 5.000 a 10.000 e acima de 10.000. Cada faixa é fechada à esquerda, ou seja, um valor exatamente igual a R$ 500 pertence à segunda faixa. Os limites são fixos, e não quantis calculados sobre os dados, por dois motivos: quantis mudariam a cada exercício e a cada execução, tornando as faixas incomparáveis ao longo do tempo, e limites redondos em reais podem ser explicados a um gestor, enquanto um quantil exige explicar a distribuição inteira. A faixa de valor, por conseguinte, privilegia a estabilidade e a legibilidade sobre a adaptação aos dados.

O histórico de pagamento

O histórico de pagamento descreve o comportamento passado do imóvel e é calculado olhando apenas para trás. A coluna `inadimplencias_anteriores` conta quantas vezes o mesmo imóvel atrasou nos exercícios anteriores, e o próprio exercício nunca entra na conta; essa regra existe porque incluir o exercício atual daria ao modelo a resposta que ele deveria prever. A coluna `inadimplente_exercicio_anterior` informa se o imóvel atrasou no ano imediatamente anterior. No primeiro exercício, esse campo fica vazio, e não zero, porque não há ano anterior observado: zero afirmaria que o imóvel pagou em dia, o que ninguém sabe. O histórico de pagamento, assim, oferece ao modelo apenas o que já tinha acontecido no momento de cada lançamento.

O vazamento temporal

O vazamento temporal é o erro de oferecer a um modelo uma informação que, no momento da previsão, ainda não existiria. Na base analítica, algumas colunas só existem depois do vencimento: o tempo de inadimplência, a situação no corte e o saldo no corte. Essas colunas descrevem o estoque da dívida e servem para priorizar a recuperação de créditos já vencidos, mas não podem ser usadas para prever, no lançamento, se um crédito vai atrasar, porque seriam a própria resposta disfarçada de atributo (Kaufman et al., 2012). Para evitar esse erro, o dicionário da base classifica cada coluna pelo momento em que ela fica disponível, e o modelo da Sprint 4 deve usar apenas as colunas disponíveis no momento da decisão que ele apoia. O cuidado com o vazamento temporal, por conseguinte, protege o modelo de um desempenho falso, excelente nos testes e inútil na prática.

A marca de origem e o tributo

A marca de origem é a coluna `origem_dado`, que vale SINTETICO em todas as linhas da base. Essa marca existe para que nenhum usuário da base, nem nenhuma consulta no banco, possa confundir um crédito simulado com um crédito real; ela acompanha o dado em todos os arquivos e é imposta por restrição no banco. As colunas do tributo seguem a codificação one-hot: `tributo_IPTU` vale 1, e as colunas do ISSQN e do ITBI valem 0, porque a simulação só gera créditos de IPTU. A coluna `inadimplente` é convertida em 0 ou 1, a forma que os modelos esperam. A marca de origem, assim, garante que a natureza sintética do dado nunca se perca.

O agregado por setor

O agregado por setor, calculado pela função `agregar_por_setor`, resume a base por setor censitário para o mapa e o painel. Para cada setor, ele calcula o número de imóveis e de créditos, a taxa de inadimplência, o estoque aberto e a taxa do último exercício. O agregado é juntado à lista completa de setores, para que mesmo os setores sem imóveis simulados apareçam na tabela, e não desapareçam do mapa como se não existissem. A taxa do último exercício é calculada à parte porque descreve a situação mais recente, que a média dos cinco exercícios dilui; na simulação atual, as duas diferem bastante, porque a taxa simulada cresce ao longo dos anos. O agregado por setor, por conseguinte, é a ponte entre a base crédito a crédito e a visualização territorial.

A supressão de grupos pequenos

A supressão de grupos pequenos esvazia a taxa e o estoque dos setores com menos de dez imóveis. Essa regra segue o princípio de que um dado agregado não deve permitir reconhecer um contribuinte (Sweeney, 2002): num setor com três imóveis, uma taxa de inadimplência de 33% revela, na prática, que um vizinho específico não pagou. A supressão vale mesmo para o dado sintético, que não identifica ninguém, por uma razão de método: o mapa e o painel já nascem com a regra que os dados reais da Sefin vão exigir, e a troca da simulação pelos dados reais não vai precisar de nenhuma mudança na visualização. Os setores sem imóvel simulado também ficam marcados como suprimidos. A supressão de grupos pequenos, assim, antecipa na simulação a proteção que a realidade vai exigir.

CONCLUSÃO — SÍNTESE

A síntese da base analítica revela que todos os seus atributos foram construídos com duas preocupações: a integridade dos cruzamentos e o momento em que cada informação existe. Os cruzamentos não perdem nem multiplicam créditos; o histórico olha só para trás; o vazamento temporal é declarado coluna a coluna; a origem sintética é marcada; e a supressão protege os contribuintes. Por isso, a base entrega à Sprint 4 atributos prontos e classificados pelo momento de uso, e a mesma estrutura poderá receber os dados reais sem ser redesenhada.

32. Validações de qualidade

INTRODUÇÃO — O TODO

As validações de qualidade, no módulo `pipeline/qualidade/validacoes.py`, são as regras que decidem se os dados podem avançar de um filtro para o seguinte. Cada regra examina uma propriedade dos dados, devolve um resultado com evidência numérica e é classificada como impeditiva, quando a sua reprovação interrompe o pipeline, ou informativa, quando apenas avisa. Essas regras existem porque um pipeline automático não tem ninguém olhando os dados a cada execução; se um arquivo vier truncado ou uma fonte mudar de formato, só uma regra escrita de antemão pode perceber. O desenvolvimento a seguir analisa o resultado de uma regra, o verificador que as acumula, cada uma das regras impeditivas e as regras informativas.

O resultado de uma regra

O resultado de uma regra é o objeto `Resultado`, que guarda seis informações: o nome da regra, a dimensão de qualidade, a etapa em que foi avaliada, se é impeditiva, se passou e a evidência. A dimensão segue a classificação de qualidade de dados da literatura (Wang; Strong, 1996; Batini et al., 2009), que distingue, entre outras, a completude, a validade, a unicidade, a consistência, a integridade, a atualidade e a exatidão. Essa classificação existe porque cada dimensão responde a uma pergunta diferente sobre o dado, e saber a dimensão de uma falha orienta a correção: uma falha de completude pede para recoletar, enquanto uma de validade pede para examinar a regra de limpeza. A evidência é sempre um texto com números, como "3 de 3.542 rejeitados", porque uma reprovação sem números obrigaria a refazer a investigação. O resultado de uma regra, assim, é ao mesmo tempo um veredito e a sua justificativa.

O verificador

O verificador, a classe `Verificador`, acumula os resultados de todas as regras de uma execução. Cada regra avaliada é registrada no log no mesmo instante, com a evidência, de modo que, mesmo que a execução pare logo depois, o log já contém tudo o que foi verificado até ali. O nível da mensagem depende do resultado: regra aprovada sai como INFO, impeditiva reprovada como ERROR e informativa reprovada como WARNING, e a palavra registrada é "ok", "REPROVADA" ou "aviso". Os níveis diferentes existem porque permitem filtrar o log e encontrar as falhas em segundos, mesmo num arquivo com centenas de linhas. O verificador, por conseguinte, transforma cada validação num evento datado e classificado da história da execução.

A exigência das regras impeditivas

A exigência das regras impeditivas é feita pelo método `exigir`, que interrompe o pipeline se alguma regra impeditiva reprovou. O método não é chamado depois de cada regra, mas ao fim de cada bloco de regras, e essa escolha tem uma razão: todas as regras do bloco são avaliadas e registradas antes da interrupção, de modo que o erro lista todas as reprovações, e não apenas a primeira. Se o pipeline parasse na primeira falha, a equipe corrigiria um problema, executaria de novo e só então descobriria o segundo, num ciclo lento de tentativa e erro. O erro levantado é um `ErroQualidade`, que leva o código de saída 2, tem o nome da primeira regra reprovada e traz, na evidência, a lista de todas as reprovações. A exigência das regras impeditivas, assim, interrompe a execução com o diagnóstico completo.

As regras impeditivas

As regras impeditivas são doze, identificadas de V01 a V12, e algumas são avaliadas mais de uma vez, uma para cada fonte ou chave. A tabela abaixo resume o que cada regra exige. Na execução de referência, foram 33 avaliações impeditivas: sete da V01, uma por fonte; seis da V02 e seis da V03, uma por filtro; quatro da V04, uma por chave; uma de cada regra de V05 a V10; três da V11, uma por domínio; e uma da V12.

Regra	Dimensão	O que exige
V01	completude	coleta completa, com o total anunciado, para cada uma das sete fontes
V02	consistência	lidos = aceitos + rejeitados + descartados, em cada um dos seis filtros
V03	validade	no máximo 1% de rejeições em cada filtro

Regra	Dimensão	O que exige
V04	unicidade	chaves sem repetição: arrecadação, setores, quadras e IPCA
V05	validade	contrato do NUCLEOGOV e do Censo: colunas e tipos
V06	consistência	conta-pai do NUCLEOGOV igual à soma dos quatro componentes
V07	completude	IPCA sem lacuna e com índice para todo mês da receita
V08	integridade	todo setor do Censo existe na malha do IBGE
V09	consistência	o cruzamento da base não perde nem multiplica créditos
V10	exatidão	taxa simulada igual ao alvo, com tolerância de 0,5 ponto percentual
V11	validade	faixa de valor, faixa de tempo e mês dentro do domínio
V12	integridade	cadeia de hashes do manifesto e arquivos conferidos

As regras da coleta e da limpeza

As regras da coleta e da limpeza, de V01 a V06, protegem a entrada do pipeline. A regra V01 exige que a coleta de cada fonte esteja completa e que o número de registros obtidos seja igual ao anunciado, porque uma coleta parcial geraria resultados parciais sem aviso. A regra V02 exige a identidade da reconciliação em cada filtro, para que nenhum registro desapareça. A regra V03 limita as rejeições a 1% dos registros lidos, um limite proposto pela apostila de ETL, porque uma proporção maior indica que a fonte mudou de formato, e não que alguns registros estão errados; quando nenhum registro é lido, a regra trata a proporção como 100% e reprova, porque uma fonte vazia não pode ser aprovada por ter "zero rejeições". A regra V04 exige chaves sem repetição; na arrecadação, a chave é o par formado pelo identificador e pelo mês, porque um registro mensal gera doze linhas com o mesmo identificador. As regras V05 e V06 exigem que o NUCLEOGOV e o Censo cumpram os seus contratos e que a conta-pai do NUCLEOGOV seja igual à soma dos componentes. As regras da coleta e da limpeza, assim, garantem que o enriquecimento receba dados completos, contados e coerentes.

As regras do enriquecimento

As regras do enriquecimento, V07 e V08, protegem as junções com o IPCA e com a malha territorial. A regra V07 exige que a série do IPCA não tenha lacunas e que todo mês de que a receita precisa tenha um índice; sem ela, a deflação falharia no meio do cálculo ou, pior, usaria um índice errado. A regra V08 exige que todo setor presente no Censo exista na malha de setores do IBGE, porque um setor do Censo sem geometria não poderia ser cruzado nem mapeado, e a sua existência indicaria que o Censo e a malha são de versões diferentes. Quando reprovam, essas regras mostram até três exemplos das chaves problemáticas, para que a correção comece sem que alguém precise abrir os arquivos. As regras do enriquecimento, por conseguinte, garantem que cada junção encontre o seu par.

As regras da simulação e dos atributos

As regras da simulação e dos atributos, de V09 a V11, protegem a saída do pipeline. A regra V09 compara o número de créditos antes e depois do cruzamento que forma a base analítica, porque uma junção mal feita pode duplicar ou perder linhas sem gerar erro. A regra V10 exige que a taxa simulada fique a no máximo meio ponto percentual do alvo de 33%, confirmando que a calibração funcionou. A regra V11 exige que a faixa de valor, a faixa de tempo e o mês do vencimento assumam apenas os valores previstos: os seis rótulos de valor, os cinco rótulos de tempo e os números de 1 a 12. A regra V11 existe porque um valor fora do domínio, como um mês 13, indicaria

um erro de cálculo que passaria despercebido até alguém montar um gráfico. As regras da saída, assim, garantem que a base entregue à Sprint 4 tenha a forma que o dicionário promete.

A regra da integridade do bruto

A regra V12 confere a integridade do bruto antes de qualquer leitura: percorre a cadeia de hashes do manifesto e confere o SHA-256 de cada arquivo registrado. Ela é a primeira regra avaliada na transformação, antes mesmo da V01, porque, se o bruto foi alterado, todas as regras seguintes estariam examinando dados que não são os coletados. Quando reprova, a regra mostra até três dos problemas encontrados, como uma linha com a cadeia quebrada ou um arquivo com hash diferente. A regra da integridade do bruto, por conseguinte, é a guarda que impede o pipeline de trabalhar sobre um bruto adulterado.

As regras informativas

As regras informativas avisam, mas não interrompem. A regra I01 avisa quando o último IPCA disponível tem mais de 120 dias; ela não bloqueia porque a defasagem pode significar apenas que o Banco Central ainda não publicou o mês, e não que o dado esteja errado. As regras I02 a I05 têm uma natureza diferente: elas sempre passam e existem para registrar contagens no log. A I02 informa a cobertura das quadras, isto é, a parcela dos domicílios em setores com quadra do GeoPalmas, que foi de 94,2%; a I03 informa quantos setores estão sob sigilo, que foram 23; a I04 informa as contagens antes e depois da unificação da receita; e a I05 informa as contagens antes e depois do cruzamento entre setores e quadras. Essas contagens atendem à exigência do Módulo 5 de registrar o tamanho dos dados antes e depois de cada cruzamento, para que uma redução da população nunca passe em silêncio. As regras informativas, assim, documentam o que não justifica uma interrupção, mas precisa ser conhecido.

CONCLUSÃO — SÍNTESE

A síntese das validações revela que elas funcionam como portões entre os filtros, cada portão examinando uma dimensão diferente da qualidade. As regras da coleta e da limpeza guardam a entrada, as do enriquecimento guardam as junções, as da simulação e dos atributos guardam a saída, e a do bruto guarda a origem. Todas registram a sua evidência no momento em que são avaliadas, e todas as de um bloco são avaliadas antes de qualquer interrupção. Por isso, só dado aprovado chega às camadas finais e ao banco, e, quando algo é reprovado, o log já contém o diagnóstico completo.

33. Encadeamento da transformação

INTRODUÇÃO — O TODO

O encadeamento da transformação, no módulo `pipeline/transformacao/fluxo.py`**, é a parte que chama os filtros na ordem certa e coloca as validações entre eles.** Esse módulo materializa o estilo de pipes e filtros: os filtros não se conhecem, e é o encadeamento que entrega a saída de um como entrada do outro. A regra central do módulo é que um filtro nunca recebe uma entrada já reprovada, porque um filtro que processa dados ruins produz resultados ruins com aparência de bons. O desenvolvimento a seguir analisa a ordem geral, a etapa de limpeza, a etapa de enriquecimento, a etapa de simulação e atributos e o objeto que reúne os produtos.

A ordem geral

A ordem geral da transformação é limpar, validar, enriquecer, validar, simular, derivar atributos e validar. Cada bloco de validações é exigido antes do filtro seguinte, de modo que o enriquecimento só começa se a limpeza foi aprovada, e a simulação só começa se o enriquecimento foi aprovado. A gravação dos arquivos não faz parte deste módulo: ela acontece no orquestrador, depois que a transformação devolve os produtos, e a transformação só os devolve depois do último bloco de validações. Essa separação garante que nenhuma gravação aconteça antes que tudo tenha sido aprovado. A ordem geral, assim, transforma a sequência dos filtros numa sequência de portões.

A limpeza e as suas validações

A etapa de limpeza, na função `limpar`, começa pela validação V01 de cada uma das sete coletas e, em seguida, chama o filtro de cada fonte. Os filtros são chamados mesmo que alguma coleta tenha reprovado na V01, porque a exigência só acontece ao fim do bloco, e as reprovações da limpeza também interessam ao diagnóstico. Depois dos filtros, a função aplica a reconciliação e a proporção de rejeição a cada um dos seis filtros, o contrato e a conciliação ao NUCLEOGOV e a unicidade às quatro chaves. Ao fim, a função de transformação exige o bloco inteiro. A etapa de limpeza, por conseguinte, produz um diagnóstico completo da entrada antes de decidir se o pipeline continua.

O tratamento do contrato do Censo

O tratamento do contrato do Censo é feito pela função `_renda`, que envolve a limpeza da renda com um cuidado especial. Se a limpeza do Censo levanta `ErroContrato`, por exemplo porque falta uma coluna, a função registra a validação V05 do Censo como reprovada e só então interrompe a execução com um `ErroQualidade`. Esse cuidado existe porque um `ErroContrato` comum interromperia o pipeline como uma exceção qualquer, com o código 1, e a falha não apareceria na lista de validações do log e do banco. Registrando a regra antes de interromper, o pipeline devolve o código 2, de qualidade, e o motivo fica documentado como uma validação reprovada. O tratamento do contrato do Censo, assim, faz com que uma falha estrutural da fonte seja relatada da mesma forma que qualquer outra falha de qualidade.

O enriquecimento e as suas validações

A etapa de enriquecimento, na função `enriquecer`, começa por unificar a receita em valores nominais, antes de deflacioná-la. Essa ordem existe porque a validação V07 precisa saber quais meses de IPCA a receita vai exigir, e só a receita unificada permite listá-los; a função `_competencias` faz essa lista, com o próprio mês para as linhas mensais e os doze meses do ano para as linhas anuais. A função avalia então as regras V07 e V08 e as exige antes da deflação, porque um mês sem índice faria a deflação falhar no meio do caminho. Aprovadas as regras, vêm a deflação, o registro da regra I04, o cruzamento territorial, o registro da I05, o enriquecimento dos setores e o registro das regras I02, I03 e I01. A função `_cobertura` calcula a parcela dos domicílios em setores que têm quadra principal, que alimenta a I02. A etapa de enriquecimento, por conseguinte, valida cada junção imediatamente antes de executá-la.

A simulação e os atributos

A etapa de simulação e atributos, na função `transformar`, começa depois do enriquecimento aprovado. A função `_principal_iptu` soma, por ano, o principal de IPTU realmente arrecadado, em valores nominais, porque o valor lançado de um exercício é cobrado na moeda daquele ano; esse total calibra o valor lançado da simulação. Depois da simulação, a regra V10 confere a calibração. Em seguida, a função calcula a sazonalidade da receita real e monta a base analítica, entregando a ela o índice sazonal do IPTU por mês. Por fim, a função `_validar_base` aplica a regra V09 e as três avaliações da V11, e o último bloco é exigido. A etapa de simulação e atributos, assim, só entrega a base depois de conferir a calibração, a cardinalidade e os domínios.

Os produtos

Os produtos são o objeto `Produtos`, que reúne tudo o que a transformação produziu: os resultados da limpeza, a receita, a referência do IPCA, o número de linhas da API fora da série, os indicadores, a sazonalidade, o cruzamento, os setores e as quadras enriquecidos, a simulação, a base analítica e o agregado por setor. O verificador vai junto, com todos os resultados das validações. Reunir tudo num único objeto permite que a gravação, a carga e o relatório de qualidade usem exatamente os mesmos dados, sem recalcular nada. Os produtos, por conseguinte, são a fotografia completa e aprovada da transformação.

CONCLUSÃO — SÍNTESE

A síntese do encadeamento mostra que a sua única função é impor a ordem e os portões do pipeline. Cada filtro recebe apenas o que foi aprovado, cada junção é validada antes de acontecer, e todas as regras de um bloco são avaliadas antes de qualquer interrupção. Como a gravação só acontece depois que o encadeamento devolve os produtos, nenhum arquivo é escrito antes da última validação, e uma execução reprovada não deixa resultados pela metade.

34. Gravação das camadas e formato dos arquivos

INTRODUÇÃO — O TODO

A gravação das camadas, nos módulos `pipeline/transformacao/gravacao.py` e `pipeline/tabelas.py`, escreve em disco as camadas limpa e pronta e o relatório de qualidade da execução. O formato dos arquivos foi escolhido para atender a três exigências ao mesmo tempo: abrir corretamente no Excel e no Power BI, não expor quem os abre a fórmulas maliciosas e ser idêntico, byte a byte, entre duas execuções com a mesma entrada. O desenvolvimento a seguir analisa o conteúdo de cada camada, o relatório de qualidade, o formato CSV, a neutralização de fórmulas, a reprodutibilidade dos bytes e a troca atômica dos arquivos.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A camada limpa

A camada limpa, gravada pela função `gravar_limpo`, contém uma tabela por fonte e a quarentena. As seis tabelas são a arrecadação da API, a receita do NUCLEOGOV, as quadras do GeoPalmas, os setores do IBGE, a renda do Censo e o IPCA, todas já tipadas e com as decisões de limpeza aplicadas. A quarentena, no arquivo `quarentena.csv`, reúne as rejeições de todos os filtros, cada uma com o filtro, a origem, a chave e o motivo. A camada limpa existe separada da pronta porque serve a um uso diferente: ela permite conferir a limpeza de cada fonte isoladamente, antes de qualquer cruzamento. A camada limpa, assim, é o ponto de auditoria entre o bruto e a análise.

A camada pronta

A camada pronta, gravada pela função `gravar_pronto`, contém as tabelas prontas para o painel e para o modelo. São nove arquivos: a receita tributária, os indicadores anuais, a sazonalidade, o cruzamento entre setores e quadras, os setores e as quadras com dados socioeconômicos, os imóveis simulados, a base analítica e a inadimplência simulada por setor. Os dois arquivos grandes, os imóveis simulados e a base analítica, são gravados compactados em `.csv.gz`, porque a base tem mais de meio milhão de linhas e ocuparia dezenas de megabytes sem compactação. A tabela de imóveis recebe, na gravação, a coluna `origem_dado` com o valor SINTETICO, para que a marca acompanhe também esse arquivo. A camada pronta, por conseguinte, entrega cada produto na forma que o seu usuário precisa.

O relatório de qualidade

O relatório de qualidade é o arquivo `qualidade_dados.json`, gravado na camada pronta ao fim de cada transformação. Ele reúne o contexto da execução, com o identificador, a versão do código e os parâmetros da simulação; a referência do IPCA; o número de linhas da API fora da série; o resumo da simulação, com a assinatura e a taxa por exercício; o resultado de todas as validações; e o número de linhas de cada arquivo gravado. Esse relatório existe porque os arquivos de dados, sozinhos, não dizem de qual execução vieram nem se foram aprovados; o relatório liga cada conjunto de arquivos à execução que o produziu e às regras que ele passou. Somados os sete arquivos da camada limpa, os nove da camada pronta e o relatório, a transformação grava 17 arquivos. O relatório de qualidade, assim, é a certidão de origem dos dados gravados.

O formato CSV

O formato CSV adotado tem cinco convenções, cada uma com uma razão. A codificação é UTF-8 com marca de ordem de bytes, porque, sem essa marca, o Excel no Windows supõe outra codificação e transforma os acentos em símbolos estranhos. O separador é o ponto e vírgula, porque, na configuração brasileira, a vírgula é o separador decimal, e um arquivo separado por vírgulas abriria com as colunas misturadas. O separador decimal é o ponto, a convenção das linguagens de programação, de modo que o próprio pipeline relê esses arquivos na EDA sem ambiguidade. O fim de linha é o caractere LF, o mesmo em qualquer sistema, para que o arquivo gerado no Windows e no Linux tenha os mesmos bytes. A ausência é gravada como campo vazio, e não como "None" ou "NaN", porque esses textos seriam lidos como palavras; e as datas saem no formato ISO, ano-mês-dia, que não se confunde entre o padrão brasileiro e o americano. O formato CSV, por conseguinte, foi ajustado para ser lido corretamente por pessoas e por programas.

A conversão dos valores em texto

A conversão dos valores em texto, feita pela função `_texto`, decide como cada tipo de valor aparece no arquivo. O valor nulo vira campo vazio, e o número de ponto flutuante inválido, chamado NaN, também. O número de ponto flutuante válido é escrito na sua representação mais curta e exata, que, lida de volta, devolve exatamente o mesmo número. O `Decimal` é escrito em notação fixa, sem notação científica, porque um valor como 1E+2 seria lido como texto por muitos programas. O texto passa pela neutralização de fórmulas, e os demais tipos são convertidos diretamente. A conversão dos valores, assim, garante que cada número sobreviva à ida e volta pelo arquivo sem perder exatidão.

A neutralização de fórmulas

A neutralização de fórmulas protege quem abre os arquivos numa planilha. Os textos dos arquivos vêm de fontes externas, como as descrições da API, os nomes do GeoPalmas e os motivos da quarentena, e um texto que comece por `=`, `+`, `-`, `@`, tabulação ou retorno seria interpretado pela planilha como fórmula. Uma fórmula maliciosa inserida numa fonte pública poderia, ao ser aberta, executar comandos ou enviar dados para fora, um ataque conhecido como injeção em CSV e descrito pela OWASP. A neutralização acrescenta um apóstrofo no início desses textos, o que faz a planilha exibi-los como texto em vez de executá-los. Só valores do tipo texto passam por essa regra, de modo que um número negativo, que também começa por `-`, não é alterado. A neutralização de fórmulas, por conseguinte, fecha um caminho de ataque que vai da fonte pública até o computador de quem analisa os dados.

A reprodutibilidade dos bytes

A reprodutibilidade dos bytes é a propriedade de que a mesma entrada produza arquivos idênticos byte a byte. Para os arquivos compactados, essa propriedade exige um cuidado: o formato gzip guarda no cabeçalho a data da compactação, e cada execução gravaria uma data diferente; por isso, a gravação fixa essa data em zero. Para o relatório JSON, a gravação usa chaves em ordem alfabética, indentação fixa, valores decimais escritos como texto exato e uma quebra de linha final. Essa propriedade existe porque é ela que permite ao gate G14 verificar, comparando hashes, que o pipeline é reprodutível; sem ela, dois arquivos com o mesmo conteúdo poderiam ter bytes diferentes, e a verificação seria impossível. A reprodutibilidade dos bytes, assim, transforma a reprodutibilidade de uma promessa numa propriedade verificável por máquina.

A troca atômica

A troca atômica é a forma como cada arquivo é gravado: primeiro como um arquivo temporário, com a extensão `.tmp`, e depois o temporário substitui o definitivo numa única operação do sistema operacional. Essa técnica existe por duas razões. A primeira é que um programa que esteja lendo o arquivo, como o Power BI atualizando um painel, nunca vê um arquivo pela metade, porque a substituição acontece de uma só vez. A segunda é que, se o pipeline for interrompido durante a gravação, o arquivo anterior continua intacto, e só o temporário fica incompleto. A troca atômica, por conseguinte, garante que cada arquivo da camada esteja sempre inteiro, seja na versão antiga, seja na nova.

CONCLUSÃO — SÍNTESE

A síntese da gravação mostra que o formato dos arquivos foi pensado para três públicos ao mesmo tempo. Para as pessoas, a codificação, o separador e as datas fazem o arquivo abrir corretamente; para a segurança, a neutralização impede a execução de fórmulas; para a verificação, a data fixa do gzip, a ordem do JSON e a troca atômica garantem arquivos idênticos e sempre inteiros. Por isso, os arquivos gravados são ao mesmo tempo legíveis, seguros e reprodutíveis, e o gate G14 confere esta última propriedade byte a byte.

35. Operações geométricas

INTRODUÇÃO — O TODO

As operações geométricas, no módulo `pipeline/geo.py`, são as funções que projetam coordenadas, medem áreas e convertem geometrias em texto. Essas funções foram reunidas num único módulo porque a limpeza, o enriquecimento e a EDA precisam medir e desenhar o território da mesma forma; se cada etapa usasse uma projeção diferente, a área de uma quadra na limpeza não seria igual à área da mesma quadra no cruzamento. O desenvolvimento a seguir analisa os sistemas de referência usados, a transformação entre eles e as funções do módulo.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O sistema de referência das coordenadas

O sistema de referência das coordenadas recebidas é o SIRGAS 2000 geográfico, identificado como EPSG:4674, em que cada ponto é dado por longitude e latitude em graus. Esse sistema é adequado para localizar pontos, mas não para medir áreas, porque um grau não tem comprimento fixo: um grau de longitude mede cerca de 111 km no equador e diminui até zero nos polos. A malha do GeoPalmas vem em WGS 84, outro sistema geográfico, mas a diferença entre WGS 84 e SIRGAS 2000 é menor que um metro, abaixo da precisão das duas malhas, e por isso as duas são tratadas como iguais. O sistema de referência das coordenadas, assim, é único para todas as fontes, e a pequena diferença entre elas é declarada.

A projeção métrica

A projeção métrica usada para medir áreas é o SIRGAS 2000 / UTM zona 22 Sul, identificada como EPSG:31982, em que cada ponto é dado em metros. A projeção UTM divide o mundo em faixas de seis graus de longitude e representa cada faixa num plano com distorção pequena; Palmas, perto de 48° oeste, está na zona 22. A distorção, porém, não é nula: medida para este relatório, uma área em Palmas sai cerca de 0,13% maior na projeção do que na superfície do elipsoide, porque a cidade fica perto da borda leste da zona. Esse erro é pequeno diante da precisão das malhas e afeta todos os polígonos quase igualmente, de modo que as frações de área, que são razões entre áreas, praticamente não mudam. A projeção métrica, por conseguinte, permite medir em metros quadrados com um erro conhecido e irrelevante para as análises.

A transformação entre os sistemas

A transformação entre os sistemas é criada uma única vez, quando o módulo é carregado, e reutilizada em todas as chamadas, porque criá-la exige consultar a base de dados de sistemas de referência e seria um custo repetido a cada polígono. A transformação é criada com a opção que fixa a ordem dos eixos como longitude e depois latitude. Essa opção é essencial: a definição oficial do EPSG:4674 coloca a latitude primeiro, enquanto os dados do GeoJSON e do GeoPalmas trazem a longitude primeiro; sem a opção, a biblioteca trocaria os eixos, e Palmas apareceria num ponto do oceano. A transformação é aplicada aos vértices de cada polígono de uma só vez, como uma matriz, em vez de ponto a ponto, o que a torna muito mais rápida. A transformação entre os sistemas, assim, é eficiente e imune à confusão de eixos.

As funções do módulo

As funções do módulo são quatro. A função `para_utm` projeta uma geometria inteira para o sistema métrico; a função `area_m2` mede a área da geometria projetada, em metros quadrados; a função `wkt` converte uma geometria em texto no formato WKT, com sete casas decimais e sem zeros finais desnecessários; e a função `ler_wkt` faz o caminho inverso. As sete casas decimais correspondem a cerca de um centímetro em coordenadas geográficas, uma precisão maior que a das malhas, e o arredondamento torna a saída determinística, porque pequenas diferenças de cálculo entre máquinas desaparecem abaixo dessa casa. As funções do módulo, por conseguinte, oferecem às outras etapas uma forma única de medir e de guardar geometrias.

CONCLUSÃO — SÍNTESE

A síntese das operações geométricas mostra que o módulo existe para garantir uma única régua territorial em todo o pipeline. Um único sistema de coordenadas, uma única projeção, uma única transformação com a ordem dos eixos fixada e uma única precisão de texto fazem com que a limpeza, o cruzamento e os mapas meçam o mesmo território da mesma forma. A distorção da projeção é conhecida, pequena e uniforme, e, por isso, não altera nenhuma das conclusões que dependem de áreas.

36. Registro da execução no banco

INTRODUÇÃO — O TODO

O registro da execução no banco, no módulo `pipeline/carga/registro.py`, grava no MySQL o início e o fim de cada execução do pipeline, com as validações avaliadas. Esse registro existe porque o log fica num arquivo da máquina que executou o pipeline, enquanto o banco é o lugar que a equipe e o painel consultam; ter a história das execuções no banco permite saber, sem acessar a máquina, quando o pipeline rodou, com que código e com que resultado. A característica central do módulo é que o início e o fim são gravados em transações próprias, separadas da carga dos dados. O desenvolvimento a seguir analisa a garantia do esquema, o registro do início e o registro do fim.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A garantia do esquema

A garantia do esquema, feita pela função `garantir_esquema`, cria no banco apenas as tabelas que ainda faltam, e a decisão é tomada examinando duas tabelas que servem de marcadores. Se a tabela `fonte_snapshot`, criada pela Sprint 2, não existe, o banco está vazio, e a função cria o esquema inteiro das Sprints 2 e 3. Se essa tabela existe mas a tabela `execucao_pipeline`, criada pela Sprint 3, não existe, o banco já tem a Sprint 2, e a função executa apenas os arquivos da Sprint 3. Se as duas existem, nada é executado, e a função devolve "existente". Essa última condição é a mais importante, porque o arquivo `sprint3.sql` apaga e recria as tabelas da Sprint 3, e executá-lo num banco já carregado apagaria os dados. A garantia do esquema, assim, permite executar o pipeline num banco novo, num banco da Sprint 2 ou num banco já em uso, sem que nenhum desses casos perca dados.

O registro do início

O registro do início, feito pela função `registrar_inicio`, insere na tabela `execucao_pipeline` uma linha com o identificador da execução, o instante de início, a situação EM_ANDAMENTO, a versão do código e as opções da execução. A transação é confirmada imediatamente, antes de qualquer etapa, porque um registro que só fosse confirmado no fim desapareceria se o pipeline caísse no meio. Com o início confirmado, uma execução que termine de forma abrupta, por exemplo por falta de energia, permanece visível no banco como EM_ANDAMENTO, o que já é um sinal de que algo a interrompeu. O registro do início, por conseguinte, garante que toda execução deixe pelo menos um vestígio no banco.

O registro do fim

O registro do fim, feito pela função `registrar_fim`, atualiza a linha da execução com o instante final, a situação, que é SUCESSO ou FALHA, o resumo em formato JSON e a mensagem de erro, se houver. Na mesma transação, a função grava cada validação avaliada numa linha da tabela `validacao_execucao`, com a regra, a dimensão, a etapa, o tipo, o resultado e a evidência. A mensagem de erro e a evidência são limitadas a 1.000 caracteres, porque são as dimensões das colunas no banco, e uma mensagem maior faria a gravação do registro falhar justamente no momento em que ele é mais necessário. O registro do fim acontece depois da carga, numa transação própria, e por isso sobrevive mesmo quando a carga é desfeita. O registro do fim, assim, deixa no banco a conclusão de cada execução e a lista completa das regras que ela passou ou reprovou.

CONCLUSÃO — SÍNTESE

A síntese do registro da execução mostra que as suas três partes compartilham uma mesma preocupação: a trilha de cada execução precisa sobreviver às falhas que ela documenta. A garantia do esquema evita que a preparação do banco apague dados; o início confirmado de imediato deixa vestígio mesmo de uma queda; e o fim em transação própria preserva o diagnóstico mesmo quando a carga é desfeita. Por isso, sucesso e falha ficam igualmente documentados no banco.

37. Carga no MySQL

INTRODUÇÃO — O TODO

A carga no MySQL, nos módulos `pipeline/carga/mysql.py` e `pipeline/carga/dados.py`, leva os produtos da transformação ao banco de dados da Sprint 2. A carga tem duas propriedades fundamentais. A primeira é ser transacional: um grupo de dados é gravado por inteiro ou não é gravado, e o banco nunca fica pela metade. A segunda é ser idempotente: repetir a carga com a mesma entrada não altera o estado do banco, de modo que executar o pipeline duas vezes não duplica nada. O desenvolvimento a seguir analisa a divisão da carga em três partes e a estratégia de gravação escolhida para cada conjunto de dados.

As três partes da carga

A carga, na função `carregar_tudo`, é dividida em três partes, cada uma confirmada separadamente. A primeira grava os dados reais, do bruto aos indicadores, numa única transação; a segunda grava a simulação noutra transação; e a terceira carrega a extração NUCLEOGOV completa no módulo da Sprint 2. A separação entre dados reais e simulação existe porque são naturezas diferentes de dado, e uma falha na gravação dos 556 mil créditos simulados não deveria desfazer a gravação dos dados reais, que já foram conferidos. Se qualquer parte falhar, a transação em curso é desfeita, e o erro é repassado ao orquestrador; o registro da execução, gravado à parte, continua no banco. As três partes da carga, assim, isolam as falhas segundo a natureza dos dados.

A carga da extração no módulo da Sprint 2

A carga da extração NUCLEOGOV no módulo da Sprint 2 usa o próprio código daquela sprint, a função `carregar` do módulo `etl.carregar_receita`. Essa carga registra a extração como um snapshot identificado pelo SHA-256 do arquivo, de modo que carregar duas vezes o mesmo arquivo não cria um segundo snapshot. A Sprint 3 usa o código da Sprint 2, em vez de reescrevê-lo, pelo mesmo motivo que vale para a limpeza: o carregador já foi testado e auditado. Na execução de referência, essa parte completou a carga das 176.993 linhas que ficara pendente na Sprint 2, com 1.092 componentes e nenhuma divergência. A carga da extração, por conseguinte, fecha uma pendência da sprint anterior sem duplicar código.

A cópia do manifesto

A cópia do manifesto, feita pela função `carregar_brutos`, grava na tabela `arquivo_bruto` uma linha para cada arquivo registrado no manifesto. A inserção ignora as linhas que já existem, porque o manifesto só cresce, e cada execução precisa acrescentar apenas os arquivos novos. A chave da tabela é o nome do arquivo, e não o SHA-256 do conteúdo, por causa de um defeito encontrado na execução real: dois arquivos diferentes tinham o mesmo conteúdo, e, com o SHA-256 como chave, o segundo era ignorado, de modo que 61 arquivos coletados resultavam em 60 linhas. O SHA-256 continua na tabela, com um índice que acelera a busca por conteúdo. A cópia do manifesto, assim, leva ao banco a linhagem completa do bruto, um registro por arquivo.

A gravação por UPSERT

A gravação por UPSERT é a estratégia usada para o IPCA, a arrecadação histórica, a receita, os setores e a renda do Censo. Nessa estratégia, uma linha nova é inserida, e uma linha cuja chave já existe é atualizada com os valores novos. A chave é a chave natural de cada tabela, como a competência no IPCA ou o código do setor na malha. O UPSERT foi escolhido para essas tabelas porque elas representam o estado atual de cada dado: se o IBGE revisar a renda de um setor, a linha do setor deve passar a ter o valor revisado, e não ganhar uma segunda linha. A gravação por UPSERT, por conseguinte, mantém essas tabelas atualizadas sem duplicação.

O mês zero da arrecadação anual

O mês zero é a convenção usada para gravar no banco os valores anuais da arrecadação e da receita. A chave primária dessas tabelas inclui o mês, e o banco não aceita valor nulo numa chave primária; por isso, o valor anual é gravado com o mês igual a zero. Para que essa convenção não seja usada por engano, uma restrição do banco exige que o mês zero apareça somente com a granularidade anual, e que a granularidade mensal tenha mês entre 1 e 12. O mês zero, assim,

resolve uma limitação técnica sem abrir espaço para incoerência.

O valor real calculado pela view

O valor real da receita não é gravado em nenhuma tabela: a tabela `receita_tributaria` guarda o valor nominal e o fator do IPCA, e a view `v_receita_real` multiplica os dois e arredonda para o centavo. Essa escolha segue uma convenção da Sprint 2, segundo a qual nenhum valor derivado é persistido, porque um valor derivado gravado pode ficar desatualizado se um dos seus componentes mudar. Como o fator gravado é exatamente o mesmo usado no Python, a view chega ao mesmo centavo que os arquivos. O valor real calculado pela view, por conseguinte, nunca diverge dos dados que o originam.

As geometrias dos setores

As geometrias dos setores são gravadas no tipo espacial do MySQL, no sistema SIRGAS 2000, com a ordem dos eixos declarada explicitamente como longitude e depois latitude. A declaração é necessária pelo mesmo motivo que vale para a projeção no Python: a definição oficial do sistema coloca a latitude primeiro, e o MySQL segue essa definição, enquanto o texto WKT gerado pelo pipeline traz a longitude primeiro. Sem a declaração, o banco trocaria os eixos. Cada geometria é gravada com o seu SHA-256, que permite verificar rapidamente se a geometria mudou. As geometrias dos setores, assim, chegam ao banco na posição correta e com uma impressão digital.

A renda do Censo no banco

A renda do Censo é gravada por setor, e os valores sob sigilo ficam nulos, nunca zero, pela mesma regra que vale na limpeza. Uma restrição do banco reforça essa regra por outro lado: quando o setor não está sob sigilo, o número de responsáveis não pode ser nulo. Essa restrição existe porque um setor sem sigilo e sem responsáveis indicaria uma carga incompleta, e o banco deve recusá-lo em vez de aceitá-lo em silêncio. A renda do Censo no banco, por conseguinte, preserva a diferença entre ausência e zero também no armazenamento.

As regiões

As regiões de planejamento são gravadas pela função `_regioes`, que procura cada região pelo nome, pela origem real e pela vigência aberta, e a cria se não existir, com o início da vigência na data da coleta. A busca pela vigência aberta existe porque a tabela de regiões vem da Sprint 2 e foi desenhada para guardar versões ao longo do tempo; uma região só deve ser reaproveitada se for a versão atual. As regiões, assim, são integradas ao modelo de vigência que a Sprint 2 já previa.

As quadras e a vigência

As quadras são gravadas segundo o modelo de dimensão de tipo 2, uma técnica de armazém de dados em que cada mudança de um atributo importante cria uma nova versão do registro, com início e fim de vigência, em vez de sobrescrever a anterior. A função `_versao_quadra` procura a versão vigente da quadra e compara a sua geometria com a nova. Se as geometrias são iguais, a versão vigente é reaproveitada; se são diferentes, a versão anterior é encerrada na véspera da coleta e uma nova versão é aberta. Esse modelo foi escolhido para as quadras porque os limites de uma quadra podem mudar com um novo loteamento, e os créditos antigos precisam continuar ligados à quadra como ela era na época. As quadras e a vigência, por

conseguinte, preservam a história do território em vez de apagá-la.

A comparação geométrica

A comparação geométrica entre a quadra nova e a vigente é feita pelo próprio MySQL, com a função `ST_Equals`, que compara as formas, e não os textos que as descrevem. Essa comparação corrigiu um defeito da execução real: antes, a versão vigente era procurada por meio de uma tabela que o script `sprint3.sql` recriava, e, depois de reaplicá-lo, o pipeline não encontrava a versão vigente e duplicava a quadra. Comparar formas, e não textos, tem também uma vantagem própria: duas descrições textuais podem diferir na ordem dos pontos ou na última casa decimal e, ainda assim, descrever a mesma quadra. A comparação geométrica, assim, reconhece a mesma quadra independentemente da forma como foi escrita.

A segunda mudança no mesmo dia

A segunda mudança no mesmo dia é um caso especial do modelo de vigência. Se a versão vigente de uma quadra começou no próprio dia da coleta e a geometria mudou de novo, o pipeline não encerra essa versão, porque encerrá-la na véspera produziria uma vigência que termina antes de começar. Em vez disso, a geometria da versão de hoje é apenas atualizada. Esse caso acontece, por exemplo, quando o pipeline é executado duas vezes no mesmo dia depois de uma correção na fonte. A segunda mudança no mesmo dia, por conseguinte, é tratada como correção, e não como nova versão.

Os atributos das quadras

Os atributos das quadras vindos do GeoPalmas, como o nome, o uso, a situação, a referência, a área e o hash da geometria, ficam na tabela `quadra_geopalmas`. Essa tabela é uma extensão de um para um da tabela `quadra` da Sprint 2, que guarda apenas a geometria e a vigência. A separação existe porque a tabela `quadra` da Sprint 2 guarda o que é comum a qualquer quadra, e acrescentar a ela as colunas específicas do GeoPalmas a tornaria dependente dessa fonte. Os atributos das quadras, assim, são acrescentados sem alterar o desenho da tabela original.

O cruzamento e os indicadores

O cruzamento entre setores e quadras e os indicadores anuais são gravados por substituição completa: os registros antigos são apagados e os novos inseridos, dentro da mesma transação. Essa estratégia foi escolhida porque esses dados são inteiramente recalculados a cada execução, e não faz sentido comparar par a par o que mudou. Como o apagamento e a inserção estão na mesma transação, uma falha entre os dois desfaz o apagamento, e o banco nunca fica sem dados; um teste de integração provoca exatamente essa falha para confirmar o comportamento. No cruzamento, as frações são limitadas a 1, porque um arredondamento poderia produzir 1,0000001 e violar a restrição do banco. Nos indicadores, só são apagados os valores cuja fonte é "Pipeline Sprint 3", de modo que os indicadores da Sprint 2 ficam intactos. A substituição completa, por conseguinte, mantém esses dados sempre iguais ao último cálculo, sem risco de perda.

Os valores dos indicadores

Os valores dos indicadores são gravados na tabela `valor_indicador` da Sprint 2, quatro para cada ano e tributo: a arrecadação real, a variação real, a participação da dívida ativa e o desvio orçamentário. Um valor ausente, como a variação de um ano incompleto, não é gravado como zero, e sim marcado como INDISPONIVEL. Cada valor traz ainda a data de corte, que é o último dia do último mês com dado, e a cobertura, que é a fração dos doze meses com dado. Esses

dois campos existem porque um indicador de 2026, calculado com sete meses, não pode ser lido como se fosse um ano completo, e o banco precisa dizer isso a quem o consulta. Os valores dos indicadores, assim, carregam junto com o número as condições em que ele vale.

A simulação no banco

A simulação é gravada no banco uma única vez por assinatura. Antes de gravar, a função `carregar_simulacao` procura a assinatura na tabela `simulacao`; se ela existe, nada é gravado, e a função devolve o identificador existente. Essa regra existe porque a simulação é determinística: os mesmos parâmetros produzem exatamente os mesmos créditos, e gravá-los de novo apenas duplicaria 556 mil linhas. Uma simulação nova grava os parâmetros em JSON, o intercepto e a taxa, e depois os créditos, em lotes de 5.000, cada um com o setor e a quadra vigente. Os valores em centavos são divididos por 100 no próprio SQL, porque essa divisão de inteiros produz o decimal exato, enquanto a conversão em Python poderia passar por ponto flutuante. A simulação no banco, por conseguinte, é gravada com exatidão e sem duplicação.

CONCLUSÃO — SÍNTESE

A síntese da carga mostra que cada conjunto de dados recebeu a estratégia de gravação que corresponde à sua natureza. O bruto, que só cresce, é inserido ignorando o existente; o estado atual das fontes é atualizado por UPSERT; as quadras, que mudam com o tempo, guardam a sua história por vigência; os dados recalculados a cada execução são substituídos por inteiro; e a simulação, que é determinística, é gravada uma vez por assinatura. Por causa dessas estratégias, a carga deixa o banco igual aos arquivos e pode ser repetida quantas vezes for preciso sem duplicar nem perder dados.

38. Esquema SQL da Sprint 3

INTRODUÇÃO — O TODO

O esquema SQL da Sprint 3, no arquivo `sql/sprint3.sql`, define as tabelas, as restrições e as views que a Sprint 3 acrescenta ao banco da Sprint 2. O esquema segue as convenções da Sprint 2: o dinheiro é guardado no tipo `DECIMAL`, que é exato, e os valores derivados ficam em views, e não em tabelas. A característica mais importante do esquema é que ele repete, no próprio banco, as regras que o Python já aplica, por meio de restrições. O desenvolvimento a seguir analisa a forma de reexecução do arquivo e, depois, as tabelas de linhagem, de receita, de território e de simulação, e as views.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A reexecução do arquivo

O arquivo pode ser executado mais de uma vez, porque começa apagando e recriando as tabelas e as views da Sprint 3. Durante o apagamento, a verificação de chaves estrangeiras é desligada temporariamente, porque as tabelas se referem umas às outras, e, com a verificação ligada, a ordem do apagamento teria de seguir exatamente as dependências. Essa forma de reexecução tem uma consequência que precisa ser conhecida: reexecutar o arquivo apaga os dados das tabelas da Sprint 3. Por isso, o pipeline só o executa quando as tabelas não existem, como garante a função `garantir_esquema`. A reexecução do arquivo, assim, é útil para recriar o esquema do zero, mas não é uma operação inofensiva.

A alteração da tabela quadra

A alteração da tabela `quadra` é a única mudança que o arquivo faz numa tabela da Sprint 2. A tabela não é apagada; o arquivo apenas alarga a coluna `codigo` para 64 caracteres, porque o identificador das quadras no GeoPalmas tem 32 caracteres, mais do que a coluna original aceitava. O alargamento é compatível com os dados existentes, porque uma coluna mais larga aceita tudo o que a mais estreita aceitava. A alteração da tabela quadra, por conseguinte, adapta a Sprint 2 à nova fonte sem perder nenhum dado.

As tabelas de linhagem

As tabelas de linhagem registram de onde vieram os dados e como cada execução terminou. A tabela `execucao_pipeline` guarda cada execução, com a situação EM_ANDAMENTO, SUCESSO ou FALHA, e uma restrição impede que o fim seja anterior ao início, o que denunciaria um relógio errado ou um registro corrompido. A tabela `validacao_execucao` guarda as validações de cada execução, com a chave formada pela execução e pela regra, e a dimensão só aceita os sete valores definidos. A tabela `arquivo_bruto` é a cópia do manifesto no banco, com o nome do arquivo como chave e um índice no SHA-256. As tabelas de linhagem, assim, permitem rastrear qualquer dado do banco até a execução e o arquivo que o produziram.

As tabelas da receita

As tabelas da receita guardam o IPCA, a arrecadação histórica e a série unificada, cada uma com restrições próprias. A tabela `ipca_mensal` exige que a competência tenha o formato ano-mês e que o número-índice seja positivo. A tabela `arrecadacao_historica` amarra o mês à granularidade, limita os componentes aos cinco valores válidos e exige que tributo e componente sejam ambos nulos, para as contas fora do escopo, ou ambos preenchidos, porque uma linha com tributo e sem componente seria uma classificação pela metade. A tabela `receita_tributaria` só aceita as fontes API_PREFEITURA e NUCLEOGOV, e a sua referência do IPCA precisa apontar para um mês existente. As tabelas da receita, por conseguinte, recusam no banco as mesmas incoerências que a limpeza recusa no Python.

As tabelas do território

As tabelas do território guardam os setores, a renda do Censo, os atributos das quadras e o cruzamento. A tabela `setor_censitario` exige, por restrição, que o código do setor comece pelo código do município, porque os sete primeiros dígitos do código de um setor são justamente o código do seu município; e a origem padrão é REAL. A tabela `censo2022_renda_setor` mantém o sigilo como nulo, nunca zero. A tabela `quadra_geopalmas` estende a quadra, e a tabela `setor_quadra` guarda o cruzamento com as frações limitadas entre 0 e 1, com uma pequena folga para arredondamento. As tabelas do território, assim, impõem no banco a coerência entre códigos, sigilo e frações.

As tabelas da simulação

As tabelas da simulação ficam separadas das tabelas operacionais da Sprint 2, para que o dado sintético nunca se misture ao real. A tabela `simulacao` guarda cada versão, com a assinatura única e os parâmetros em JSON. A tabela `credito_simulado` guarda os créditos, e uma restrição exige que a origem seja SINTETICO, de modo que nenhum crédito real possa ser gravado ali por engano, nem um crédito simulado possa ser apresentado como real. Outras restrições protegem a coerência de cada crédito: o principal precisa ser positivo; o valor pago e o saldo não

podem ser negativos; a ausência de data de pagamento equivale a valor pago igual a zero; e a inscrição em dívida ativa, quando existe, vem depois do vencimento. As tabelas da simulação, por conseguinte, garantem pela estrutura do banco que o sintético é sempre reconhecível e internamente coerente.

A view do valor real

A view `v_receita_real` calcula o valor real de cada linha da receita, multiplicando o valor nominal pelo fator do IPCA e arredondando para o centavo. O resultado é igual ao dos arquivos, porque o fator gravado é o mesmo do cálculo em Python, e o arredondamento segue o mesmo critério. A view existe para que o painel e as consultas obtenham o valor real sem que ele precise ser gravado. A view do valor real, assim, é a forma de oferecer um valor derivado sem o risco de ele ficar desatualizado.

As views dos atributos

As views dos atributos calculam em SQL os mesmos atributos da base analítica do Python, e foram divididas em duas por uma razão de desempenho. A view `v_credito_atributos` calcula os atributos que dependem só da própria linha: o tempo de inadimplência, a faixa de valor e o mês do vencimento. A view `v_base_analitica_credito` acrescenta o histórico de pagamento, que exige uma função de janela, isto é, um cálculo que olha para as outras linhas do mesmo imóvel; a janela soma apenas os exercícios anteriores, nunca o próprio. A divisão existe porque o MySQL consegue fundir uma view sem função de janela na consulta que a usa, calculando só o necessário, mas não consegue fundir uma view com função de janela, que obriga a calcular o histórico de todos os 556 mil créditos. Quando o painel agregava por faixa de valor usando a view com histórico, a consulta levava de 6 a 9 s; com a divisão, passou a levar 0,4 s. As views dos atributos, por conseguinte, são conferidas contra o Python e otimizadas para o uso do painel.

As views do painel

As views do painel, `v_faixa_valor_simulacao` e `v_inadimplencia_setor`, entregam ao Power BI os agregados já prontos. A primeira agrega os créditos por faixa de valor, com o número de créditos, a taxa de inadimplência e o estoque aberto. A segunda agrega por setor e suprime, com valor nulo, a taxa e o estoque dos setores com menos de dez imóveis, aplicando no banco a mesma regra de supressão que o Python aplica nos arquivos. As duas views trazem a coluna `origem_dado` com o valor SINTETICO, para que o painel nunca apresente esses números sem a marca. As views do painel, assim, levam até a visualização as regras de proteção e de transparência do pipeline.

CONCLUSÃO — SÍNTESE

A síntese do esquema SQL revela que ele não é apenas um lugar para guardar dados, mas uma segunda barreira de qualidade. As restrições repetem no banco as regras que o Python aplica: o mês coerente com a granularidade, o sigilo como nulo, o código do setor coerente com o município, a origem sintética obrigatória. As views calculam os valores derivados sem persisti-los e reproduzem a base analítica de forma conferível. Por isso, uma carga errada é barrada por restrição, e não apenas por teste, e o banco permanece coerente mesmo diante de um erro de programação futuro.

39. EDA: leitura, estatísticas e estilo

INTRODUÇÃO — O TODO

A análise exploratória, chamada EDA, começa por três módulos de base: `dados.py`**, que lê as camadas,** `estatisticas.py`**, que calcula os números, e** `formato.py` **e** `estilo.py`**, que definem como números e gráficos são apresentados.** A EDA foi dividida dessa forma porque separa o cálculo da apresentação: os números são calculados por funções puras, que podem ser testadas, e só depois são formatados em frases e desenhados em gráficos. A EDA só lê as camadas; ela nunca coleta nem carrega dados, e um contrato de dependência impede que importe a rede, o banco ou os filtros. O desenvolvimento a seguir analisa a leitura, as estatísticas descritivas, as medidas da receita, as medidas do território, o I de Moran, a formatação e o estilo dos gráficos.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A leitura das camadas

A leitura das camadas, feita pela função `carregar`**, abre as tabelas de** `data/pronto` **e de** `data/limpo` **de que a EDA precisa e o relatório de qualidade da execução.** Os códigos de setor e de quadra são lidos como texto, e não como números, porque são identificadores, e não quantidades: lidos como números, poderiam perder zeros à esquerda e deixariam de casar com os mesmos códigos lidos como texto em outra tabela. Da base analítica, que tem mais de meio milhão de linhas, são lidas apenas as nove colunas que o relatório usa, porque carregar todas as colunas ocuparia memória sem nenhuma utilidade. A leitura das camadas, assim, entrega ao relatório exatamente o que ele precisa, com os tipos corretos.

As estatísticas descritivas

As estatísticas descritivas, calculadas pela função `resumo`**, descrevem cada variável por nove números: a quantidade de valores, a média, o desvio-padrão, o mínimo, os três quartis, o máximo e a assimetria.** O desvio-padrão é o amostral, e os valores ausentes são descartados antes do cálculo, porque um ausente não é um valor e não pode entrar numa média. Os quartis e a assimetria acompanham a média porque, em variáveis assimétricas como a renda, a média é puxada pelos valores extremos e deixa de representar o caso típico, papel que a mediana cumpre melhor. A assimetria, calculada pela função `assimetria`, usa o coeficiente de Fisher–Pearson ajustado, o mesmo do pandas e do Excel, para que o número do relatório possa ser conferido nessas ferramentas; com menos de três valores, o coeficiente não tem sentido, e a função devolve zero. As estatísticas descritivas, por conseguinte, resumem cada variável sem esconder a sua forma.

A série real e as lacunas

A série real, montada pela função `serie_real`**, organiza a arrecadação real anual de cada tributo em milhões de reais, usando apenas os anos com os doze meses.** Depois de montada, a série é reindexada por todos os anos do intervalo, de modo que um ano sem dado aparece como ausente, e não desaparece. Essa reindexação existe porque, num gráfico de linhas, um ano que simplesmente faltasse na tabela faria a linha ligar 2017 diretamente a 2019, e o leitor não perceberia que 2018 está fora da série. A série real, assim, mantém a lacuna visível em vez de escondê-la.

O crescimento anual médio

O crescimento anual médio, calculado pela função `crescimento_anual_medio`, é a taxa composta entre dois anos: a raiz do quociente entre o valor final e o inicial, com índice igual ao número de anos, menos um. A taxa composta é usada, em vez da média simples das variações anuais, porque é a única taxa constante que, aplicada ano após ano, leva exatamente do valor inicial ao final; a média simples das variações pode sugerir crescimento mesmo quando o valor final é igual ao inicial. O resultado é expresso em porcentagem ao ano. O crescimento anual médio, por conseguinte, resume uma trajetória de vários anos num único número que a reproduz.

A composição do IPTU

A composição do IPTU, calculada pela função `composicao_iptu`, separa a arrecadação real do imposto em três grupos: o exercício, que reúne o principal e as multas e juros do próprio ano; a dívida ativa; e as multas e juros da dívida ativa. Os ajustes ficam fora desses grupos e são devolvidos à parte, porque são valores negativos, e, num gráfico de colunas empilhadas, um valor negativo seria desenhado abaixo de zero ou subtrairia visualmente de outro grupo, distorcendo a leitura da soma. A partir da composição, a função `participacao_divida_ativa` calcula a parcela do IPTU que veio da dívida ativa, somando os dois grupos da dívida e dividindo pelo total do ano. A composição do IPTU, assim, mostra de onde vem a arrecadação do imposto sem que os ajustes confundam a figura.

A sazonalidade e a correlação

A sazonalidade e a correlação são calculadas por funções curtas que alimentam as frases de leitura. A função `pico_sazonal` encontra o mês de maior índice sazonal de um tributo, e a função `amplitude_sazonal` encontra o menor e o maior índice, que descrevem quanto a arrecadação varia ao longo do ano. A função `correlacao_postos` calcula o coeficiente de Spearman, que é a correlação de Pearson calculada sobre as posições dos valores, e não sobre os valores. O coeficiente de Spearman foi escolhido porque a renda é muito assimétrica, e alguns setores muito ricos teriam peso desproporcional numa correlação de Pearson; trabalhar com posições elimina esse peso e mede apenas se as duas variáveis crescem ou decrescem juntas. As funções de sazonalidade e de correlação, por conseguinte, produzem os números exatos que as frases citam.

As classes dos mapas

As classes dos mapas são definidas pela função `limites_quantis`, que calcula os cortes de cinco classes com o mesmo número de setores cada uma. Esse método, chamado classificação por quantis, foi escolhido porque Brewer e Pickle (2002) o consideraram o mais legível para comparar mapas diferentes. Cortes repetidos são removidos, porque, quando muitos setores têm o mesmo valor, dois cortes podem coincidir e criar uma classe vazia. A função `classe` devolve a classe de cada valor e não atribui classe a um valor ausente, que o mapa pinta de cinza. As classes dos mapas, assim, distribuem os setores de forma equilibrada e deixam a ausência visível.

A média por região

A média por região, calculada pela função `media_por_regiao`, resume uma variável dos setores pelas regiões de planejamento do GeoPalmas. O setor chega à região pela sua quadra principal, porque o Censo não informa a região, e a quadra é o elo entre os dois recortes. A média é ponderada, por responsáveis no caso da renda e por imóveis no caso da inadimplência simulada, porque uma média simples daria o mesmo peso a um setor com dez domicílios e a outro com mil. Os

setores sem quadra principal ficam fora da média regional, porque não há como atribuí-los a uma região. A média por região, por conseguinte, traduz os resultados por setor para a escala em que a Prefeitura planeja.

O I de Moran

O I de Moran, calculado pela função `moran`, mede se setores vizinhos se parecem mais do que se esperaria por acaso (Moran, 1950). Para cada setor, o índice compara o desvio do seu valor em relação à média com o desvio médio dos seus vizinhos; os pesos são padronizados por linha, de modo que cada setor dá o mesmo peso total aos seus vizinhos, tenha ele dois ou dez. Um índice perto de zero indicaria um mosaico aleatório; um índice positivo indica que vizinhos têm valores parecidos. Setores sem nenhum vizinho ficam fora do cálculo, porque não há com quem compará-los. O I de Moran, assim, transforma a impressão visual de "manchas" no mapa numa medida.

O valor-p por permutação

O valor-p do I de Moran é obtido por permutação, e não por uma fórmula que suponha uma distribuição. O procedimento embaralha os valores entre os setores 999 vezes e recalcula o índice a cada vez; o valor-p é a proporção de índices embaralhados iguais ou maiores que o observado, com uma correção que soma um ao numerador e ao denominador. A permutação dispensa supor que os valores sigam uma distribuição normal, suposição que a renda claramente não satisfaz. Com 999 permutações, o menor valor-p possível é 0,001, e por isso o relatório escreve "p ≤ 0,001", e não "p = 0,001", quando nenhum embaralhamento supera o observado. A semente das permutações é fixa, para que o valor-p seja o mesmo a cada execução. O valor-p por permutação, por conseguinte, avalia a significância do padrão sem impor aos dados uma forma que eles não têm.

A formatação dos números e das figuras

A formatação dos números, no módulo `formato.py` e na função `br` do estilo, escreve todos os valores no padrão brasileiro: o milhar com ponto, o decimal com vírgula e o sinal de menos tipográfico, que tem a largura do sinal de mais e não se confunde com o hífen. A função `tabela_md` escreve tabelas em Markdown, alinhando os números à direita. O objeto `Figura` guarda o arquivo, o título, a frase de leitura, a fonte, a marca de dado simulado e uma tabela com os mesmos dados da figura; essa tabela existe como alternativa acessível ao gráfico, para quem não pode vê-lo, e como forma de conferir os números exatos. O objeto `Secao` reúne o título, os parágrafos, as figuras e uma tabela opcional. A formatação, assim, faz com que o relatório fale a língua do leitor e ofereça cada gráfico também em forma de tabela.

O estilo e a paleta

O estilo dos gráficos, definido no módulo `estilo.py`, segue uma paleta de cores validada para contraste e para a separação entre cores sob daltonismo simulado. Há uma cor por tributo, e a cor segue a entidade em todos os gráficos: o IPTU é sempre azul, o ISSQN sempre laranja e o ITBI sempre verde, porque, se as cores mudassem de um gráfico para outro, o leitor teria de reaprender a legenda a cada figura. Os mapas usam uma rampa de um só matiz, do azul claro ao escuro, porque as classes são ordenadas, e uma sequência de claridade comunica ordem melhor que cores diferentes; o setor sem valor fica em cinza, fora da rampa. O texto nunca usa a cor da série: valores e rótulos ficam em tons neutros, porque um texto colorido pode ficar ilegível sobre o fundo. O estilo e a paleta, por conseguinte, tornam os gráficos consistentes e legíveis para leitores com

diferentes capacidades visuais.

A interpretação de fórmulas desligada

A interpretação de fórmulas do matplotlib foi desligada por causa de um defeito real. O matplotlib interpreta o texto entre dois cifrões como uma fórmula matemática, e um título com dois valores em reais, como "R$ 2.155 a R$ 7.613", tinha o trecho entre os dois "R$" convertido em fórmula, o que corrompia o texto. Como os gráficos do projeto nunca precisam de fórmulas, a interpretação foi desligada para todos eles. A configuração também usa o modo de desenho sem janela, porque o relatório é gerado por script, inclusive na integração contínua, em que não há tela. A interpretação desligada, assim, protege os títulos em reais de uma conversão indesejada.

A marca dos dados simulados

A marca dos dados simulados é aplicada pela função `marcar_simulado`, que escreve a palavra SIMULADO, grande, inclinada e quase transparente, ao fundo de todo gráfico com dados da simulação. A marca fica atrás dos dados, para não esconder nenhum ponto, mas é visível o bastante para que ninguém copie o gráfico para uma apresentação sem perceber que ele não descreve a realidade. Essa marca existe porque um gráfico circula separado do texto que o explica, e a advertência precisa viajar junto com a imagem. A marca dos dados simulados, por conseguinte, é a forma de a circularidade da simulação ser declarada no próprio gráfico.

A gravação das figuras e as colunas

A gravação das figuras é feita pela função `salvar`, que grava cada gráfico em PNG a 150 dpi, a resolução pedida pelo Módulo 5, e sem o metadado que registra a versão do programa. Esse metadado é removido porque, sem ele, a mesma entrada gera os mesmos bytes no mesmo ambiente, o que permite verificar a reprodutibilidade também das figuras. Depois de salva, a figura é fechada, para liberar a memória. As colunas dos gráficos seguem regras próprias: a ponta de dado é arredondada com um raio de 4 pixels e a base é reta, de modo que o olho perceba onde está o valor; a largura nunca passa de 24 pixels, para que as colunas não pareçam blocos; e uma borda da cor do fundo separa colunas vizinhas e segmentos empilhados. A gravação e as colunas, assim, produzem figuras reprodutíveis e de leitura limpa.

CONCLUSÃO — SÍNTESE

A síntese da base da EDA revela que todas as suas partes servem para que cada número das frases de leitura possa ser conferido e cada gráfico possa ser lido sem engano. A leitura entrega os tipos certos, as estatísticas usam medidas adequadas à forma dos dados, o I de Moran mede o que o olho apenas sugere, a formatação fala a língua do leitor, e o estilo garante consistência, acessibilidade e a marca do simulado. Como o cálculo está separado da apresentação, qualquer número do relatório pode ser verificado por um teste, sem passar pelo gráfico.

40. EDA: gráficos e mapas

INTRODUÇÃO — O TODO

Os gráficos e os mapas da EDA, nos módulos `graficos.py` e `mapas.py`, transformam os números calculados em figuras. As escolhas desses módulos seguem a literatura sobre gráficos honestos: magnitudes são mostradas por posição sobre um eixo comum que começa em zero, há um único eixo vertical por gráfico, lacunas aparecem como lacunas, distribuições são mostradas inteiras e dados simulados são marcados. Essas escolhas existem porque um gráfico pode enganar sem conter nenhum número falso, apenas pela forma como os números são desenhados. O desenvolvimento a seguir analisa cada tipo de gráfico e, depois, a construção dos mapas.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O gráfico de evolução

O gráfico de evolução, desenhado pela função `evolucao`, mostra a arrecadação real anual dos três tributos ao longo dos anos. As três linhas dividem um único eixo vertical, que começa em zero, porque a posição sobre uma escala comum é a codificação que as pessoas leem com mais precisão (Cleveland; McGill, 1984), e um eixo que não começa em zero exageraria visualmente as variações. O último valor de cada linha recebe um rótulo direto, com o nome do tributo e o valor, para que o leitor não precise consultar a legenda. O ano sem dado aparece como uma faixa cinza com o rótulo "fora da série", e cada trecho da série recebe, no alto, o nome da fonte que o cobre, de modo que o leitor vê onde termina a API e começa o NUCLEOGOV. O gráfico de evolução, assim, mostra a tendência sem esconder a lacuna nem a quebra de fonte.

O gráfico de composição

O gráfico de composição, desenhado pela função `composicao`, mostra o IPTU de cada ano em colunas empilhadas, com os três grupos de origem do crédito. Só a ponta da pilha é arredondada, porque o arredondamento marca o fim do valor total, e arredondar cada segmento sugeriria que cada um termina ali. O ano sem dado fica sem coluna e com a mesma faixa cinza do gráfico de evolução. As colunas empilhadas foram escolhidas porque mostram, ao mesmo tempo, o total do ano, pela altura da coluna, e a participação de cada grupo, pela altura de cada segmento. O gráfico de composição, por conseguinte, permite ver de uma só vez quanto o IPTU arrecadou e quanto disso veio da dívida ativa.

O gráfico de sazonalidade

O gráfico de sazonalidade, desenhado pela função `sazonalidade`, mostra o índice sazonal de cada mês para os três tributos. Uma linha horizontal marca o valor 1, com o rótulo "média do ano = 1", porque o índice só tem sentido em relação a essa referência: acima dela, o mês arrecada mais que a média; abaixo, menos. O pico do IPTU recebe um rótulo com o mês e o múltiplo da média, porque é o fato mais marcante do gráfico e aquele que orienta a simulação. O gráfico de sazonalidade, assim, torna visível o calendário da arrecadação de cada tributo.

O histograma

O histograma, desenhado pela função `distribuicao`, mostra a distribuição completa de uma variável, com classes de largura fixa e a mediana marcada por uma linha vertical. O histograma foi preferido a um gráfico de barras de médias porque uma barra mostra apenas um número e esconde a forma da distribuição, inclusive a assimetria, que é a característica mais

importante da renda (Weissgerber et al., 2015). As classes têm largura fixa, de R$ 1.000 no caso da renda, para que a altura de cada coluna seja comparável com a das outras. A mediana é marcada, e não a média, porque, numa distribuição assimétrica, ela representa melhor o caso típico. O histograma, por conseguinte, mostra a distribuição inteira, e não um resumo que a deformaria.

O gráfico de relação

O gráfico de relação, desenhado pela função `relacao`, mostra cada setor como um ponto, com a renda no eixo horizontal e a inadimplência simulada no vertical. A renda usa escala logarítmica, porque, em escala comum, a maioria dos setores ficaria espremida à esquerda e uns poucos setores ricos ocupariam quase todo o eixo. Uma linha mostra a média da inadimplência em cada decil de renda, isto é, em cada grupo de 10% dos setores ordenados pela renda, porque a nuvem de pontos sozinha é difícil de resumir, e a linha mostra a tendência sem impor a ela uma forma matemática. O gráfico de relação sempre leva a marca SIMULADO, porque a relação que ele mostra foi imposta pelo parâmetro da simulação e não é evidência sobre Palmas. O gráfico de relação, assim, demonstra o funcionamento da simulação e declara essa natureza na própria imagem.

O recorte dos mapas

O recorte dos mapas é definido pela função `geometrias_urbanas`, que seleciona os setores urbanos e, nas seções do relatório, apenas os do distrito-sede de Palmas. Os setores urbanos de outros distritos ficam fora do recorte porque estão distantes da sede, e incluí-los obrigaria a reduzir a escala do mapa a ponto de tornar os setores da sede ilegíveis. O subtítulo de cada mapa declara o recorte e diz quantos setores ficaram fora, porque um recorte não declarado faria o leitor supor que o mapa mostra a cidade inteira. O recorte dos mapas, por conseguinte, privilegia a legibilidade sem esconder o que foi deixado de fora.

A vizinhança entre os setores

A vizinhança entre os setores, calculada pela função `vizinhos`, encontra os pares de setores contíguos, que o I de Moran usa. Dois setores são vizinhos se se tocam por uma borda ou por um vértice, critério chamado de contiguidade "rainha", por analogia com os movimentos da rainha no xadrez. A busca usa uma árvore espacial e uma tolerância de 1 m, porque as malhas digitalizadas têm pequenas frestas entre polígonos que, na realidade, são vizinhos, e sem a tolerância esses setores seriam considerados separados. A vizinhança entre os setores, assim, reflete a contiguidade real, apesar das imperfeições do desenho.

A orientação dos anéis

A orientação dos anéis, feita pela função `caminho`, converte cada polígono num caminho de desenho do matplotlib e corrige um defeito real dos mapas. Um polígono pode ter furos, como um setor que contorna um parque ou um lago, e o matplotlib só deixa um furo vazio se o anel externo e o anel do furo forem percorridos em sentidos opostos. A função, por isso, orienta o anel externo no sentido anti-horário e os furos no sentido horário. Antes dessa correção, os furos apareciam preenchidos, como se o setor cobrisse também a área que ele contorna, e um teste que rasteriza o desenho e verifica a cor do furo protege a correção. A orientação dos anéis, por conseguinte, faz o mapa mostrar exatamente a área de cada setor.

O mapa coroplético

O mapa coroplético, desenhado pela função `coropletico`, pinta cada setor com a cor da sua classe. A legenda mostra os cortes de cada classe, para que o leitor saiba exatamente que valores cada cor representa, e inclui o cinza com uma descrição do que ele significa, como "setor sob sigilo do IBGE", porque um setor cinza sem explicação poderia ser lido como valor baixo. O mapa traz uma barra de escala de 5 km e a indicação do norte, e é desenhado na projeção UTM, a mesma das áreas, de modo que a forma e a escala ficam preservadas na área da cidade e a barra mede metros reais. Quando o mapa mostra dado simulado, ele recebe a marca SIMULADO. O mapa coroplético, assim, combina as convenções cartográficas com as regras de honestidade do resto do relatório.

CONCLUSÃO — SÍNTESE

A síntese dos gráficos e dos mapas revela que cada escolha de desenho previne um engano específico. O eixo que começa em zero impede o exagero; a faixa cinza impede que a lacuna desapareça; o rótulo da fonte impede que a quebra de série passe por mudança real; o histograma impede que a assimetria se esconda atrás de uma média; a escala logarítmica impede que poucos setores dominem o gráfico; o cinza nomeado impede que ausência seja lida como zero; e a marca SIMULADO impede que a premissa seja lida como achado. Por isso, as figuras mostram os dados sem esconder lacunas nem fabricar padrões.

41. EDA: seções e relatório

INTRODUÇÃO — O TODO

As seções e a montagem do relatório, nos módulos `secoes_receita.py`, `secoes_territorio.py` e `relatorio.py`, escrevem o texto da EDA e o organizam nos arquivos `EDA.md` e `GALERIA.md`. A característica central dessas seções é que nenhuma frase traz um número digitado à mão: cada seção calcula os seus números, desenha as suas figuras e redige as suas frases com esses números. Essa característica existe porque um relatório digitado à mão envelhece no momento em que os dados mudam, e um número esquecido numa frase continua afirmando algo que já não é verdade. O desenvolvimento a seguir analisa cada seção com figura, as seções fixas e a montagem do relatório.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A seção de evolução

A seção de evolução compara a arrecadação dos tributos ao longo dos anos, mas faz a comparação apenas dentro do trecho mais recente de uma mesma fonte, que é o NUCLEOGOV, de 2019 a 2025. Essa restrição existe porque comparar anos de fontes diferentes misturaria a mudança real da arrecadação com a diferença entre as fontes. A seção calcula o crescimento anual médio do ISSQN e do IPTU e a participação do ISSQN nos três tributos, e o título da figura afirma o achado, em vez de apenas descrever o eixo; na execução de referência, o ISSQN cresceu 9,5% ao ano acima da inflação, e o IPTU recuou 2,9% ao ano. A seção de evolução, assim, afirma uma tendência apenas onde a comparação é segura.

O aviso de cautela sobre o IPTU

O aviso de cautela sobre o IPTU, produzido pela função `_cautela_iptu`, é acrescentado ao texto quando o maior IPTU da API supera o maior IPTU do NUCLEOGOV em mais de 10%. Esse aviso existe porque um valor da fonte antiga muito acima de todos os da fonte atual pode indicar um problema na fonte antiga, e não uma arrecadação realmente maior. Na execução de referência, o IPTU de 2017 na API supera em 18% o maior ano do NUCLEOGOV, e o aviso aparece, recomendando conferir esse valor com o balanço do município. O aviso é calculado, e não escrito à mão, para que desapareça sozinho se uma correção da fonte eliminar a diferença. O aviso de cautela, por conseguinte, é uma ressalva que acompanha os dados, e não uma opinião fixa.

A seção de composição

A seção de composição mede o peso da dívida ativa no IPTU, informando o ano de maior parcela, o último ano e o maior ajuste anual. Na execução de referência, a dívida ativa respondeu por 47,7% do IPTU arrecadado em 2019 e por 15,7% em 2025. A seção apresenta duas leituras para essa queda, e declara que a base pública não permite escolher entre elas: a queda pode indicar melhor adimplência no exercício ou menor esforço de cobrança dos créditos antigos. Essa honestidade é necessária porque distinguir as duas leituras exige o histórico por contribuinte, exatamente o dado que a Secretaria da Fazenda ainda não forneceu. A seção de composição, assim, apresenta o achado e o limite da sua interpretação.

A seção de sazonalidade

A seção de sazonalidade informa o pico do IPTU, em março, com um índice de 3,53 vezes a média mensal, e a faixa em que ficam o ISSQN e o ITBI, entre 0,82 e 1,26 da média. O texto liga o pico de março ao vencimento escolhido para a simulação e observa que, por isso, a única informação sazonal da base analítica vem dos dados reais, e não de uma premissa. A seção de sazonalidade, por conseguinte, mostra como um achado real orientou uma decisão da simulação.

A seção de distribuição

A seção de distribuição descreve a renda dos setores urbanos pela mediana, pelo terceiro quartil e pelo máximo. Na execução de referência, metade dos 637 setores urbanos com renda divulgada tinha renda média do responsável abaixo de R$ 3.129. O texto explica por que a média fica acima da mediana: poucos setores muito ricos puxam a média para cima, o padrão típico de grandezas que se formam por efeitos multiplicativos, como a renda (Limpert et al., 2001). O texto também lembra que os setores sob sigilo ficam sem renda e que nada foi imputado. A seção de distribuição, assim, descreve o setor típico com a medida adequada e explica a escolha.

A seção de relação

A seção de relação calcula o coeficiente de Spearman entre a renda e a inadimplência simulada e declara, no próprio texto e no título, que essa correlação é circular: ela mede o parâmetro da simulação, e não o comportamento dos contribuintes. A seção mantém a figura no relatório por dois motivos declarados. O primeiro é demonstrar que o pipeline leva o dado do setor até o crédito e de volta ao mapa. O segundo é definir o teste que os dados reais terão de passar, porque a mesma figura, sem a marca SIMULADO, dirá se o gradiente existe em Palmas; esse é o uso que Morris, White e Crowther (2019) recomendam para dados simulados, avaliar métodos com uma verdade conhecida. A seção de relação, por conseguinte, usa a simulação

como ferramenta, e não como evidência.

A seção de mapas

A seção de mapas desenha dois mapas, o da renda real e o da inadimplência simulada, e pede que eles sejam lidos juntos. O mapa de renda mostra que a renda média vai de R$ 2.155 na região Sul a R$ 7.613 na região Sudoeste e que setores vizinhos têm rendas parecidas, com I de Moran de 0,76 no logaritmo da renda. O mapa de inadimplência repete o desenho do mapa de renda com o sinal invertido, e o texto explica que essa semelhança confirma o funcionamento da simulação, e não uma descoberta. O texto alerta ainda para dois riscos de interpretação: padrões por setor não valem para cada imóvel (Robinson, 1950), e mudariam com outro recorte territorial (Fotheringham; Wong, 1991). A seção de mapas, assim, separa o que o território mostra do que a simulação reproduz.

A seção de indicadores

A seção de indicadores apresenta, numa tabela, os valores reais por ano e tributo, apenas para os anos completos. A tabela traz a arrecadação real, a variação real, a participação da dívida ativa, o desvio orçamentário e uma coluna de quebra de fonte, que marca a troca da API para o NUCLEOGOV para que ela não seja lida como mudança real da arrecadação. Os valores ausentes aparecem como travessão, e não como zero, pela mesma regra que vale em todo o pipeline. A seção de indicadores, por conseguinte, oferece os números de gestão numa forma que pode ser consultada diretamente.

As seções fixas

As seções fixas são as que não têm figura: a introdução, as bases e a qualidade dos dados, as estatísticas descritivas, os atributos da base analítica e as limitações. Mesmo nelas, os números vêm dos arquivos, e não de texto digitado. A seção de qualidade, por exemplo, lê o relatório de qualidade da execução e informa quantas regras foram avaliadas, quantas impeditivas passaram, quantas linhas da API ficaram fora da série, quantos registros foram para a quarentena e quantos setores estão sob sigilo. A seção de limitações declara o alcance das conclusões e os próximos passos, como a substituição da simulação pelos dados reais. As seções fixas, assim, completam o relatório com o contexto que dá sentido às figuras.

A montagem do relatório

A montagem do relatório, feita pela função `gerar`, reúne as doze seções na ordem definida e escreve dois arquivos. No `EDA.md`, cada figura recebe um número, o título, a imagem, a frase de leitura, a fonte e a tabela com os mesmos dados, guardada num bloco recolhível para não interromper a leitura. O cabeçalho identifica a execução e a versão do código que produziram o relatório e avisa que o arquivo não deve ser editado à mão, porque os números mudam a cada execução. O `GALERIA.md` lista as figuras numa tabela e diz, para cada uma, se o dado é real ou simulado. As referências citadas saem do catálogo do projeto, pela função que formata a bibliografia. A montagem do relatório, por conseguinte, entrega um documento completo, rastreável e sempre coerente com os dados.

CONCLUSÃO — SÍNTESE

A síntese das seções e da montagem mostra que o relatório da EDA é um produto do pipeline, e não um texto escrito à parte. Cada seção calcula, desenha e redige a partir dos dados; os achados reais são afirmados onde a comparação é segura, e os limites são declarados onde ela não é; a simulação aparece sempre como ferramenta, nunca como evidência. Por isso, se os dados mudarem, as frases mudam junto, e o relatório nunca afirma algo que os dados deixaram de sustentar.

42. Referências e conferência no Crossref

INTRODUÇÃO — O TODO

As referências bibliográficas da Sprint 3 são tratadas por um módulo, `pipeline/referencias.py`**, e por um script,** `scripts/verificar_referencias.py`**, a partir de um catálogo em** `docs/sprint3/referencias.json`**.** A regra central desse conjunto é que nenhum metadado bibliográfico é escrito de memória: autores, revista, volume e páginas só entram numa referência depois de conferidos na base do Crossref, que registra os metadados oficiais de cada DOI. Essa regra existe porque uma referência escrita de memória pode trazer um autor errado, um ano trocado ou um DOI que aponta para outra obra, e um erro desse tipo compromete a credibilidade de todo o trabalho. O desenvolvimento a seguir analisa o catálogo, a formatação das entradas, a conferência no Crossref e o modo de verificação sem rede.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

O catálogo

O catálogo é o arquivo `referencias.json`**, em que cada entrada tem um identificador, um tipo, o título, o ano e uma descrição do uso da obra no projeto.** As entradas com DOI não guardam autores, porque os autores só devem entrar depois da conferência; as entradas sem DOI, que são livros e material didático, guardam autores e veículo, porque não há base oficial onde conferi-los, e são citadas por capítulo ou seção. Em 21/10/2026, o catálogo tinha 81 entradas, 75 delas com DOI, e nenhum DOI estava conferido. O catálogo, assim, é a fonte única das referências do projeto, e o que ele não sabe com certeza ele não registra.

A formatação das entradas

A formatação das entradas, feita pela função `formatar`**, escreve cada referência no estilo ABNT simplificado, conforme a situação da entrada.** Uma entrada sem DOI sai com os autores, o título e o veículo do catálogo. Uma entrada com DOI ainda não conferido sai apenas com o título, o ano e o DOI, acompanhada da marca "Conferência no Crossref pendente", para que o leitor saiba que os demais metadados ainda não foram verificados. Uma entrada conferida sai com os metadados do Crossref, com até três autores e a expressão "et al." quando houver mais. A função `lista` recusa, com erro, qualquer citação cujo identificador não esteja no catálogo, porque uma citação fora do catálogo seria, por definição, uma referência escrita de memória. A formatação das entradas, por conseguinte, mostra ao leitor exatamente o grau de verificação de cada referência.

A conferência no Crossref

A conferência no Crossref, feita pelo script, consulta a base do Crossref para cada DOI do catálogo e compara o título e o ano encontrados com os registrados. A consulta usa o mesmo cliente HTTP do pipeline, com as mesmas regras de cortesia e de segurança: robots.txt, TLS e intervalo mínimo de um segundo entre pedidos. O título confere se as palavras dos dois títulos forem parecidas o suficiente, segundo a semelhança de Jaccard, que divide o número de palavras em comum pelo número total de palavras distintas, com limite de 0,6; essa medida tolera subtítulos, pontuação e diferenças de maiúsculas, e as palavras com até duas letras são ignoradas, porque artigos e preposições não distinguem um título de outro. O ano confere com diferença de até um ano, porque a publicação on-line e a impressa de um mesmo artigo costumam ter anos diferentes. O resultado de cada conferência, com a data e os metadados do Crossref, vai para o arquivo `referencias_verificadas.json`, e a lista formatada vai para `REFERENCIAS.md`. A conferência no Crossref, assim, substitui a memória por uma verificação contra a fonte oficial.

A autorização da conferência

A conferência no Crossref depende de autorização explícita da equipe, porque ela envia cada DOI do catálogo a um serviço externo. Embora um DOI não seja um dado sensível, a regra do projeto é que nenhuma comunicação externa não pedida aconteça por iniciativa do pipeline ou do assistente. Até a autorização, as referências permanecem marcadas como pendentes, e o gate G13 permanece reprovado. A autorização da conferência, por conseguinte, mantém nas mãos da equipe a decisão sobre qualquer contato com serviços de terceiros.

O modo sem rede

O modo sem rede, ativado pela opção `--offline`, não acessa a internet e apenas verifica se a conferência já feita cobre todos os DOIs do catálogo. Além de exigir que cada DOI tenha sido conferido, esse modo exige que o DOI guardado na conferência seja igual ao DOI atual do catálogo, porque, se alguém corrigir um DOI no catálogo, a conferência anterior deixa de valer para ele. É esse modo que o gate G13 usa, para que a rodada dos gates não dependa da internet nem envie dados a terceiros. O script devolve o código de saída 1 se algum DOI não confere ou não foi conferido. O modo sem rede, assim, permite verificar a bibliografia em qualquer máquina, sem nova comunicação externa.

CONCLUSÃO — SÍNTESE

A síntese das referências mostra que a bibliografia segue a mesma regra que governa o resto do projeto: o que não foi conferido aparece como pendente, e não como certo. O catálogo registra apenas o que se sabe, a formatação mostra o grau de verificação de cada entrada, a conferência compara com a fonte oficial, e o modo sem rede permite verificar sem novo contato externo. Por isso, a bibliografia da Sprint 3 nunca apresenta como verificado um metadado que ninguém verificou.

Parte III — Verificação e resultados

43. Testes e mini-Palmas

INTRODUÇÃO — O TODO

Os testes automatizados são o conjunto de programas que verificam, sem intervenção humana, se cada regra do pipeline faz o que deve. Os testes existem porque o pipeline tem centenas de regras, e conferir cada uma à mão depois de cada mudança seria impossível; uma mudança aparentemente inofensiva numa regra de limpeza pode quebrar um cálculo três etapas adiante, e só um teste automático percebe isso a tempo. Na rodada dos quality gates de 21/10/2026, eram 749 testes. O desenvolvimento a seguir analisa os três tipos de teste, os testes da coleta, a cidade em miniatura chamada mini-Palmas e os testes que provocam falhas de propósito.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

Os testes unitários

Os testes unitários verificam uma função ou um componente de cada vez, sem rede e sem banco de dados. Eram 689 na rodada dos gates. Eles exercitam as funções puras, como a classificação das receitas, o cálculo do índice sazonal ou a decisão de espera do cliente HTTP, e os componentes cujas dependências foram substituídas, como o cliente HTTP com um transporte simulado. Os testes unitários dispensam a rede e o banco porque um teste que depende de um servidor externo falha quando o servidor está fora do ar, e então o resultado do teste deixa de dizer alguma coisa sobre o código. Os testes unitários, assim, dão uma resposta rápida e confiável sobre cada regra isolada.

Os testes de integração

Os testes de integração verificam o funcionamento conjunto do pipeline com o MySQL. Eram 45 na rodada dos gates, e ficam em arquivos cujo nome começa por `test_banco`. Esses testes existem porque algumas propriedades só podem ser verificadas com o banco real: se as restrições recusam uma carga errada, se as views chegam aos mesmos valores que o Python, se uma transação desfeita deixa o banco como estava. Os testes usam um banco de teste separado, para que a sua execução nunca altere os dados de trabalho. Os testes de integração, por conseguinte, verificam o que nenhum teste unitário consegue verificar: o contrato entre o código e o banco.

Os testes de aceitação

Os testes de aceitação descrevem comportamentos do sistema em frases escritas na linguagem Gherkin, no formato "dado, quando, então". Eram 15 na rodada dos gates. Uma frase desse tipo poderia dizer, por exemplo, que, dado um bruto adulterado, quando a transformação é executada, então ela é recusada antes da leitura. Esse formato existe porque permite que pessoas que não programam leiam e confiram o comportamento esperado, e liga cada teste a um critério de aceitação de uma história de usuário. Os testes de aceitação, assim, traduzem os requisitos em verificações executáveis.

Os testes da coleta

Os testes da coleta verificam o cliente HTTP e os adaptadores das fontes usando respostas HTTP gravadas, por meio do módulo `tests/apoio_http.py`**, que simula os servidores.** Nenhum teste da coleta acessa a internet. Os testes exercitam cada regra do cliente separadamente: a exigência de TLS, a leitura do robots.txt, o ritmo por servidor, a repetição das falhas passageiras, o cabeçalho Retry-After, o limite de taxa e o envio de credenciais apenas ao host certo. Há também testes para cada adaptador de fonte, com respostas no formato real de cada servidor. Os testes da coleta usam respostas gravadas porque só assim é possível provocar, de forma controlada, situações raras como um erro 503 seguido de um Retry-After, que um servidor real quase nunca produz quando se quer. Os testes da coleta, por conseguinte, verificam o comportamento do cliente diante de situações que a rede real não oferece sob encomenda.

A mini-Palmas

A mini-Palmas é um bruto pequeno e coerente das sete fontes, montado pelo módulo `tests/mini_palmas.py`**, em que cada fonte vem no formato real do seu servidor.** A cidade em miniatura tem sete setores urbanos dispostos numa grade, um deles sob sigilo do IBGE, e nove quadras, uma delas na divisa entre dois setores e outra fora de todos os setores. Esses detalhes foram escolhidos de propósito, porque cada um exercita uma regra: o setor sob sigilo exercita a regra da ausência, a quadra na divisa exercita o cruzamento com duas frações, e a quadra fora dos setores exercita a cobertura incompleta. A receita da mini-Palmas cobre 2016 e 2017 na API e inclui 2018, que a regra D-REC-05 exclui; o NUCLEOGOV usa a amostra real de dez linhas da Sprint 2. A simulação é reduzida a 600 imóveis em dois exercícios, para rodar em segundos. A mini-Palmas, assim, permite executar o pipeline inteiro, da leitura do bruto à gravação, em segundos e com resultados conhecidos.

Os testes que provocam falhas

Os testes que provocam falhas verificam que as proteções do pipeline reagem quando devem. Uma rejeição em 14 registros, proporção acima de 1%, interrompe a execução com a regra V03; um Censo sem a coluna V06004 interrompe com a V05; um setor do Censo fora da malha interrompe com a V08; um bruto adulterado é recusado antes da leitura; e os códigos de saída 2 e 3 são conferidos. Nos testes de integração, uma queda provocada entre o apagamento e a inserção dos indicadores deixa o banco exatamente como estava, e uma execução reprovada aparece no banco como FALHA, com a regra que a reprovou. Esses testes existem porque uma validação que nunca foi vista reprovando pode ser decorativa, isto é, pode estar escrita de modo que nunca reprova, e só um teste que a faz reprovar prova que ela funciona. Os testes que provocam falhas, por conseguinte, demonstram que cada proteção funciona exatamente quando é necessária.

CONCLUSÃO — SÍNTESE

A síntese dos testes revela que cada tipo de teste verifica uma camada diferente do pipeline: os unitários verificam as regras, os de integração verificam o contrato com o banco, os de aceitação verificam os requisitos, e os testes de falha verificam as proteções. A mini-Palmas une todas essas camadas numa execução completa e rápida, com uma cidade desenhada para exercitar cada caso difícil. Por isso, o pipeline chegou à execução real com cada regra já demonstrada, e os defeitos que a execução real revelou foram justamente os que nenhum dado montado à mão poderia prever.

44. Quality gates, mutação e integração contínua

INTRODUÇÃO — O TODO

Os quality gates são catorze verificações automáticas, executadas pelo script `scripts/quality_gate.py`, cada uma com um limite declarado. Um único gate reprovado reprova a rodada inteira. Os gates existem porque a qualidade do código, sem um critério objetivo, vira uma questão de opinião; com um limite numérico, a pergunta "o código está bom?" passa a ter uma resposta que qualquer pessoa pode verificar. Os gates G1 a G10 vêm da Sprint 2, e os gates G11 a G14 foram criados na Sprint 3. O desenvolvimento a seguir apresenta a tabela dos gates, analisa como cada grupo de gates mede e, depois, trata da mutação e da integração contínua.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A tabela dos gates

A tabela abaixo lista os catorze gates com a verificação que cada um faz e o limite que precisa ser atingido. Os limites só mudam com uma justificativa registrada, porque um limite que pode ser baixado sempre que um gate reprova deixa de proteger alguma coisa.

Gate	Verificação	Limite
G1	testes unitários	100% passando, nenhum ignorado
G2	testes de integração com o MySQL	100% passando, nenhum ignorado
G3	testes de aceitação em Gherkin	100% passando, nenhum ignorado
G4	cobertura de linhas e ramos	total ≥ 90%; cada módulo ≥ 80%
G5	complexidade ciclomática	≤ 10 por função
G6	índice de manutenibilidade	≥ 20 por módulo
G7	tamanho	módulo ≤ 250 linhas de código; função ≤ 50 linhas
G8	contratos de dependência (import-linter)	todos mantidos
G9	verificação de tipos (Pyright)	zero erros
G10	testes de mutação	≥ 80% de mutantes mortos em cada módulo
G11	desempenho das consultas do painel	P95 ≤ 3 s por consulta
G12	qualidade da última execução	100% das impeditivas; banco igual aos arquivos
G13	referências	todos os DOIs conferidos no Crossref
G14	reprodutibilidade	mesmos bytes em duas transformações

Os gates dos testes e da cobertura

Os gates G1 a G4 executam o conjunto de testes uma única vez e leem dois relatórios produzidos por essa execução. O relatório em XML separa os três tipos de teste, o que permite exigir, para cada tipo, que todos passem e que nenhum seja ignorado; a exigência de nenhum teste ignorado existe porque um teste marcado para ser pulado é, na prática, um teste que não existe. O relatório de cobertura, em JSON, informa quantas linhas e quantos ramos de decisão do código foram executados pelos testes, e o gate exige 90% no total e 80% em cada módulo; o limite por módulo existe porque uma média alta pode esconder um módulo inteiro sem teste. Na rodada de 21/10/2026, a cobertura foi de 99,3%. Os gates dos testes e da cobertura, assim, verificam que os testes passam e que alcançam quase todo o código.

Os gates da estrutura do código

Os gates G5 a G7 medem a estrutura do código com a ferramenta radon. O gate G5 limita a complexidade ciclomática a 10 por função, uma medida do número de caminhos independentes dentro de uma função; uma função com muitos caminhos é difícil de entender e de testar por inteiro. O gate G6 exige um índice de manutenibilidade de pelo menos 20 por módulo, um número que combina tamanho, complexidade e volume de operações. O gate G7 limita cada módulo a 250 linhas de código e cada função a 50 linhas, e, para medir o tamanho de cada função, percorre a árvore sintática do código, porque só ela sabe onde cada função começa e termina. Esses limites existem porque o código é lido muito mais vezes do que é escrito, e funções curtas e simples são as que uma pessoa nova na equipe consegue entender. Os gates da estrutura, por conseguinte, mantêm o código num tamanho e numa complexidade que permitem a manutenção.

Os gates das dependências e dos tipos

Os gates G8 e G9 verificam propriedades que o Python não verifica sozinho. O gate G8 executa o `lint-imports` e confere os nove contratos de dependência, contando os mantidos e os quebrados; é esse gate que transforma a arquitetura de pipes e filtros de intenção em regra. O gate G9 executa o Pyright, um verificador de tipos, com as bibliotecas instaladas no mesmo Python que executa o pipeline, e exige zero erros. A verificação de tipos existe porque o Python só descobre que uma função recebeu um tipo errado quando essa linha é executada, às vezes em produção; o verificador descobre o erro antes, lendo o código. Os gates das dependências e dos tipos, assim, encontram por leitura erros que, de outro modo, só apareceriam na execução.

O gate do desempenho

O gate G11 mede o tempo de quatro consultas do painel no banco carregado: o mapa por setor, as faixas de valor, a receita real anual e os indicadores. Cada consulta é executada 31 vezes; a primeira execução é descartada, porque serve para aquecer o cache do banco, e o gate calcula o percentil 95 das outras 30, isto é, o tempo abaixo do qual ficam 95% das execuções. O percentil 95 é usado, e não a média, porque a experiência de quem usa o painel é marcada pelas consultas lentas, e uma média baixa pode esconder picos. O gate também declara as condições da medição: o volume de créditos, a concorrência de uma consulta por vez, o cache aquecido, o banco local sem latência de rede e a máquina usada. Essa declaração existe porque um tempo medido sem as suas condições não pode ser comparado com nenhum outro. Na rodada de referência, o pior percentil 95 foi de 1,38 s, no mapa por setor, abaixo do limite de 3 s. O gate do desempenho, por conseguinte, transforma o requisito de rapidez do painel num número medido e qualificado.

O gate da qualidade da execução

O gate G12 lê o relatório de qualidade da última execução e exige duas coisas. A primeira é que todas as validações impeditivas tenham passado. A segunda é que o banco seja igual aos arquivos, crédito a crédito: o gate lê, da view `v_base_analitica_credito`, seis colunas de cada crédito, o identificador, o tempo de inadimplência, a faixa de valor, o mês do vencimento, o número de inadimplências anteriores e o indicador de inadimplência, e compara cada uma com a coluna correspondente da base analítica gravada em arquivo. Essa comparação existe porque o banco e os arquivos são calculados por caminhos diferentes, o SQL das views e o Python dos filtros, e só a igualdade entre os dois prova que ambos implementam a mesma regra. O gate da qualidade, assim, confere não só que a execução passou nas validações, mas também que os dois lados do pipeline concordam.

Os gates das referências e da reprodutibilidade

Os gates G13 e G14 verificam a bibliografia e a reprodutibilidade. O gate G13 executa a conferência das referências no modo sem rede e só passa se todos os DOIs do catálogo já tiverem sido conferidos; em 21/10/2026, ele estava reprovado, porque a conferência ainda aguardava autorização. O gate G14 transforma o mesmo bruto duas vezes, cada vez numa pasta temporária nova, calcula o SHA-256 de todos os arquivos produzidos e compara as duas rodadas. Quando não há bruto real, como na integração contínua, o gate usa a mini-Palmas e diz no resultado qual bruto foi usado. Na rodada de referência, os 17 arquivos foram idênticos nas duas transformações. Os gates das referências e da reprodutibilidade, por conseguinte, verificam que nada do que o projeto afirma depende de memória ou de acaso.

A mutação

A mutação é a técnica que mede a força dos testes, e é executada pelo script `scripts/mutacao.py` **com a ferramenta cosmic-ray.** A ferramenta altera o código de propósito, criando versões defeituosas chamadas mutantes, como trocar um sinal de "menor que" por "menor ou igual a", e executa os testes contra cada mutante. Um bom conjunto de testes falha diante de um mutante, o que se chama "matar" o mutante; um mutante que sobrevive revela uma alteração que nenhum teste percebeu. A mutação existe porque a cobertura mede apenas quais linhas foram executadas, e não se os testes conferem os resultados dessas linhas; um teste que executa uma linha sem verificar o resultado aumenta a cobertura e não protege nada. O gate G10 exige que pelo menos 80% dos mutantes de cada módulo sejam mortos. A mutação, assim, verifica a qualidade dos próprios testes.

A execução isolada da mutação

A execução da mutação acontece numa cópia isolada do repositório, e não na pasta de trabalho, porque a ferramenta altera o código-fonte para injetar cada mutante; se a execução fosse interrompida na pasta de trabalho, um mutante poderia ficar no código. A cópia exclui os dados, os logs e o arquivo `.env`, porque os testes não precisam deles e porque copiar credenciais para pastas temporárias multiplicaria os lugares onde elas podem vazar. Sete módulos da Sprint 3 passam por mutação: a política de coleta, a limpeza da API, a limpeza do Censo e do IPCA, o enriquecimento da receita, os atributos, as validações e as estatísticas da EDA. Esses módulos foram escolhidos porque contêm as regras que decidem o que entra, o que sai e o que é calculado. Na última rodada, somados os módulos das Sprints 2 e 3, a mutação matou 93,7% dos mutantes, e todos os módulos ficaram acima de 80%. A execução isolada, por conseguinte, mede a força dos testes sem arriscar o código de trabalho.

A integração contínua

A integração contínua é a execução automática dos gates num servidor, definida no arquivo `.github/workflows/qualidade.yml` **para o serviço GitHub Actions.** A cada envio ao GitHub, o servidor executa os gates sem a mutação, porque a mutação é lenta; a mutação roda toda segunda-feira e sob demanda. O servidor sobe um MySQL 8.4 descartável, usa o Python 3.12 e publica os relatórios como artefatos da execução. A integração contínua existe porque garante que os gates sejam executados num ambiente limpo, igual para todos, e não apenas na máquina de quem fez a mudança. Os gates, porém, nunca foram executados no Python 3.12 localmente, porque o ambiente de desenvolvimento usa o Python 3.14; essa diferença é a pendência P5. A integração contínua, assim, leva a verificação para um ambiente neutro, com uma diferença de versão que ainda precisa

ser conferida.

CONCLUSÃO — SÍNTESE

A síntese dos gates mostra que eles transformam a qualidade em critério objetivo, verificando camadas cada vez mais profundas do trabalho. Os gates dos testes verificam que o código faz o que se espera; os da estrutura verificam que ele pode ser mantido; os das dependências e dos tipos verificam a arquitetura e a coerência; a mutação verifica os próprios testes; e os gates da Sprint 3 verificam o desempenho, a concordância entre banco e arquivos, a bibliografia e a reprodutibilidade. Na rodada de 21/10/2026, treze dos catorze gates passaram, e o único reprovado, o G13, depende de uma autorização, e não de código.

45. Execução de referência

INTRODUÇÃO — O TODO

A execução de referência é a execução real do pipeline completo que serve de evidência para os números deste relatório e dos demais documentos da Sprint 3. Ela começou às 6h57 de 21/10/2026, recebeu o identificador `20261021T065740-9b9054` e rodou sobre o código do commit `ea64aa6`. Uma execução de referência é necessária porque os testes provam que as regras funcionam com dados montados, mas só uma execução real prova que o pipeline funciona com os dados verdadeiros, com todas as suas irregularidades. O desenvolvimento a seguir analisa as etapas e as suas durações, as requisições, as contagens da limpeza, o enriquecimento, a simulação, os achados com dado real e uma ressalva sobre a versão do código.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

As etapas

A tabela abaixo resume as quatro etapas da execução de referência. O percurso inteiro levou cerca de três minutos e meio, e o código de saída foi 0, que indica sucesso. A etapa mais longa foi a carga, porque grava no banco os 556.260 créditos simulados e a extração NUCLEOGOV completa.

Etapa	Duração	Resultado
extract	56,5 s	7 fontes completas; 61 requisições; nenhuma falha
transform	11,7 s	33 de 33 validações impeditivas aprovadas
load	140,3 s	556.260 créditos simulados; NUCLEOGOV carregado como snapshot
EDA	4,4 s	relatório, galeria e 7 figuras

As requisições

As 61 requisições da coleta se distribuem pelas fontes de uma forma que confirma o comportamento do cliente. Foram 6 consultas de robots.txt, uma para cada host; 36 páginas da API da Prefeitura; 1 tentativa ao portal NUCLEOGOV; 7 pedidos ao GeoPalmas, um para as regiões e um para cada uma das seis regiões; 4 páginas do WFS do IBGE; 4 pedidos ao servidor do IBGE para os arquivos do Censo, dois índices e dois arquivos; e 3 janelas do IPCA. Cinco hosts responderam 404 ao robots.txt, o que significa que não publicam regras, e o portal NUCLEOGOV respondeu 403. Os dois arquivos do Censo responderam 304, porque não tinham mudado desde a coleta anterior, e foram reaproveitados sem novo download. As requisições, assim, mostram na prática o robots.txt consultado uma vez por host, a recusa registrada e o download condicional funcionando.

As contagens da limpeza

As contagens da limpeza mostram fontes em bom estado e regras que não rejeitam dados bons. A API teve 3.542 registros lidos e nenhum rejeitado; o NUCLEOGOV teve 176.993 linhas lidas e nenhuma rejeitada; o GeoPalmas teve 284 quadras e uma rejeitada, a "Arse 152", por um anel sem pontos suficientes. Os setores foram 733, sem rejeição; o Censo teve 683 setores de Palmas, sem rejeição; e o IPCA teve 321 meses, sem lacuna. As contagens da limpeza, por conseguinte, confirmam que as regras de rejeição agem apenas sobre o que é de fato defeituoso.

O enriquecimento

O enriquecimento produziu a receita unificada e o cruzamento territorial. A receita unificada teve 2.055 linhas; a API contribuiu com 2.849 linhas tributárias, das quais 86, as de 2018, ficaram fora da série, e o NUCLEOGOV contribuiu com 1.092 componentes. O cruzamento formou 1.635 pares entre 733 setores e 283 quadras, e cerca de 94,2% dos domicílios ficaram em setores com quadra do GeoPalmas; os 5,8% restantes estão em setores que não tocam nenhuma quadra do GeoPalmas. Vinte e três setores estavam sob sigilo do IBGE. O enriquecimento, assim, levou a receita à mesma moeda e a renda ao território com perdas medidas e declaradas.

A simulação

A simulação gerou 556.260 créditos para 111.252 imóveis, um crédito por imóvel em cada um dos cinco exercícios. O intercepto calibrado foi −1,259, e a taxa média de inadimplência ficou em 33,00%, igual ao alvo. O estoque simulado de dívida ativa na data de corte foi de cerca de R$ 79,9 milhões. As situações no corte se distribuíram assim: 372.694 créditos pagos em dia, 82.463 pagos em atraso, 22.981 recuperados da dívida ativa e 78.122 em dívida ativa; nenhum crédito ficou como vencido e não inscrito. A simulação, por conseguinte, atingiu a taxa-alvo e produziu uma distribuição de situações coerente com os parâmetros.

Os achados com dado real

Os achados com dado real são os únicos resultados desta sprint que dizem algo sobre Palmas, porque vêm de fontes observadas, e não da simulação. A receita real mostrou uma mudança de composição: entre 2019 e 2025, na mesma fonte, o ISSQN cresceu 9,5% ao ano acima da inflação, enquanto o IPTU recuou 2,9% ao ano. A dívida ativa pesou muito no IPTU, com 47,7% do imposto arrecadado em 2019 e 15,7% em 2025. O IPTU concentra a arrecadação em março, com um índice de 3,53 vezes a média mensal, enquanto o ISSQN e o ITBI ficam entre 0,82 e 1,26 da média. A renda dos setores é desigual e agrupada no espaço: metade dos 637 setores urbanos com renda tem renda média do responsável abaixo de R$ 3.129, e a renda forma blocos contíguos, com I de Moran de 0,76. Os achados com dado real, assim, já sustentam perguntas para a Sprint 4, como a dependência crescente do ISSQN e a concentração do IPTU em março.

A ressalva sobre a versão do código

A ressalva sobre a versão do código trata de uma imprecisão nos documentos de evidência. Os quality gates rodaram às 7h07, depois de dois commits posteriores à execução de referência: o `fd739bf`, que mudou o fechamento do registro em caso de falha não prevista, e o `34524eb`, que mudou apenas o script dos gates. O caminho alterado pelo `fd739bf` não é percorrido numa execução bem-sucedida, de modo que os resultados da execução de referência não são afetados. Mesmo assim, a frase dos documentos de evidência segundo a qual os gates rodaram na "mesma versão do código" da execução é imprecisa, e a sua correção é a pendência P8. A ressalva

sobre a versão, por conseguinte, mantém a exatidão do registro mesmo num detalhe que não altera os resultados.

CONCLUSÃO — SÍNTESE

A síntese da execução de referência mostra que o pipeline funciona de ponta a ponta com dado real, e que cada um dos seus mecanismos foi observado em ação: o robots.txt, a recusa registrada, o download condicional, as validações, a calibração e a carga. Os achados reais e os resultados simulados aparecem separados, e a única imprecisão encontrada nos registros está declarada. Cada número desta seção está no log da execução ou no relatório de qualidade, e por isso pode ser conferido por qualquer pessoa.

46. Proposta HU-04 fora do pipeline

INTRODUÇÃO — O TODO

A proposta HU-04 é um protótipo, guardado na pasta `propostas/leitor_identidades/`, para a história de usuário HU-04, que pede que documentos enviados pelo contribuinte sejam transformados em registros que um servidor possa revisar. O protótipo foi mantido fora do pipeline por decisão da equipe, depois de uma avaliação crítica. Essa decisão merece registro porque um código útil pode, ainda assim, não estar pronto para ser integrado, e a razão da recusa precisa ficar clara para quem retomar o trabalho. O desenvolvimento a seguir analisa a origem do protótipo, o que dele pode ser aproveitado e o que impede a sua integração.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A origem

A origem do protótipo é um pacote fornecido por um integrante da equipe, e a sua procedência foi registrada com o SHA-256 do arquivo recebido. O código foi copiado sem alteração, para que a avaliação recaísse sobre o que foi de fato entregue, e não sobre uma versão modificada. O registro da procedência existe porque um código vindo de fora do repositório precisa ter a origem rastreável, como qualquer outro dado do projeto. A origem do protótipo, assim, está documentada da mesma forma que as fontes de dados.

O que pode ser aproveitado

O que pode ser aproveitado do protótipo são duas peças. A primeira é a validação dos dígitos verificadores do CPF, que é determinística, não depende de banco e mantém o número como texto, preservando os zeros à esquerda. A segunda é o registro de evidência: cada candidato extraído de um documento guarda a página, a posição e o trecho de onde veio, de modo que a decisão do servidor que revisa se apoia na fonte, e não apenas no valor extraído. Essas duas peças são úteis porque resolvem problemas que a HU-04 terá de qualquer forma. O que pode ser aproveitado, por conseguinte, é a lógica, e não a forma de armazenamento.

O que impede a integração

O que impede a integração do protótipo é, principalmente, a privacidade. O pacote grava o CPF em texto claro, e isso contraria a exigência do projeto de pseudonimizar os identificadores antes de qualquer armazenamento, que decorre da Lei Geral de Proteção de Dados. Além disso, o pacote usa um banco SQLite paralelo, que duplicaria entidades já existentes no MySQL da Sprint 2, e valida apenas o CPF, e não o CNPJ, embora contribuintes possam ser pessoas jurídicas. Integrar o

pacote como está introduziria no pipeline o primeiro dado pessoal sem proteção. O que impede a integração, assim, é uma combinação de risco legal e de duplicação de estrutura.

CONCLUSÃO — SÍNTESE

A síntese da proposta HU-04 mostra que o protótipo foi avaliado pelo mesmo critério que governa todo o projeto: nenhum dado pessoal entra sem proteção. As peças úteis foram identificadas, os impedimentos foram nomeados, e o protótipo foi isolado em vez de descartado. Por isso, a proposta só entrará no pipeline depois que a pseudonimização estiver implementada e o armazenamento for unificado com o banco existente.

47. Observações da releitura do código

INTRODUÇÃO — O TODO

As observações da releitura são os pontos que a leitura completa do código, feita para este relatório, encontrou e que merecem registro. Nenhuma delas invalida a execução de referência, e todas estão descritas para que a equipe decida o que fazer. Essas observações existem porque reler o código com o objetivo de explicá-lo, linha a linha, obriga a perguntar por que cada coisa é como é, e algumas perguntas não têm boa resposta. O desenvolvimento a seguir analisa sete observações, das que afetam o comportamento às que afetam apenas a documentação.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A espera depois da última tentativa

A primeira observação é que o cliente HTTP espera mesmo depois da última tentativa. No laço de tentativas, cada falha passageira chama o método de espera, que dorme e anuncia no log uma "nova tentativa"; mas, depois da quinta falha, o laço termina e o cliente desiste, de modo que a última espera não serve para nada. Com a base de 1 s, essa espera inútil pode chegar a 16 s, e o log anuncia uma "nova tentativa (5/5)" que nunca acontece, o que pode confundir quem investiga uma falha. O efeito foi confirmado nesta releitura, executando o cliente com um transporte simulado que sempre falha e com o sorteio fixado no valor máximo: as esperas registradas foram de 1, 2, 4, 8 e 16 s. A correção é simples: não esperar quando não há próxima tentativa. A espera depois da última tentativa, assim, é um desperdício de tempo e uma mensagem enganosa no log, ambos fáceis de eliminar.

A taxa simulada cresce ao longo dos exercícios

A segunda observação é que a taxa de inadimplência simulada, embora tenha média de 33%, não é constante ao longo dos exercícios. Ela vai de 24,4% em 2021 para 32,3% em 2022, 35,4% em 2023, 36,2% em 2024 e 36,7% em 2025. A causa é o efeito do histórico: no primeiro exercício, nenhum imóvel tem histórico de atraso, e o efeito é zero para todos; nos exercícios seguintes, os imóveis que atrasaram recebem o acréscimo do histórico, e a taxa sobe até se estabilizar. A calibração ajusta apenas a média dos cinco anos, e não a taxa de cada ano. Esse padrão é uma propriedade do modelo, e não de Palmas, e por isso precisa ser declarado ao lado de qualquer gráfico de tendência da simulação, para que uma inadimplência crescente não seja lida como piora da cobrança. Uma alternativa seria dar ao primeiro exercício um histórico sorteado com a taxa de equilíbrio. A taxa crescente, por conseguinte, é um efeito do desenho do modelo que deve ser conhecido antes de qualquer interpretação temporal.

Uma situação que não ocorre

A terceira observação é que a situação VENCIDO_NAO_INSCRITO nunca ocorre com os parâmetros atuais. O último vencimento simulado é 10/03/2025; somados os 365 dias até a inscrição e a folga máxima de 60 dias, a inscrição mais tardia ocorre em maio de 2026, antes do corte de 31/07/2026. Assim, todo crédito inadimplente que não pagou já está inscrito na data de corte, e a situação de vencido e ainda não inscrito fica vazia. A situação existe no código e no banco porque ocorreria com outros parâmetros, por exemplo com um corte mais próximo do último vencimento, e a tabela de situações da execução de referência confirma a sua ausência. Uma situação que não ocorre, assim, não é um defeito, mas é uma informação que o leitor da simulação precisa ter.

A abertura do registro fora do tratamento de falhas

A quarta observação é que a abertura do registro da execução no banco acontece antes do bloco que trata as falhas, na função `executar`**.** Se o banco estiver inacessível ou a senha estiver ausente do `.env`, e a execução não usar a opção `--sem-banco`, a conexão falha antes de qualquer etapa, e o erro não passa pelo tratamento que escreve "execução interrompida" no log. O processo termina com o código 1, o mesmo das demais falhas não previstas, mas a mensagem do erro vai apenas para o terminal, e o arquivo de log fica só com a linha de abertura. Essa observação foi obtida pela leitura do código, e não por uma execução, porque executar o pipeline contra o banco de trabalho teria efeitos que não foram pedidos. A abertura do registro fora do tratamento, por conseguinte, deixa um caso de falha com um registro menos completo que os demais.

Um comentário desatualizado

A quinta observação é um comentário desatualizado no arquivo da integração contínua, que afirma que o repositório usa só a branch `main`**, no modelo de desenvolvimento baseado no tronco.** A Sprint 3, porém, foi feita na branch `sprint-3`, por decisão da equipe, e o merge na `main` ainda depende de autorização. O comentário não altera o comportamento da integração contínua, mas descreve um processo que não é o seguido. Um comentário desatualizado, assim, precisa ser corrigido para que a documentação não contradiga a prática.

A versão do código na pasta sem Git

A sexta observação é que, na pasta `PI2_Sprint3`**, a versão do código registrada pelas execuções é "desconhecida", porque a pasta não tem o histórico Git que a função** `versao_codigo` **consulta.** A execução funciona normalmente, mas o log, o banco e o relatório de qualidade ficam sem o vínculo com um commit; o relatório de qualidade presente nessa pasta, por exemplo, vem de uma execução feita nela e registra a versão como "desconhecida". A versão do código na pasta sem Git, por conseguinte, é uma limitação conhecida da cópia de trabalho, e as evidências oficiais continuam sendo as do repositório com histórico.

A reexecução do esquema

A sétima observação é que reexecutar o arquivo `sprint3.sql` **apaga os dados das tabelas da Sprint 3, porque o arquivo foi escrito para recriar essas tabelas do zero.** O pipeline está protegido contra esse efeito, porque só executa o arquivo quando as tabelas não existem; uma pessoa que o execute à mão, porém, apagará a carga sem aviso. A reexecução do esquema, assim, é uma operação que precisa de um aviso explícito na documentação de quem administra o banco.

CONCLUSÃO — SÍNTESE

A síntese das observações mostra que nenhuma delas altera os resultados entregues, mas todas apontam para o mesmo tipo de cuidado: tornar visível o que o código faz em situações pouco comuns. A espera inútil e a abertura do registro afetam o diagnóstico de falhas; a taxa crescente e a situação ausente afetam a interpretação da simulação; e o comentário, a versão desconhecida e a reexecução do esquema afetam a documentação. Por isso, as observações funcionam como uma lista de ajustes pequenos e de premissas que precisam ficar escritas ao lado dos resultados.

48. Conclusão

INTRODUÇÃO — O TODO

A conclusão reúne o que a análise das partes deste relatório permitiu compreender sobre a Sprint 3 como um todo. A Sprint 3 entregou um pipeline completo, verificável e honesto, que leva o dado público de Palmas da fonte ao banco e à análise sem passos manuais escondidos. O desenvolvimento a seguir retoma, em síntese, cada grande etapa e identifica o padrão que as une.

DESENVOLVIMENTO — ANÁLISE DAS PARTES

A coleta

A coleta respeita as fontes e confere tudo o que recebe. O cliente HTTP lê o robots.txt, mantém o ritmo, repete só o que é passageiro e exige criptografia; o bruto é gravado sem jamais ser sobrescrito e encadeado por hashes; e a recusa do portal NUCLEOGOV foi respeitada e documentada, com a extração oficial aceita apenas por prova de identidade. A coleta, assim, combina cortesia com o servidor e desconfiança do que chega.

A transformação

A transformação só avança com dado aprovado. Trinta e três avaliações impeditivas protegem as camadas finais, todas avaliadas antes de qualquer interrupção e registradas com evidência numérica; as decisões de limpeza estão escritas, identificadas por código e medidas na execução real; e a ausência é tratada como informação, nunca como zero. A transformação, por conseguinte, entrega dados limpos cuja limpeza pode ser auditada regra a regra.

A carga

A carga deixa o banco igual aos arquivos. Cada conjunto de dados recebe a estratégia de gravação que corresponde à sua natureza, as restrições do banco repetem as regras do Python, e as views calculam os valores derivados de forma conferida contra o Python. A carga, assim, faz do banco uma segunda barreira de qualidade, e não apenas um depósito.

A análise exploratória

A análise exploratória separa achado de premissa. Os achados reais tratam da composição da receita, da dívida ativa, da sazonalidade e da renda; os padrões da inadimplência são declarados como propriedade da simulação e marcados como tal em cada gráfico. A análise exploratória, por conseguinte, diz o que os dados sustentam e declara o que eles não sustentam.

A conduta

A conduta seguiu regras explícitas. Nada foi enviado ao GitHub sem pedido, nenhuma credencial saiu do `.env`, nenhum dado pessoal entrou no pipeline, e os erros de registro foram corrigidos e declarados em vez de apagados. A conduta, assim, mostra que a disciplina técnica e a disciplina ética foram tratadas como partes do mesmo trabalho.

CONCLUSÃO — SÍNTESE

A síntese final revela o padrão que atravessa todas as partes da Sprint 3: em cada etapa, o pipeline substitui a confiança por uma verificação. A coleta verifica a origem, a transformação verifica a qualidade, a carga verifica a concordância com os arquivos, a análise verifica o alcance das conclusões, e a conduta verifica a autorização de cada ação com efeito externo. O próximo passo é, portanto, fechar a entrega e buscar o dado real: a base da Secretaria da Fazenda, pseudonimizada na entrada, substituirá a simulação, e as mesmas figuras, sem a marca SIMULADO, dirão se o gradiente de renda existe de fato em Palmas.

Referências

Os DOIs das obras abaixo ainda não foram conferidos no Crossref (pendência P1). Por isso, as entradas com DOI trazem só o título e o ano registrados no catálogo do projeto, sem autores escritos de memória. As entradas completas ficam em docs/sprint3/REFERENCIAS.md depois da conferência.

[alm2014] Detroit Property Tax Delinquency: Social Contract in Crisis. 2014. *DOI registrado no catálogo; conferência no Crossref pendente.*

[batini2009] Methodologies for data quality assessment and improvement. 2009. *DOI registrado no catálogo; conferência no Crossref pendente.*

[brewer2002] Evaluation of Methods for Classifying Epidemiological Data on Choropleth Maps in Series. 2002. *DOI registrado no catálogo; conferência no Crossref pendente.*

[cleveland1984] Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods. 1984. *DOI registrado no catálogo; conferência no Crossref pendente.*

[fotheringham1991] The Modifiable Areal Unit Problem in Multivariate Statistical Analysis. 1991. *DOI registrado no catálogo; conferência no Crossref pendente.*

[kaufman2012] Leakage in data mining: Formulation, detection, and avoidance. 2012. *DOI registrado no catálogo; conferência no Crossref pendente.*

[krotov2020] Tutorial: Legality and Ethics of Web Scraping. 2020. *DOI registrado no catálogo; conferência no Crossref pendente.*

[lam1983] Spatial Interpolation Methods: A Review. 1983. *DOI registrado no catálogo; conferência no Crossref pendente.*

[limpert2001] Log-normal Distributions across the Sciences: Keys and Clues. 2001. *DOI registrado no catálogo; conferência no Crossref pendente.*

[metcalfe1976] Ethernet: distributed packet switching for local computer networks. 1976. *DOI registrado no catálogo; conferência no Crossref pendente.*

[moran1950] Notes on Continuous Stochastic Phenomena. 1950. *DOI registrado no catálogo; conferência no Crossref pendente.*

[morris2019] Using simulation studies to evaluate statistical methods. 2019. *DOI registrado no catálogo; conferência no Crossref pendente.*

[parnas1972] On the criteria to be used in decomposing systems into modules. 1972. *DOI registrado no catálogo; conferência no Crossref pendente.*

[perry1992] Foundations for the study of software architecture. 1992. *DOI registrado no catálogo; conferência no Crossref pendente.*

[persons1919] Indices of Business Conditions. 1919. *DOI registrado no catálogo; conferência no Crossref pendente.*

[rfc6585] Additional HTTP Status Codes. 2012. *DOI registrado no catálogo; conferência no Crossref pendente.*

[rfc8996] Deprecating TLS 1.0 and TLS 1.1. 2021. *DOI registrado no catálogo; conferência no Crossref pendente.*

[rfc9110] HTTP Semantics. 2022. *DOI registrado no catálogo; conferência no Crossref pendente.*

[rfc9309] Robots Exclusion Protocol. 2022. *DOI registrado no catálogo; conferência no Crossref pendente.*

[robinson1950] Ecological Correlations and the Behavior of Individuals. 1950. *DOI registrado no catálogo; conferência no Crossref pendente.*

[sweeney2002] k-anonymity: a model for protecting privacy. 2002. *DOI registrado no catálogo; conferência no Crossref pendente.*

[wang1996] Beyond Accuracy: What Data Quality Means to Data Consumers. 1996. *DOI registrado no catálogo; conferência no Crossref pendente.*

[weissgerber2015] Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm. 2015. *DOI registrado no catálogo; conferência no Crossref pendente.*

KUROSE, James F.; ROSS, Keith W. *Redes de computadores e a Internet: uma abordagem top-down*. 6. ed. São Paulo: Pearson, 2013. Citado por seção.

VALENTE, Marco Tulio. *Engenharia de Software Moderna: princípios e práticas para desenvolvimento de software com produtividade*. Belo Horizonte: edição do autor, 2020. Citado por seção.

MATERIAL DIDÁTICO DO MÓDULO 5, EIXO II BIINAR/UFT. *ETL: apostila completa — do dado bruto à informação confiável*. Edição de outubro de 2026. Citado por seção.

BRASIL. Lei nº 5.172, de 25 de outubro de 1966 (Código Tributário Nacional), art. 161, §1º.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).

BRASIL. Decreto nº 9.772, de 26 de abril de 2019 (horário de verão).